

Universitatea POLITEHNICA din București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

**Arhitectură Agentică pentru Asistență Inteligentă în
Dezvoltarea Software**

Lucrare de licență

Prezentată ca cerință parțială pentru obținerea
titlului de *Inginer*
în domeniul *Calculatoare și Tehnologia Informației*
programul de studii *Ingineria Informației*

Conducător științific
Ș.L. dr. ing. Mihai DOGARIU

Absolvent
Eduard-Daniel NĂNESCU

Anul 2026

TEMA PROIECTULUI DE DIPLOMĂ
a studentului **Eduard-Daniel NĂNESCU, 441A**

1. Titlul temei: Arhitectură Agentică pentru Asistență Inteligentă în Dezvoltarea Software

2. Descrierea temei:

Contribuția originală a lucrării va consta în dezvoltarea unei arhitecturi software bazată pe Modele Mari de Limbaj (LLMs - Large Language Models), capabilă să asiste utilizatorul în scrierea, depanarea, documentarea și explicarea codului sursă. Aplicațiile vizate sunt eficientizarea procesului de dezvoltare software și automatizarea sarcinilor repetitive întâlnite în programare. Sistemul propus nu vizează antrenarea de la zero a modelelor, ci implementarea unei soluții agentice care utilizează modele pre-antrenate pentru a executa acțiuni complexe. Cercetarea aferentă va include: sinteza bibliografică a realizărilor actuale din domeniu, utilizarea unui mecanism care permite modelului să utilizeze instrumente pentru a interacționa cu mediul virtual, implementarea unor tehnici de inginerie a prompturilor și management-ul contextului pentru a asigura relevanța răspunsurilor, validarea experimentală a prototipului dezvoltat prin scenarii de utilizare reale utilizând medii de dezvoltare precum PyCharm/VS Code și biblioteci specifice precum LangChain, LangGraph sau GoogleADK, concluzionarea rezultatelor obținute și a contribuției originale, cât și enunțarea perspectivelor ulterioare de cercetare.

3. Contribuția originală:

Proiectarea și implementarea arhitecturii agentului, incluzând logica de selecție și utilizare a instrumentelor, gestionarea contextului și dezvoltarea unei interfețe sau integrarea cu interfețe existente care să permită interacțiunea facilă cu asistentul.

4. Resurse și bibliografie:

- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. "Attention is all you need." Advances in neural information processing systems 30 (2017).
- Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. "React: Synergizing reasoning and acting in language models." In The eleventh international conference on learning representations. 2022.
- Bozkurt, Aras, and Ramesh C. Sharma. "Generative AI and prompt engineering: The art of whispering to let the genie out of the algorithmic world." Asian Journal of Distance Education 18, no. 2 (2023): i-vii.
- Alto, Valentina. Building LLM Powered Applications: Create intelligent apps and agents with large language models. Packt Publishing Ltd, 2024.
- Zhu, He, Tianrui Qin, King Zhu, Heyuan Huang, Yeyi Guan, Jinxiang Xia, Yi Yao et al. "Oagents: An empirical study of building effective agents." arXiv preprint arXiv:2506.15741 (2025).

5. Data înregistrării temei: 2025-12-11 22:54:00

Conducător(i) lucrare,
Ș.L. dr. ing. Mihai DOGARIU

Student,
Eduard-Daniel NĂNESCU

Director departament,
Conf. dr. ing Bogdan FLOREA

Decan,
Prof. dr. ing. Mihnea UDREA

Cod Validare: **621d9dfde3**

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul *Arhitectură Agentică pentru Asistență Inteligentă în Dezvoltarea Software*, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității “Politehnica” din București ca cerință parțială pentru obținerea titlului de *Inginer* în domeniul Inginerie Electronică și Telecomunicații/ Calculatoare și Tehnologia Informației, programul de studii *Ingineria Informației* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate. Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare. Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, Iunie 2026.

Absolvent: Eduard-Daniel NĂNESCU

.....

Cuprins

LISTA FIGURILOR	2
LISTA TABELELOR	4
LISTA DE ACRONIME	5
INTRODUCERE	6
CONTEXT ȘI MOTIVAȚIE	6
SCOPUL ȘI OBIECTIVELE LUCRĂRII	7
1. LUCRĂRI ANTERIOARE	8
1.1. Evoluția către modelele mari de limbaj (Large Language Model (LLM))	8
1.2. Maturizarea modelelor mari de limbaj și cercetări actuale	13
1.3. Soluții agentice pentru industria dezvoltării software	19
1.3.0.1. Analiza preliminară a soluțiilor existente pe piață	24
1.4. Cerințe pentru o soluție agentică în dezvoltarea software	27
1.5. Limitări ale arhitecturilor agentice	27
1.6. Justificarea alegerilor de implementare	28
2. PLASAREA ÎN INDUSTRIE	31
2.1. Software open-source	31
2.2. Flux agentic ce rulează local	31
2.3. Observabilitate	32
2.4. Securitate și confidențialitate	32
2.5. Cost	32
2.6. Aplicații Open Source Software (OSS) concurente	33
3. IMPLEMENTARE	35
3.1. Mediul de dezvoltare	35
3.2. Structura	35
3.3. Configurare	36
3.4. Interfața cu utilizatorul	36
3.5. Instalare și utilizare	39
3.5.1. Cerințe preliminare	39
3.5.2. Instalare	39
3.5.3. Configurare la prima rulare	39
3.5.4. Utilizare	39
3.5.4.1. Modul headless	40
3.6. Gestionarea contextului și a memoriei	41
3.6.1. Memoria și istoricul conversației	41
3.6.2. Contextul	42
3.6.2.1. Retrieval Augmented Generation (RAG)	43

3.6.2.2.	Gestionarea datelor multi-modale	43
3.6.2.3.	Compactarea	44
3.6.2.4.	Tehnici pentru eficientizarea ferestrei de context	47
3.7.	Componente ale arhitecturilor agentice moderne	47
3.7.1.	Unelte	47
3.7.2.	Modul de planificare	49
3.7.3.	Skills	50
3.7.4.	Subagenți	51
3.8.	Extensibilitate	52
3.8.1.	Model Context Protocol (MCP) și adăugarea de unelte	52
3.8.2.	Adăugarea de subagenți	53
3.8.3.	Creare de skill-uri	54
3.8.4.	Suportul pentru furnizori de modele de limbaj	54
3.8.5.	Adăugarea tehnicilor de compactare	55
3.9.	Securitatea în mediul agentic	56
3.9.1.	Sandboxing	56
3.9.2.	Permisuni	57
3.9.3.	Guardrails, intenții periculoase și cereri în afara scopului (Out of Scope (OOS))	58
3.10.	Observabilitate	58
3.10.1.	Langfuse	58
3.10.2.	Panou web	60
4.	REZULTATE EXPERIMENTALE	62
4.1.	Poziționarea față de concurența	62
4.1.0.1.	Funcționalități prezente în concordanță cu aplicațiile concurente proprietare și Open Source Software (OSS):	65
4.1.0.2.	Funcționalități absente față de aplicațiile concurente:	65
4.2.	Influența lungimii și a exactității prompt-ului asupra performanței. De ce un prompt prea exact nu garantează rezultate mai bune	66
4.3.	Contextul în mediul agentic și compactare. Consecințele metodelor de compactare	69
4.3.1.	Stadii care taie date din context	69
4.3.2.	Stadii care parafrazează date din context	69
4.3.3.	Stadii auxiliare	69
4.3.4.	Secvența de stadii de compactare recomandată	70
4.4.	Interacțiunea dintre modele, context și unelte. Modele optime pentru sarcini de programare	70
4.4.1.	Rezultate	71
4.4.1.1.	Analiza rezultatelor	72
4.4.1.2.	Modelele open-source: evaluarea valorii și a potențialului de utilizare	73
5.	DEZVOLTĂRI VIITOARE	74
5.1.	Automatizări, integrări cu alte programe	74
5.2.	Integrarea cu mediile de dezvoltare	74
5.3.	Suport nativ pentru Windows	74
5.4.	Construirea unui graf de cunoștințe pentru optimizarea înțelegerii proiectelor	74

5.5. Suport pentru metode de compactare și sandboxing avansate și Software Development Kit (SDK) pentru agenți	75
6. CONCLUZII	76
BIBLIOGRAFIE	77

LISTA FIGURILOR

1.1. Modelul unui perceptron (sursa)	9
1.2. Convoluția cu un nucleu a unei imagini (sursa)	9
1.3. Recurrent Neural Network (RNN) și Long Short Term Memory (LSTM) com- parate (sursa)	10
1.4. Arhitectura transformer [1]	12
1.5. Evoluția recentă a modelelor mari de limbaj [2]	13
1.6. Evoluția modelelor Generative Pre-trained Transformer (GPT) [2]	13
1.7. Performanța tipurilor de arhitecturi de tipul "Longformer" [3]. În timp ce meca- nismul tipic de atenție utilizează memorie exponențial, arhitectura Longformer folosește memorie liniară odată cu creșterea contextului	14
1.8. Diferite configurații de atenție folosite în arhitectura Longformer în comparație cu mecanismul obișnuit de atenție [3]	14
1.9. Principiul de bază pentru Rotary Position Embedding [4]	15
1.10. Fine-tuning-ul se realizează doar pe A și B [5]	15
1.11. Rezultatele procesului de Simple Self Distillation [6]	16
1.12. Îmbunătățiri în sarcini de programare ale modelelor după ce au fost supuse pro- cesului de Simple Self Distillation[6]	17
1.13. Comparația procedurilor de fine-tuning: fine-tuning clasic vs LoRA vs QLoRA [7]	17
1.14. Ilustrarea procedurii de Reinforcement Learning (RL) [8]	18
1.15. Exemple de Chain-of-Thought (CoT) [9]	19
1.16. Diferența între procedee de gândire [10]	19
1.17. Tipar simplificat al buclei ReAct.	20
1.18. Paradigma ReAct [11]	20
1.19. Îmbunătățirile pe care le aduce ReAct [11]	20
1.20. Ideea din spatele ReWOO [12]	21
1.21. Comparația de performanță între paradigme pentru îmbunătățirea gândirii din punctul de vedere al acurateții vs. numărul de tokeni consumați [12]	21
1.22. Îmbunătățirea adusă de ReWOO este delegarea mai multor libertăți către worker ce poate apela mai multe unelte până să producă un răspuns. P = Plan, E = Evidence, A = Answer, O = Observation, A = Act, T = Thought [12]	22
1.23. Modele tind să aibă o acuratețe foarte slabă contra întrebărilor ce vizează informații care se afla la mijlocul ferestrei curente de context [13]	23
1.24. Vizualizarea problemei și al rezultatului dorit la întrebare. De multe ori, modelele întâmpina dificultăți în accesarea acestor informații ascunse în mijlocul ferestrei de context [13]	23
1.25. În majoritatea modelelor se observa o scădere a performanțelor după 32K tokeni pe sarcini de gândire complexă [14]	24
1.26. Clasamentul modelelor în SWE-Bench Pro [15] la momentul actual în funcție de performanța rezolvării sarcinilor complexe în limba engleză	26
1.27. Clasamentul modelelor în SWE-Bench Pro [15] la momentul actual în funcție de cost	27

1.28. Reprezentarea ferestrei de context într-o conversație uzuala (sursa)	28
2.1. Reprezentarea ferestrei de context într-o conversație în Claude Code	32
2.2. Reprezentarea utilizării în Claude (varianta web) (sursa)	32
3.1. Structura directoarelor vitale ale proiectului	36
3.2. Prima versiune a interfeței, realizata cu ajutorul librăriei Textual	37
3.3. Continuarea conversației cu prima versiune a interfeței, realizata cu ajutorul librăriei Textual	37
3.4. A doua versiune a interfeței, realizata fără librăria Textual	37
3.5. Continuarea conversației cu a doua versiune a interfeței, realizata fără librăria Textual	37
3.6. Împărțirea ferestrei de context (sursa)	42
3.7. Configurarea pipeline-ului de compactare a contextului	46
3.8. Implementarea unei funcții pe care agentul o folosește pentru a lista directoirile și fișierele la o anumita cale pe sistem	48
3.9. Configurarea gardării execuției uneltelor	49
3.10. Implementarea unei capabilitati de Pull Request (PR) review	50
3.11. Implementarea unei funcții pe care agentul o folosește pentru a porni un subagent	51
3.12. Configurarea subagenților	52
3.13. Înregistrarea unui nou subagent	53
3.14. Configurarea sandbox-ului Docker pentru execuția izolata a comenzilor	57
3.15. Cererea de aprobare a permisiunii de a scrie un fișier	58
3.16. Interfața Langfuse	59
3.17. Imagini cu interfața web a aplicației	61

LISTA TABELELOR

1.1.	Comparația câtorva din modelele existente și a capacitățile acestora în comparație cu prețul de utilizare	25
1.2.	Comparația framework-urilor pentru întrebuințări agentice	29
1.3.	Comparație biblioteci pentru interfața cu utilizatorul	30
2.2.	Top 6 asistenți open source după numărul de "stele" pe GitHub	33
2.1.	Produse Inteligență Artificială (IA) ale principalilor furnizori de modele de limbaj	34
3.1.	Hardware recomandat pentru modele open-source	35
3.2.	Comenzi disponibile în interfață	40
4.1.	Comparație între soluția dezvoltată și principalele unelte agentice de programare pentru terminal disponibile pe piață	63
4.2.	Comparație între soluția propusă și principalii asistenți de programare Open Source Software (OSS) din ecosistem	64
4.3.	Rezultatele pentru prompt-ul detaliat: metrici per proiect și pe nivele de severitate	67
4.4.	Rezultatele pentru prompt-ul simplu: metrici per proiect și pe nivele de severitate	68
4.5.	Modelele testate: tip, număr de parametri, mărime pe disc și suport multi-modal.	71
4.6.	Clasamentul modelelor după scor mediu, durata totală de rulare și cost. Modelele rulate local prin ollama au cost zero. Modelele proprietare nu au datele referitoare la mărimea modelului publice	71

LISTA DE ACRONIME

AML Adversarial Machine Learning	MoE Mixture of Experts
API Application Programming Interface	NLP Natural Language Processing
BLEU Bilingual Evaluation Understudy	OOS Out of Scope
BTT Backpropagation Through Time	OSS Open Source Software
CD Continuous Deployment	PDF Portable Document Format
CI Continuous Integration	POSIX Portable Operating System Interface
CLI Command Line Interface	PR Pull Request
CNN Convolutional Neural Network	RAG Retrieval Augmented Generation
CoT Chain-of-Thought	RAM Random Access Memory
CPU Central Processing Unit	RCA Root Cause Analysis
CSS Cascading Style Sheets	RL Reinforcement Learning
DOM Document Object Model	RNN Recurrent Neural Network
E2E End to End	ROUGE Recall-Oriented Understudy for Gisting Evaluation
FN False Negative	SDD Spec-Driven Development
FP False Positive	SDK Software Development Kit
GPT Generative Pre-trained Transformer	SHA Secure Hash Algorithms
GPU Graphics Processing Unit	SSE Server-Side Events
GUI Graphical User Interface	STDIO Standard Input-Output
HTML Hyper-Text Markup Language	STDOUT Standard Output
HTTP Hyper-Text Transfer Protocol	TDD Test-Driven Development
IA Inteligență Artificială	TN True Negative
IDE Integrated Development Environment	ToT Tree-of-Thought
JSON Java Script Object Notation	TP True Positive
KV Key-Value	TUI Terminal User Interface
LLM Large Language Model	URL Uniform Resource Locator
LSTM Long Short Term Memory	UUID Universally Unique Identifier
MCP Model Context Protocol	VRAM Video Random Access Memory
MHA Multi-Head Attention	YAML YAML Ain't Markup Language
ML Machine Learning	
MLOps Machine Learning Operations	

INTRODUCERE

CONTEXT ȘI MOTIVAȚIE

Motivația acestei lucrări izvorăște din experiența personală în industria dezvoltării software. Atribuțiile unui inginer sunt numeroase, diverse și adesea repetitive. Scrierea efectivă de cod a ajuns să fie doar o parte restrânsă din activitatea zilnică, uzual delegată colegilor juniori, pierzând teren în fața design-ului, a documentației, a review-urilor, a testelor unitare, de integrare și End to End (E2E), a instrumentării și monitorizării, a proceselor de Continuous Integration (CI)/Continuous Deployment (CD) și a administrării infrastructurii, iar atunci când apar probleme, în fața debugging-ului și a analizei cauzelor principale (Root Cause Analysis (RCA)).

Un inginer software care lucrează într-o companie ce deservește milioane de clienți din întreaga lume trebuie să fie competitiv, iar aceasta competitivitate se rezuma, printre altele, pe lângă proficiența în programare, la a fi la curent cu cele mai noi practici din domeniu, securitate cibernetică, rețelistică și infrastructura, dar să și retina simultan informații vitale despre arhitectura și sistemele interne ale companiei, totul pe lângă instruirea colegilor noi sau ședințe.

Timpul unui inginer este astfel împărțit de foarte multe procese dinamice care trebuiesc adresate de un angajat capabil să susțină acest ritm alert.

Având în vedere cele de mai sus, cât și evoluția recentă a sistemelor bazate pe Inteligență Artificială (IA) ce folosesc un Large Language Model (LLM) pentru înțelegerea și generarea de text, era natural ca direcția în care dezvoltarea software urma să se schimbe către un proces în care multe din sarcinile pe care un inginer le-ar fi avut sunt acum delegate sistemelor automate. Astfel, atenția inginerului se muta de la a scrie cod, la supravegherea instrumentelor de programare bazate pe inteligența artificială și asumarea deciziilor arhitecturale. Restul sarcinilor triviale pot fi astfel abordate de aceste instrumente, optimizând setul de responsabilități pe care un profesionist le avea până acum.

Trecerea de la o experiență pur conversațională cu aceste modele la întrebuințări pentru căutare prin limbaj natural, iar în final integrându-se cu mediile de dezvoltare software și cu terminalul sistemului de operare pentru a avea acces la un context înlesnit în date factuale, dar și pentru execuția de comenzi nu a fost una dificilă, având în vedere competențele avansate de prelucrare, înțelegere și generare de text ale acestor modele.

Momentan, în domeniul modelelor de limbaj, există câțiva furnizori mari precum Google, Anthropic și OpenAI care controlează o bună parte din cota de piață. Aceștia au ajuns în stadiul în care își fortifica ecosistemul pentru a maximiza retenția utilizatorilor. Fiecare distribuitor de astfel de servicii lansează produse care se folosesc de aceste modele. De la asistenți de cercetare, la asistenți de programare și la programe ce automatizează sarcini zilnice. Odată cu aceste produse, încearcă să restricționeze accesul la platformele lor sau să ofere tarife crescute pentru utilizatori ce folosesc modelele pentru astfel de sarcini în afara ecosistemului lor.

Există și modele de limbaj de tipul Open Source Software (OSS), de diferite mărimi și cu diverse proprietăți și specializări, dar ecosistemul uneltelor gratuite sau OSS care folosesc aceste modele pentru sarcini avansate încă este în proces de dezvoltare. Astfel, este un mediu propice pentru un produs ce livrează un asistent inteligent pentru dezvoltarea software, independent de furnizor, construit pentru costuri reduse, performanță ridicată și abordarea problemelor complexe ce poate rula local, precum cel pe care imi propun să îl realizez în aceasta lucrare.

SCOPUL ȘI OBIECTIVELE LUCRĂRII

Scopul acestei lucrări este de a realiza un **asistent inteligent extensibil și transparent** bazat pe IA, **agnostic față de platforma sau de furnizori de modele de limbaj**, cu modele ce rulează local sau în cloud și care poate interacționa cu mediul virtual în care rulează (sistem de operare, mediu de dezvoltare, internet) prin intermediul uneltelor (tool-uri ¹), realizat de un workflow agentic ²

Obiectivul principal care trebuie atins în cadrul acestei lucrări este realizarea unui produs funcțional ce poate ajuta un inginer software în cat mai multe aspecte ale meseriei sale. Pentru a îndeplini acest obiectiv, trebuie să avem în vedere următoarele necesități: interfață versatila ce poate fi folosita oriunde prin terminalul sistemului de operare (Terminal User Interface (TUI)), suficiente unelte pentru a îndeplini sarcini de o complexitate ridicata, management-ul conversației, contextului și al memoriei, monitorizare și observabilitate (metrici) care oferă transparența în utilizarea produsului, al modelelor dar și din punctul de vedere al costurilor de operare și siguranță prin medii sigure de execuție a codului și a comenzilor de terminal.

¹În IA folosita cu ajutorul agenților **tool**-urile sunt funcții cu descrieri detaliate de folosire pe care agentul le populează cu parametrii, le executa și recepționează răspunsul pentru a-l folosi mai departe în rezolvarea problemelor complexe

²**Agenții** sunt aplicații ce folosesc inteligența artificială pentru a interacționa cu mediul înconjurător

Capitolul 1

LUCRĂRI ANTERIOARE

O buna înțelegere a tehnologiilor, standardelor și a obstacolelor actuale din industrie necesită studierea literaturii de specialitate, a lucrărilor și arhitecturilor existente deja în domeniu. Acest pas este obligatoriu pentru alegeri informate pentru proiectarea și implementarea proiectului evitând problemele consacrate și conferă o vedere de ansamblu asupra stadiului final al lucrării.

1.1 Evoluția către modelele mari de limbaj (LLM)

Domeniul inteligenței artificiale a fost studiat pentru o perioadă îndelungată de timp iar câteva publicații au marcat industria prin concepte revoluționare și progrese importante.

Unul din conceptele de bază ale inteligenței artificiale a apărut pentru prima dată publicat în anul 1958 de către F. Rosenblatt în lucrarea "The perceptron: A probabilistic model for information storage and organization in the brain." [16], unde este prezentat **perceptronul**: un model inspirat din fiziologia neuronului biologic, capabil să realizeze sarcini de clasificare binară. Perceptronul primește un set de date de intrare, fiecărei intrări fiindu-i asociată o pondere. Rezultatul final este determinat prin combinarea acestor valori și aplicarea unei funcții de activare. Procesul de învățare are loc prin ajustarea iterativă a ponderilor în funcție de eroarea obținută la ieșire, mecanism ce reprezintă baza învățării automate moderne. Aceasta abordare a demonstrat pentru prima dată posibilitatea construirii unor sisteme capabile să învețe din experiență, fără a fi programate explicit pentru fiecare situație (Figura 1.1).

Formal, pentru un vector de intrare $\mathbf{x} \in \mathbb{R}^n$, un vector de ponderi $\mathbf{w} \in \mathbb{R}^n$ și un termen de prag b , ieșirea perceptronului este dată de:

$$y = \varphi\left(\sum_{i=1}^n w_i x_i + b\right) = \varphi(\mathbf{w}^\top \mathbf{x} + b), \quad (1.1)$$

unde $\varphi(\cdot)$ este funcția de activare (treapta în formularea originală). Regula de învățare constă în actualizarea ponderilor proporțional cu eroarea $e = y_{\text{tintă}} - y$ și cu o rată de învățare η , $\mathbf{w} \leftarrow \mathbf{w} + \eta e \mathbf{x}$.

Cu toate acestea, deși perceptronul a reprezentat un pas revoluționar în dezvoltarea inteligenței artificiale, modelul prezenta limitări importante. În forma sa inițială, acesta putea clasifica doar date liniar separabile, ceea ce restricționa capacitatea de rezolvare a problemelor complexe. Din acest motiv, perceptronul simplu era eficient doar pentru reprezentări de date de intrare relativ simple, într-un singur strat de procesare. Aceste limitări au dus ulterior la dezvoltarea rețelelor neuronale multi-strat și a algoritmilor mai avansați de antrenare, care au stat la baza progreselor moderne din domeniul inteligenței artificiale.

Un progres major s-a ivit odată cu introducerea rețelelor neurale convoluționale (Convolutional Neural Network (CNN)) de către Yann LeCun și colaboratorii săi în lucrarea "Gradient-based learning applied to document recognition" [17]. Aceasta abordare a fost concepută pentru a depăși limitările modelelor anterioare, precum perceptronul simplu care nu putea procesa eficient informații complexe din imagini. Apariția rețelelor convoluționale a marcat începutul

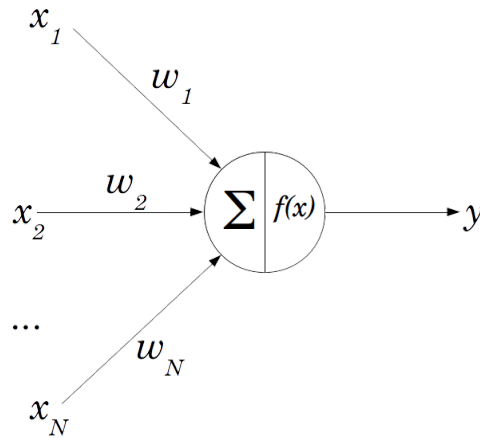


Figura 1.1: Modelul unui perceptron (sursa)

aplicării pe scara largă a inteligenței artificiale în domeniul procesării și analizei imaginilor.

Ideea fundamentală din spatele acestor rețele era relativ rudimentară, însă extrem de eficientă: datele de intrare nu mai sunt tratate ca pe un vector unidimensional, astfel modelul analizează porțiuni locale ale imaginii prin intermediul unor "ferestre" de convoluție. Aceste filtre parcurg imaginea și extrag automat trăsături relevante, precum muchii, forme sau texturi. În plus, utilizarea mai multor straturi succesive permite rețelei să învețe reprezentări din ce în ce mai abstracte și mai complexe ale datelor vizuale.

Un alt element important introdus de rețelele convenționale a fost utilizarea operațiilor specializate, precum convoluția (Figura 1.2) și subsampling-ul (pooling), care reduceau dimensionalitatea datelor și îmbunătățeau capacitatea de generalizare a modelului. Astfel, CNN au devenit capabile să identifice tipare spațiale complexe și să obțină performanțe superioare în sarcini precum recunoașterea facială, clasificarea imaginilor sau detecția obiectelor. Aceste inovații au reprezentat fundamentul dezvoltării ulterioare a tehnologiilor moderne de computer vision și deep learning.

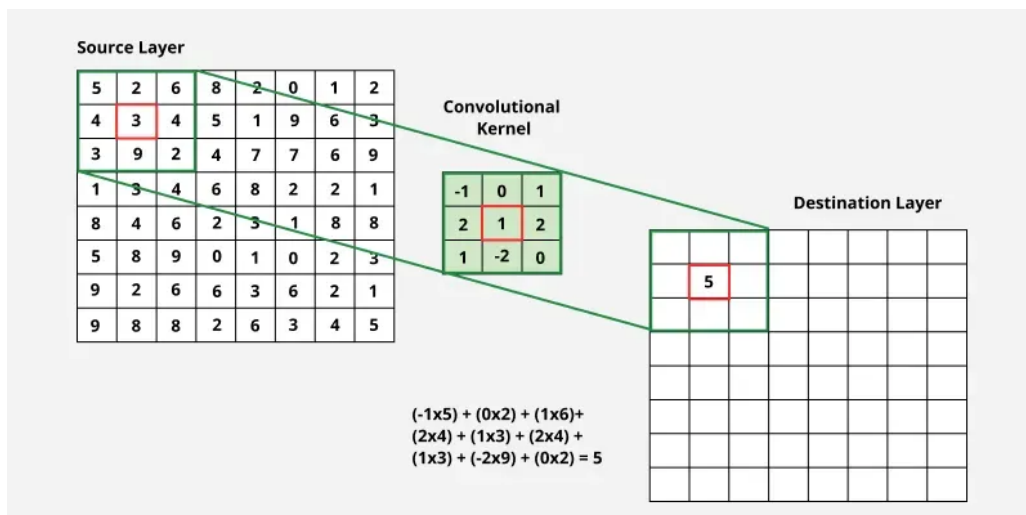


Figura 1.2: Convoluția cu un nucleu a unei imagini (sursa)

Deși rețelele neurale convoluționale (CNN) au demonstrat performanțe remarcabile în procesarea datelor statice, precum imaginile, acestea nu erau potrivite pentru aplicații în care informația are caracter secvențial. Domenii precum procesarea limbajului natural (Natural Language Processing (NLP)), recunoașterea vorbirii sau analiza datelor în timp presupun existența

unei dependențe între elementele succesive ale datelor, informația desfășurându-se în timp și nu într-o structură fixă bidimensională. Din acest motiv, utilizarea unor "ferestre" locale nu era suficientă pentru captarea relațiilor pe termen lung dintre elementele unei secvențe.

Primele încercări de rezolvare a acestei probleme au fost realizate prin intermediul rețelelor neuronale recurente (Recurrent Neural Network (RNN)), arhitecturi capabile să păstreze informația din pașii anteriori prin intermediul unei stări interne. Astfel, modelul putea utiliza contextul anterior pentru a influența predicțiile curente, ceea ce reprezenta un progres important pentru procesarea datelor temporale. Cu toate acestea, antrenarea RNN clasice s-a confruntat rapid cu o problemă fundamentală a învățării profunde: dispariția gradientului.

Această problemă apărea în timpul procesului de propagare inversă prin timp (Backpropagation Through Time (BTT)), atunci când secvențele analizate deveneau foarte lungi. Gradientii calculați în timpul antrenării se diminuau exponențial pe măsura ce informația era propagată către începutul secvenței, devenind atât de mici încât actualizările ponderilor ajungeau neglijabile. În consecință, rețeaua "uita" informațiile importante de la începutul secvenței și nu putea învăța *dependențe pe termen lung*.

Un progres major a fost realizat de Sepp Hochreiter și Jürgen Schmidhuber prin introducerea arhitecturii Long Short Term Memory (LSTM) în lucrarea "Long Short-Term Memory" [18]. Acest model a oferit rețelelor neuronale o formă funcțională de memorie pe termen lung, capabila să păstreze informații relevante pentru perioade extinse de timp. Elementul central al acestei arhitecturi îl reprezintă mecanismul de "porți" care controlează fluxul informației în interiorul rețelei. Prin aceste mecanisme, modelul poate decide ce informații trebuie păstrate, ce informații trebuie actualizate și ce informații trebuie eliminate din memorie.

Datorită acestei capacități de gestionare eficientă a contextului temporal, rețelele LSTM au devenit fundamentul multor aplicații moderne de inteligență artificială, fiind utilizate cu succes în traducere automată, recunoaștere vocală, generare de text și analiza de secvențe temporale, deși problema dispariției gradientilor nu dispăruse complet. O comparație schematică între cele două arhitecturi este prezentată în Figura 1.3.

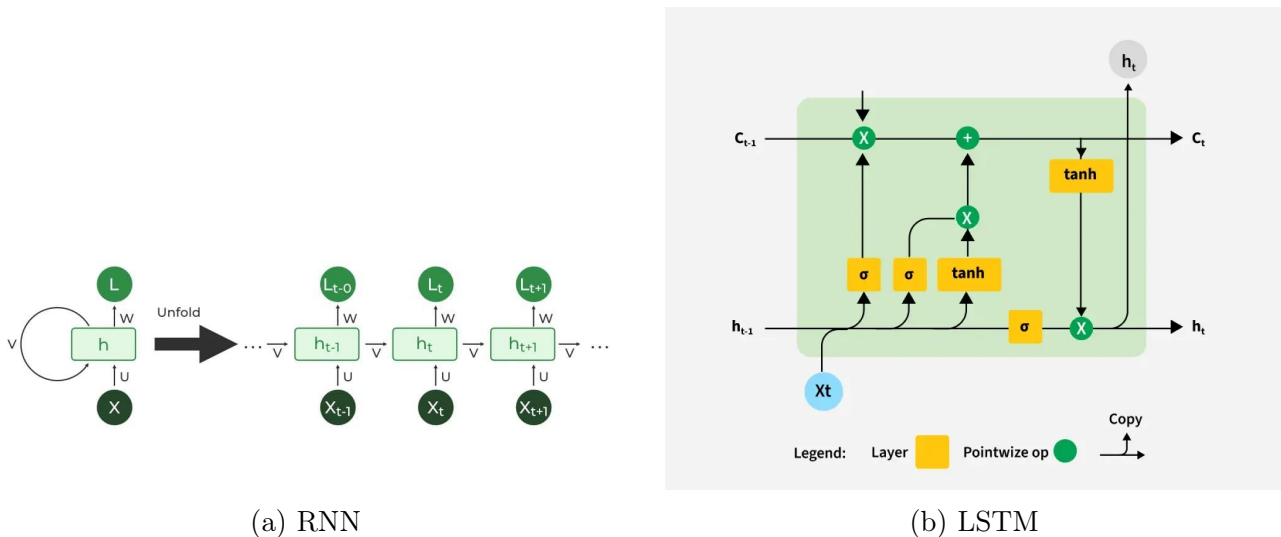


Figura 1.3: RNN și LSTM comparate (sursa)

La pasul de timp t , dinamica unei celule LSTM poate fi descrisa prin trei porți (uitare, intrare și ieșire) și o stare de celula $\mathbf{c}_t$:

$$\begin{aligned}\mathbf{f}_t &= \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f), \\ \mathbf{i}_t &= \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i), \\ \mathbf{o}_t &= \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o), \\ \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c), \\ \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t),\end{aligned}\tag{1.2}$$

unde σ este funcția sigmoid, $\odot$ produsul Hadamard, iar $[\cdot, \cdot]$ desemnează concatenarea vectorilor. Stocarea informației în $\mathbf{c}_t$ printr-o cale aditivă (controlată de $\mathbf{f}_t$ și $\mathbf{i}_t$) este principalul motiv pentru care gradientii nu se diminuează la fel de rapid ca în cazul unei RNN clasice.

Punctul de cotitura în evoluția inteligenței artificiale a fost reprezentat de introducerea arhitecturii "transformer", prezentată de Vaswani și colaboratorii săi în lucrarea devenită celebra "Attention is All you Need" [1]. Pana în acel moment, modelele bazate pe LSTM dominau procesarea limbajului natural, însă acestea prezentau o limitare importantă: datele secvențiale trebuiau procesate pas cu pas, fiecare element depinzând de rezultatul precedent. Aceasta caracteristica făcea antrenarea pe volume foarte mari de date să fie lentă și dificil de paralelizat, limitând scalabilitatea modelelor.

Arhitectura transformer (Figura 1.4) a schimbat fundamental această abordare prin introducerea mecanismului de **self-attention**. În loc să proceseze secvențial, modelul putea analiza simultan toate elementele unei propoziții și putea calcula relațiile dintre acestea indiferent de distanța lor în text. Astfel, fiecare cuvânt primea informații contextuale din întreaga secvență, ceea ce permitea captarea mult mai eficientă a dependențelor semantice și sintactice. În plus, eliminarea procesării strict secvențiale a permis paralelizarea operațiilor pe hardware modern precum Graphics Processing Unit (GPU), reducând semnificativ timpul de antrenare.

Mecanismul de *scaled dot-product attention* este definit, pentru matricile de interogări $\mathbf{Q} \in \mathbb{R}^{n \times d_k}$, chei $\mathbf{K} \in \mathbb{R}^{n \times d_k}$ și valori $\mathbf{V} \in \mathbb{R}^{n \times d_v}$, prin:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V},\tag{1.3}$$

unde factorul $1/\sqrt{d_k}$ stabilizează gradientii pentru valori mari ale dimensiunii cheilor.

Mai târziu, mecanismul de "self-attention" s-a transformat în "multi-head attention" care avea câte un cap de atenție pentru caracteristici specifice, putând astfel să învețe multiple tipuri de dependente între date, fiecare cap specializându-se pe un anumit tip de dependență între date, mai mult sau mai puțin abstract. Pe lângă deblocarea înțelegerii relațiilor complexe, Multi-Head Attention (MHA) a adus și o optimizare în timpul de antrenare și de inferență, dar și la îmbunătățirea capabilității de generalizare a modelelor. Formal, pentru h capete de atenție:

$$\begin{aligned}\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O, \\ \text{head}_i &= \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V),\end{aligned}\tag{1.4}$$

cu matricile de proiecție $\mathbf{W}_i^Q, \mathbf{W}_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $\mathbf{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ și $\mathbf{W}^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

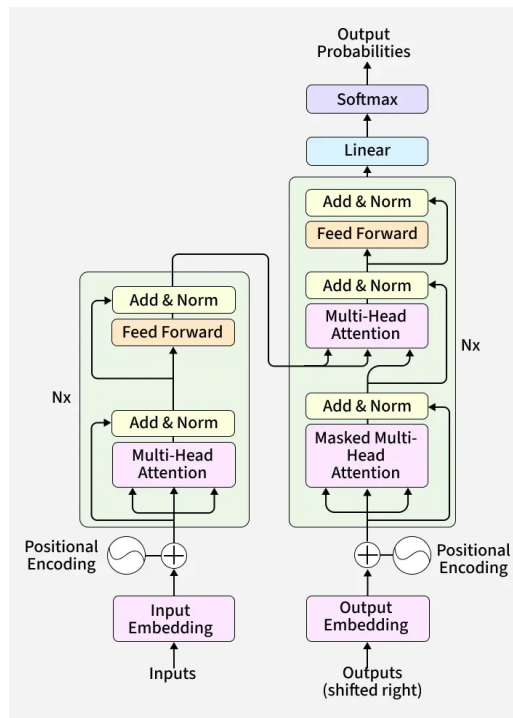


Figura 1.4: Arhitectura transformer [1]

Odată eliminată limitarea din punct de vedere computațional, cercetătorii au putut antrena modele pe seturi de date de dimensiuni fără precedent. Acest progres a deschis drumul către apariția modelelor mari de limbaj, LLM. Acestea sunt capabile să învețe reprezentări complexe ale limbajului natural prin analizarea unor cantități uriașe de text provenite din cărți, articole, pagini web și alte surse disponibile pe internet. Transformerul a devenit rapid fundamentul majorității sistemelor moderne de inteligență artificială dedicate limbajului natural, influențând profund atât cercetarea academică, cât și aplicațiile comerciale.

Modelele mari de limbaj reprezintă rezultatul direct al antrenării rețelelor pe volume masive de date și folosind un număr foarte mare de parametri. Cercetătorii au observat că performanța modelelor continua să crească odată cu extinderea dimensiunii acestora și a cantității de date utilizate la antrenare. Pornind de la această idee, echipa OpenAI a dezvoltat modelul Generative Pre-trained Transformer (GPT)-3, care includea aproximativ **175 de miliarde de parametri**, descoperirile fiind prezentate în lucrarea „Language Models are Few-Shot Learners” [19]. La momentul apariției, GPT-3 reprezenta unul dintre cele mai mari și mai performante modele lingvistice create.

Una din cele mai importante realizări ale acestui model a fost capacitatea de a rezolva sarcini complexe fără a necesita o antrenare suplimentară specifică fiecărei probleme. Prin simpla furnizare a câtorva exemple sau chiar doar a unei instrucțiuni formulate în limbaj natural, modelul putea genera cod sursă, realiza sumări de text, răspunde la întrebări, traduce texte sau crea conținut aparent creativ precum poezii și eseuri. Această particularitate a fost documentată în articolul prezentat anterior și a demonstrat că modelele de mari dimensiuni pot generaliza cunoștințe în domenii diferite și își pot adapta comportamentul fără modificări suplimentare ale parametrilor interni.

Prin aceste progrese, inteligența artificială a făcut tranziția de la sisteme specializate, concepute pentru abordarea unei sarcini predefinite, la modele capabile să manifeste competențe generale avansate. Această schimbare a reprezentat începutul unei noi etape în dezvoltarea inteligenței artificiale moderne, în care accentul s-a mutat de la proiectarea unor soluții dedicate către construirea unor modele inteligente universale, capabile să interacționeze natural cu oamenii și să rezolve probleme complexe.

² ³ îmbunătățind astfel gândirea pe termen lung, lucru documentat în lucrarea "TriAttention: Efficient Long Reasoning with Trigonometric KV Compression" de Mao și colaboratorii săi [21].

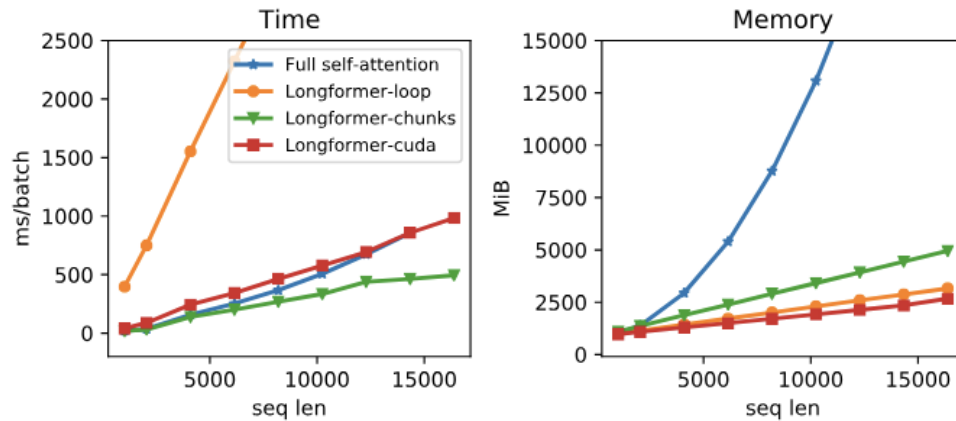


Figura 1.7: Performanța tipurilor de arhitecturi de tipul "Longformer" [3]. În timp ce mecanismul tipic de atenție utilizează memorie exponențial, arhitectura Longformer folosește memorie liniară odată cu creșterea contextului

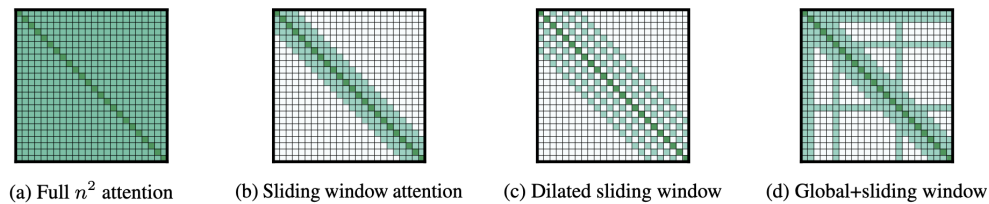


Figura 1.8: Diferite configurații de atenție folosite în arhitectura Longformer în comparație cu mecanismul obișnuit de atenție [3]

Încă un punct de optimizare al arhitecturii curente este îmbunătățirea embedding-urilor, ⁴ precum studiul realizat în lucrarea "RoFormer: Enhanced Transformer with Rotary Position Embedding" [4] care își propune să optimizeze poziția exactă a token-ului în secvența cu o matrice de rotație în timp ce încorporează dependența poziției relative în formula mecanismului de atenție, reușind astfel să capteze informații importante din secvența de intrare precum, lungimea secvenței sau relațiile dintre tokeni care se diminuează pe măsura ce distanța relativă dintre ei crește, echipând astfel mecanismul de atenție cu o codare pozițională relativă.

²Un token reprezintă datele de intrare pentru arhitectura transformer și uzual sunt cuvinte, părți din cuvinte sau caractere. Acestea permit modelului să spargă un cuvânt posibil mai rar în părți mai cunoscute.

³Cache-ul KV este o optimizare fundamentală a arhitecturii transformer ce elimină calculele redundante ale token-urilor anterioare în generarea auto-regresivă, conform lucrării "KV Cache Optimization Strategies for Scalable and Efficient LLM Inference" de Xu și colaboratorii [20]

⁴Embedding-ul este transformarea token-ului într-un vector abstract multi-dimensional ce reprezintă informația extrasa de model din acel token, plus un vector ce aduce informații despre poziția acestui în secvența.

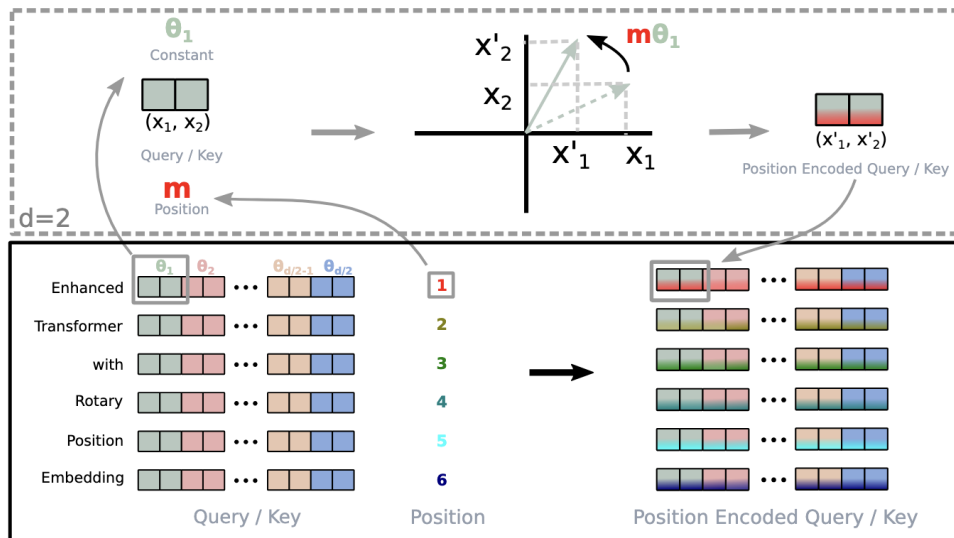


Figura 1.9: Principiul de baza pentru Rotary Position Embedding [4]

Îmbunătățiri s-au adus și în partea de fine-tuning ⁵ mai ales prin lucrarea LoRA: Low-Rank Adaptation of Large Language Models" [5] care aduce îmbunătățiri prin înghețarea ponderilor modelului și injectarea unor matrici antrenabile de descompunere în fiecare strat al arhitecturii transformer, reducând numărul de parametri antrenabili, reducând numărul acestor de **pana la 10.000 de ori** și puterea de calcul necesara de **3 ori** în comparație cu fine-tuning-ul normal. Concret, pentru o matrice de ponderi pre-antrenata $\mathbf{W}_0 \in \mathbb{R}^{d \times k}$, actualizarea este aproximata printr-o descompunere de rang mic:

$$\mathbf{W} = \mathbf{W}_0 + \Delta\mathbf{W} = \mathbf{W}_0 + \mathbf{B}\mathbf{A}, \quad \mathbf{B} \in \mathbb{R}^{d \times r}, \mathbf{A} \in \mathbb{R}^{r \times k}, r \ll \min(d, k), \quad (1.5)$$

unde doar $\mathbf{A}$ și $\mathbf{B}$ sunt antrenate, în timp ce $\mathbf{W}_0$ rămâne înghețată.

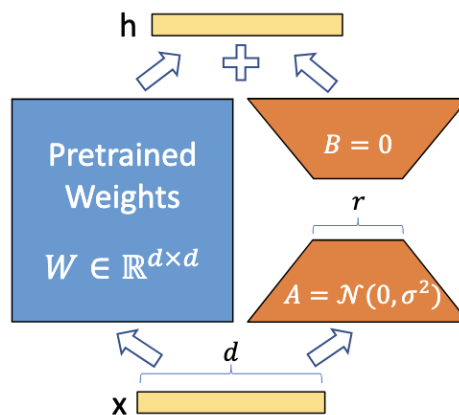


Figura 1.10: Fine-tuning-ul se realizează doar pe A și B [5]

Viteza de inferență a văzut de asemenea progrese, spre exemplu prin procedeul de decodare speculativa prezentata în articolul "Fast Inference from Transformers via Speculative Decoding"

⁵Fine tuning-ul este un procedeu prin care parametrii unui model sunt ajustați pentru eficienta sporita într-un anumit domeniu.

[22] care ilustrează problema timpului de inferență ⁶ care continua să crească pe măsură ce mărimea modelelor crește și ea. Autorii propun un algoritm care să permită livrarea mai multor tokeni în paralel. La baza lucrării stau doua observații fundamentale: (1) majoritatea sarcinilor abordate de modelele mari de limbaj pot fi scindate în obiective mai ușoare care pot fi abordate de modele mai mici și (2) folosind execuția speculativă și o noua metoda de eșantionare, decodarea poate fi accelerată rulându-le în paralel pe baza ieșirilor modelele mai mici ce aproximează pe cat de bine pot, generând mai multi tokeni simultan fără a modifica distribuția.

Continuând încercările de a eficientiza modelele mari de limbaj, a apărut întrebarea: "Dacă mărimea modelul încetinește antrenarea și inferența, de ce nu cautăm să micșorăm modelul?". Astfel, au apărut procedeele de cuantizare ⁷ și distilare ⁸. Problema distilării este nevoia unui model superior care să poarte rolul de "învățător". Zhang și echipa sa de cercetători documentează o alternativa a acestei metode prin procedeul de distilare proprie în lucrarea "Embarrassingly Simple Self-Distillation Improves Code Generation" [6] prin eșantionarea cu anumite valori ale temperaturii ⁹ și configurații ale trunchierii, în final executând fine-tuning supervizat ¹⁰ pe acele eșantioane.

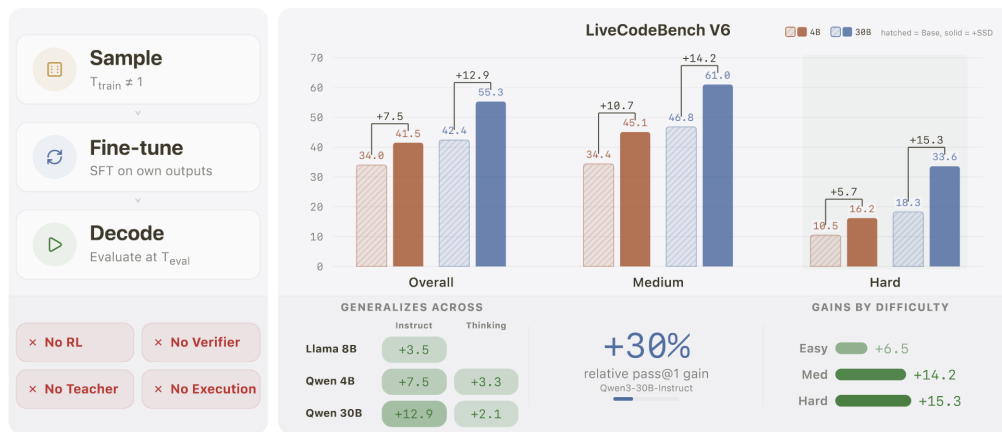


Figura 1.11: Rezultatele procesului de Simple Self Distillation [6]

⁶Inferența este procedeul prin care un model de limbaj ia datele de intrare, aplica transformările necesare și produce un răspuns final.

⁷Cuantizarea este reducerea tipului de date care stochează parametrii rețelei. Spre exemplu dacă în loc de 8 biți, stocăm parametrii pe 4 biți, înjumătățim amprenta modelului

⁸Distilarea este antrenarea unui model mai mic prin obiectivul reproducerii răspunsurilor modelului mai mare

⁹Temperatura este o setare pentru modelele de limbaj care controlează "creativitatea" modelului. O temperatură mică (spre exemplu 0) obligă modelul să scoată la ieșirea sa cel mai probabil token posibil, pe când o temperatură mare (spre exemplu 0.9), permite modelului să aleagă un alt token probabil.

¹⁰Supervizarea se referă la abilitatea modelului de a folosi exemple etichetate corect de oameni în prealabil pentru antrenare. Antrenarea supervizată nu include răspunsurile corecte, iar modelul trebuie să descopere singur tiparele ce reprezintă răspunsul corect

	LiveCodeBench v6 02/2025 - 05/2025								LiveCodeBench v5 08/2024 - 02/2025							
	PASS@1				PASS@5				PASS@1				PASS@5			
	ALL	EASY	MED	HARD	ALL	EASY	MED	HARD	ALL	EASY	MED	HARD	ALL	EASY	MED	HARD
THINKING																
Qwen3-4B-Thinking	54.5	97.4	59.2	29.7	67.5	100.0	75.4	45.8	59.6	98.0	69.5	31.4	70.3	98.9	80.1	47.5
+ SSD	57.8 +3.3	98.7 +1.3	62.8 +3.6	33.8 +4.1	71.4 +3.9	100.0	77.3 +1.9	53.1 +7.3	63.1 +3.5	98.5 +0.5	74.1 +4.6	35.7 +4.3	74.7 +4.4	98.9	86.8 +6.7	52.8 +5.3
Qwen3-30B-Thinking	66.1	99.8	73.0	41.5	76.7	100.0	84.7	59.7	69.1	98.6	81.7	43.9	79.8	98.9	91.4	61.1
+ SSD	68.2 +2.1	100.0 +0.2	76.4 +3.4	46.7 +5.2	80.2 +3.5	100.0	87.1 +2.4	65.8 +6.1	69.9 +0.8	98.4 -0.2	82.9 +1.2	44.9 +1.0	80.5 +0.7	98.9	91.8 +0.4	62.4 +1.3
INSTRUCT																
Llama-3.1-8B-Instruct	12.7	46.8	5.4	0.0	23.0	76.7	15.1	0.8	12.7	45.1	4.1	0.7	20.8	64.7	11.6	2.8
+ SSD	16.2 +3.5	52.6 +5.8	10.0 +4.6	1.6 +1.6	24.0 +1.0	73.3 -3.4	17.1 +2.0	3.3 +2.5	13.5 +0.8	45.2 +0.1	5.9 +1.8	1.3 +0.6	21.9 +1.1	67.1 +2.4	12.3 +0.7	3.7 +0.9
Qwen3-4B-Instruct	34.0	79.7	34.4	10.5	41.0	90.0	40.5	16.5	34.3	85.4	28.4	10.1	45.4	96.0	45.4	17.5
+ SSD	41.5 +7.5	86.5 +6.8	45.1 +10.7	16.2 +5.7	56.8 +15.8	98.1 +8.1	59.7 +19.2	34.1 +17.6	45.7 +11.4	98.2 +4.8	52.1 +23.7	16.6 +6.5	61.9 +16.5	98.8 +2.8	73.1 +27.7	33.6 +16.1
Qwen3-30B-Instruct	42.4	84.5	46.8	18.3	53.5	93.3	56.8	31.1	45.8	91.2	49.8	17.7	58.7	96.1	69.2	38.6
+ SSD	55.3 +12.9	91.0 +6.5	61.0 +14.2	33.6 +15.3	71.6 +18.1	99.9 +6.6	76.4 +19.6	54.1 +23.0	54.3 +8.5	92.9 +1.7	64.1 +14.3	25.9 +8.2	70.7 +12.0	97.6 +1.5	83.0 +13.8	47.2 +16.6

Figura 1.12: Îmbunătățiri în sarcini de programare ale modelelor după ce au fost supuse procesului de Simple Self Distillation[6]

Pe de alta parte, problema cuantizării este dezbătută în lucrarea "TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate" [23] ce își propune să cuantizeze vectori multi-dimensional fără distorsiuni majore ale datelor. Acest lucru este posibil prin transformări ale vectorilor de intrare, precum rotații aleatoare inducând o distribuție "Beta" și folosindu-se de independența aproximativă ale coordonatelor distincte în vectorii multi-dimensional. Se reușește astfel minimizarea memoriei necesare, a timpului de inferență și a lățimii de banda necesare pentru rularea modelului prin reducerea preciziei numerice. Acesta este eficient și în cazul ponderilor modelului, dar este foarte important și în cache-ul KV deoarece permite rularea modelelor cu ferestre de context ¹¹ lungi, optimizând necesitățile de memorie. Odată cu aceasta descoperire, au venit adaptări ale procedeelelor anterioare, precum LoRA [5] în cazul modelelor cuantizate. O astfel de lucrare este "QLoRA: Efficient Finetuning of Quantized LLMs" [7] care își propune să îmbunătățească fine-tuning-ul pentru modele cuantizate.

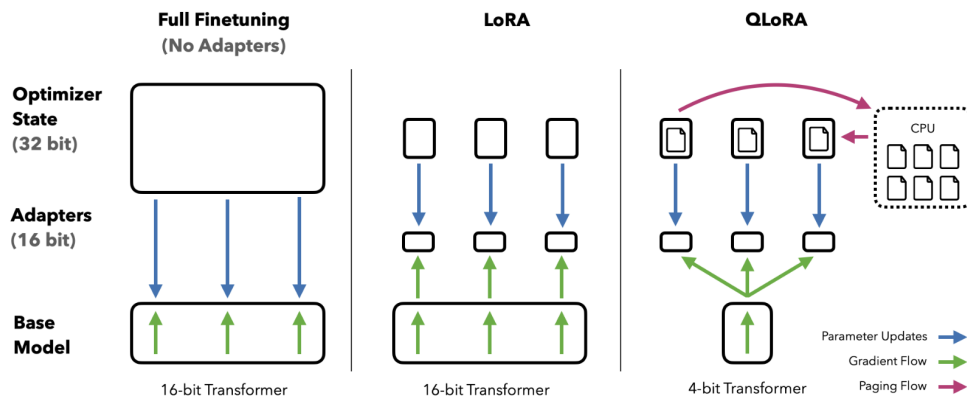


Figura 1.13: Comparația procedeele de fine-tuning: fine-tuning clasic vs LoRA vs QLoRA [7]

Pe lângă optimizările aduse arhitecturii transformer, o buna parte din evoluția gândirii adânci se bazează pe procedeul de Reinforcement Learning (RL). Pe scurt, modelul este recompensat mai mult sau mai puțin în funcție de răspunsurile pe care le da, iar obiectivul sau este să

¹¹Contextul se refera la informația pe care modelul o are disponibilă pentru a gândi un răspuns la cererea utilizatorului. Pe scurt, datele de intrare ale modelului de limbaj.

fie recompensat cat mai mult. Răspunsurile care prezintă gândire, mai exact Chain-of-Thought (CoT) [9] sunt recompensate mai mult decât cele ce nu prezintă aceasta particularitate, iar răspunsurile greșite, care nu sunt fundamentate sau conțin fraze nedorite sunt penalizate. În felul acesta, modelele învață să prefere să scoată date de ieșire care prezintă particularități ale gândirii, care duc în final la răspunsuri mai logice și capacitatea de a rezolva probleme mai complexe, având în vedere funcționarea decodului din transformer care generează pas cu pas, în funcție de pașii anteriori. În lucrarea "HARDTESTGEN: A High-Quality RL Verifier Generation Pipeline for LLM Algorithmic Coding" [24], de exemplu e discutată importanța RL și a modalității de a avea semnale mai bune de recompensa pentru aplicații de programare prin generarea de cazuri de test complexe, pe care verificatorul le evaluează și recompensează modelul în funcție de răspunsul dat.

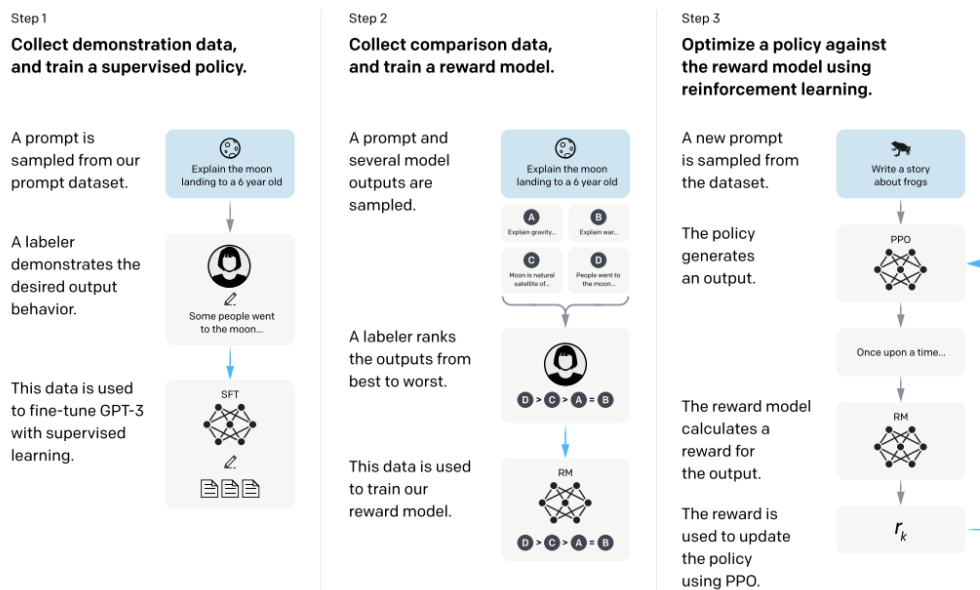


Figura 1.14: Ilustrarea procedurii de RL [8]

În final, o altă latură care a beneficiat de atenție sporită este prompt-ul ¹² pe care îl primește modelul. Modelele de limbaj, fiind auto-regresive adică generând date de ieșire pe baza datelor anterioare sunt dependente de un prompt bine gândit care maximizează contextul pe care îl are modelul și care îi impune limite și directive pe care să le respecte. Importanța acestui prompt fiind accentuată și de o nouă arie de specializare, de "Prompt Engineering" ce presupune construcția unor prompt-uri eficiente, lucru ilustrat în lucrarea "The Prompt Report: A Systematic Survey of Prompt Engineering Techniques" [25].

Având în vedere nevoia de prompt-uri ce produc rezultate consistente și versionabile, echipa condusă de Khattab a publicat lucrarea "DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines" [26], ce propune o abordare sistematică pentru a crea și optimiza prompt-uri. Au introdus astfel un model de programare ce abstractizează aceste prompt-uri prin grafuri de transformări textuale sau grafuri computaționale imperative și un compilator care funcționează cu acest nou limbaj, rezultatele fiind instrucțiuni optimizate ce întrec performanțele prompt-urilor scrise de mână.

¹²Prompt-ul este reprezentat de datele de intrare ale modelelor de limbaj, instrucțiunile pentru care trebuie să producă un răspuns la ieșire. Prompt-ul face parte din context.

1.3 Soluții agentice pentru industria dezvoltării software

Arhitecturile agentice presupun înzestrarea modelelor mari de limbaj cu unelte necesare interacționării cu ecosistemul în care rulează. Fundamentul acestor arhitecturi a fost pus de procedeul CoT [9], precedat mai târziu de Tree-of-Thought (ToT) [10] care a demonstrat ca un model de limbaj poate naviga probleme complexe prin descompunerea raționamentului în pași intermediari expliciți. Aceasta tehnica a deschis calea către generarea de planuri structurate, iar Wang și colaboratorii săi au extins aceasta idee prin metoda Plan-and-Solve [27], în care modelul este instruit să conceapă mai întâi un plan de acțiune ce divide sarcina în sub-obiective mai mici, iar apoi să le execute conform planului stabilit. Abordarea a reușit să elimine erorile de pași lipsa frecvente în CoT clasic, obținând performanțe comparabile cu promptarea few-shot pe probleme de raționament matematic, fără a necesita exemple manuale.

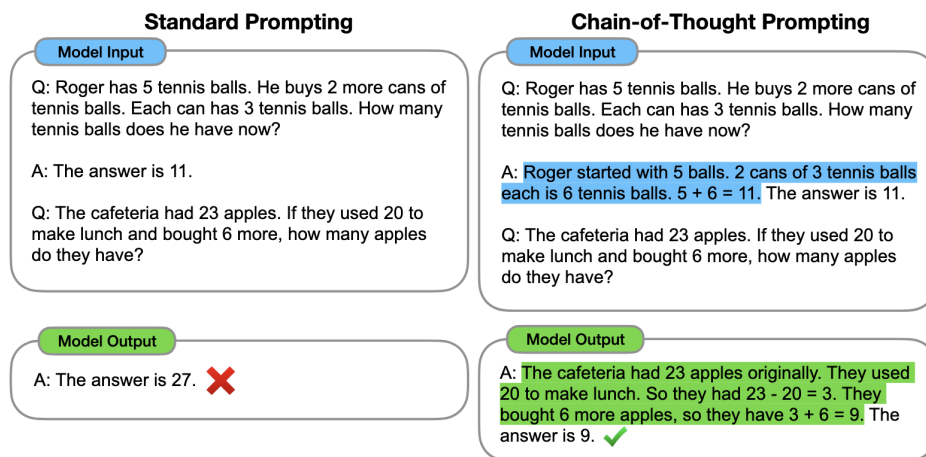


Figura 1.15: Exemple de CoT [9]

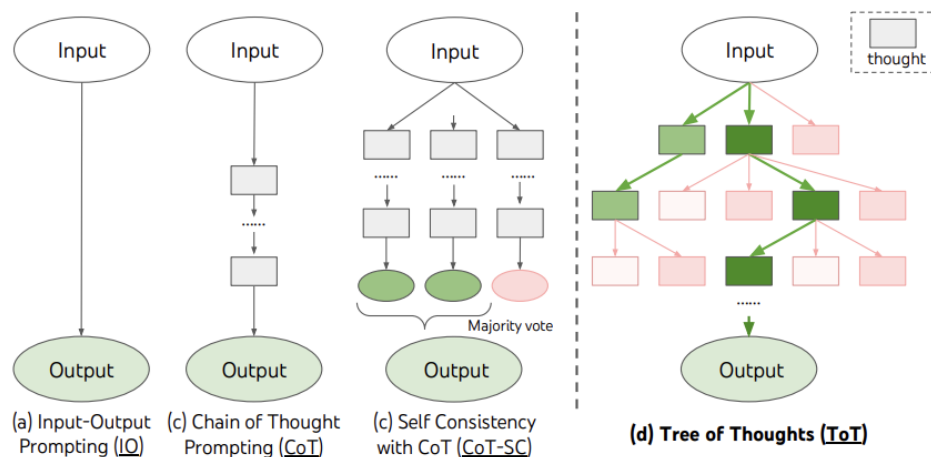


Figura 1.16: Diferența între procedee de gândire [10]

Mai târziu, abordarea ReAct [11] a combinat raționamentul cu acțiunea, permițând modelului să alterneze între generarea de gânduri intermediare și execuția de acțiuni concrete în mediul extern. La nivel operațional, bucla ReAct urmează un tipar de tipul *Thought* → *Action* → *Observation*, repetat pana la producerea răspunsului final. Un exemplu de astfel de bucla poate fi consultat în figura 1.17


```

Question: În ce limbaj este scris proiectul X?
Thought: Trebuie să inspectez fisierele din repository.
Action: list_files(repo="X")
Observation: ["main.py", "utils.py", "README.md"]
Thought: Extensia .py indica Python.
Action: finish("Python")

```

Figura 1.17: Tipar simplificat al buclei ReAct.

Aceasta sinergie între gândire și acțiune a încercat să combată fenomenul halucinațiilor¹³ și inconsistența în răspunsuri cu care se confruntau primele modele mari de limbaj. Totuși, paradigma ReAct presupunea o intercalare costisitoare între raționament și observații: modelul genera un gând, apela o unealta, aștepta răspunsul, apoi decidea următoarea acțiune pe baza tuturor token-urilor anterioare.

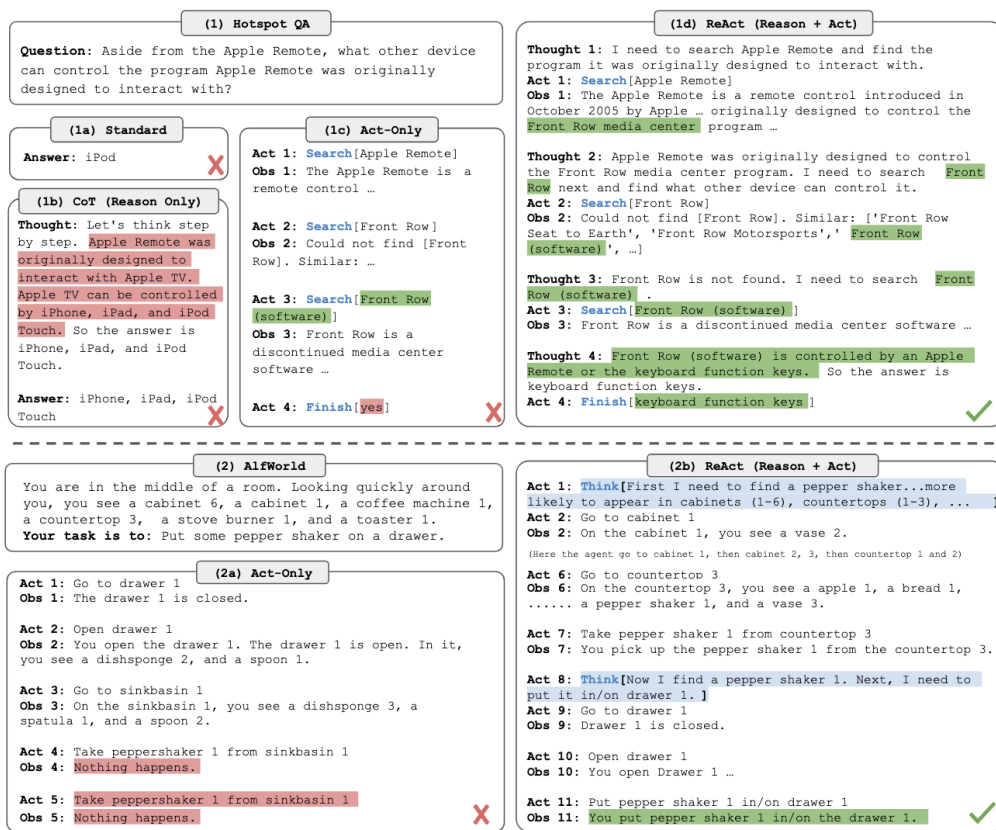


Figura 1.18: Paradigma ReAct [11]

	Type	Definition	ReAct	CoT
Success	True positive	Correct reasoning trace and facts	94%	86%
	False positive	Hallucinated reasoning trace or facts	6%	14%
Failure	Reasoning error	Wrong reasoning trace (including failing to recover from repetitive steps)	47%	16%
	Search result error	Search return empty or does not contain useful information	23%	-
	Hallucination	Hallucinated reasoning trace or facts	0%	56%
	Label ambiguity	Right prediction but did not match the label precisely	29%	28%

Figura 1.19: Îmbunătățirile pe care le aduce ReAct [11]

¹³Fenomenul de halucinații în modelele de limbaj se refera la generarea unor răspunsuri care nu sunt ancorate în fapte reale sau care nu au legătura cu prompt-ul de intrare

Xu și colaboratorii săi au identificat aceasta ineficiență și au propus arhitectura ReWOO [12] care decuplează complet procesul de raționament de observațiile externe. În loc să aștepte răspunsul fiecărei unelte înainte de a planifica următorul pas, modelul generează întregul plan de acțiuni dintr-o singură trecere, iar observațiile sunt colectate independent și integrate ulterior. Aceasta separare a redus consumul de tokeni **de pana la 5 ori** și a îmbunătățit acuratețea cu **4%** pe benchmark-ul HotpotQA, demonstrând totodată robustețe crescută în scenarii de eșec al uneltelor.

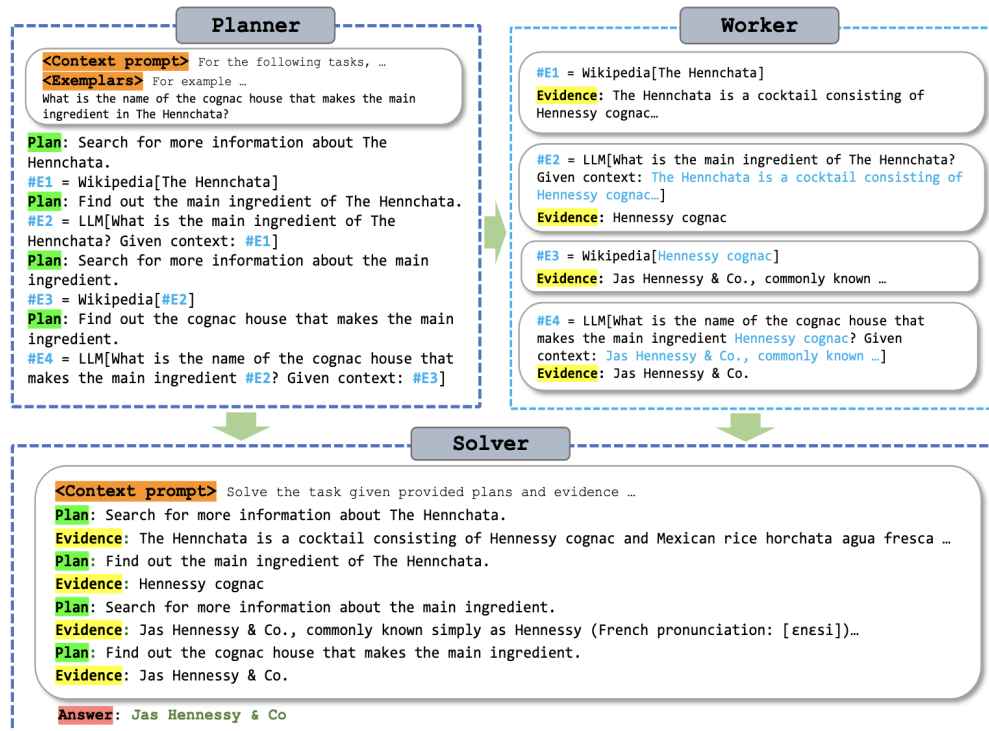


Figura 1.20: Ideea din spatele ReWOO [12]

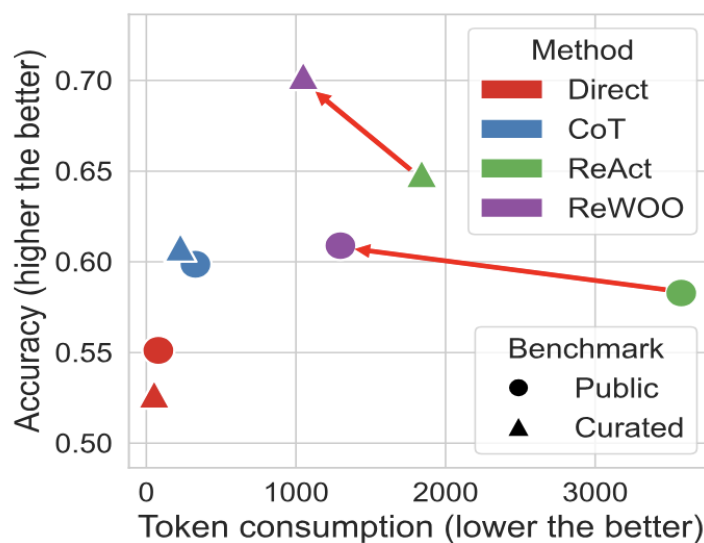


Figura 1.21: Comparația de performanță între paradigme pentru îmbunătățirea gândirii din punctul de vedere al acurateții vs. numărul de tokeni consumați [12]

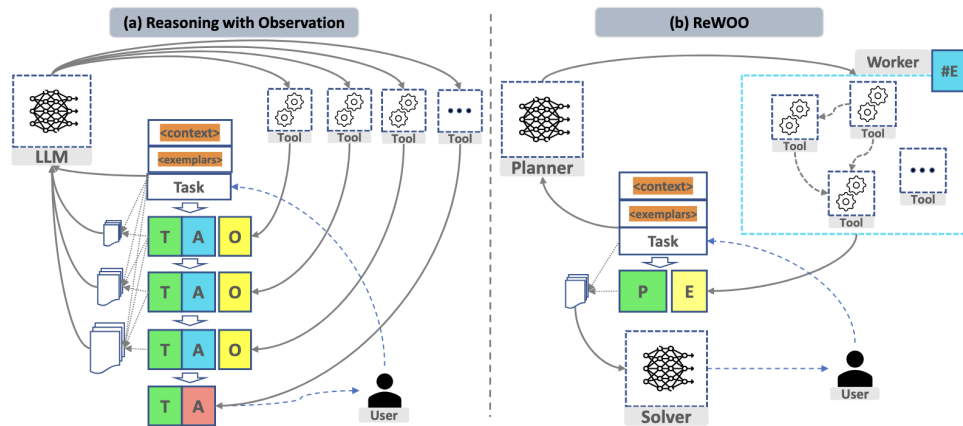


Figura 1.22: Îmbunătățirea adusa de ReWOO este delegarea mai multor libertăți către worker ce poate apela mai multe unelte pana să producă un răspuns. P = Plan, E = Evidence, A = Answer, O = Observation, T = Thought [12]

Încrederea în procesul de gândire al modelelor de inteligența artificială s-a cimentat pe măsura ce aceste tehnici au început să producă rezultate consistente, iar astfel a urmat înzestrarea acestora cu unelte care să le permită interacțiunea cu mediul în care rulează. Astfel, numeroase unelte ce le oferă capacitatea de a cauta pe internet pentru a găsi date actuale, rularea de comenzi în terminalul sistemului de operare sau chiar preluarea controlului browserului personal pentru automatizări, au apărut pe piață, numărul lor crescând exponențial și odată cu el și numărul de aplicații ale acestor modele.

Cu toate acestea, pe măsura ce agenții au început să proceseze cantități din ce în ce mai mari de informații din mediul în care operează, fereastra de context a devenit o limitare critică. Deși, multe modele sunt promovate ca având o fereastră de context maximă de 1 milion de tokeni precum Opus 4.8 de la Anthropic sau GPT-5.5 de la OpenAI, modelele se lovesc de anumite limitări în a accesa tot acel context. Liu și colaboratorii săi au demonstrat în lucrarea "Lost in the Middle" [13] că *performanța modelelor de limbaj se degradează semnificativ atunci când informația relevantă se afla în mijlocul unui context lung*. Modelele tind să utilizeze eficient informațiile de la începutul și de la sfârșitul secvenței de intrare, dar pierd din vedere elementele plasate în zona centrală, chiar și în cazul modelelor concepute explicit pentru contexte lungi (Figura 1.23). Aceasta descoperire are implicații directe asupra agenților care acumulează istoricul conversației, apelurile la unelte și observațiile într-un context tot mai voluminos.

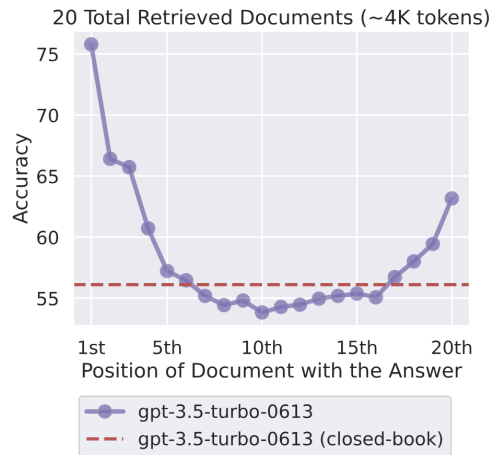


Figura 1.23: Modele tind să aibă o acuratețe foarte slabă contra întrebărilor ce vizează informații care se afla la mijlocul ferestrei curente de context [13]

Input Context

Write a high-quality answer for the given question using only the provided search results (some of which might be irrelevant).

Document [1] (Title: Asian Americans in science and technology) Prize in physics for discovery of the subatomic particle J/ψ. Subrahmanyan Chandrasekhar shared...

Document [2] (Title: List of Nobel laureates in Physics) The first Nobel Prize in Physics was awarded in 1901 to Wilhelm Conrad Röntgen, of Germany, who received...

Document [3] (Title: Scientist) and pursued through a unique method, was essentially in place. Ramón y Cajal won the Nobel Prize in 1906 for his remarkable...

Question: who got the first nobel prize in physics

Answer:

Desired Answer

Wilhelm Conrad Röntgen

Figura 1.24: Vizualizarea problemei și al rezultatului dorit la întrebare. De multe ori, modelele întâmpina dificultăți în accesarea acestor informații ascunse în mijlocul ferestrei de context [13]

Pentru a evalua riguros aceste limitări, Vodrahalli și echipa sa au propus framework-ul Michelangelo [14], o metoda de evaluare sintetica și minimala care depășește testele clasice de tip 'needle-in-a-haystack'. În loc să măsoare doar capacitatea de a regăsi o singura informație dintr-un context lung, Michelangelo testează abilitatea modelului de a sintetiza și corecta informații dispersate, prelevând o structura latentă din datele de intrare. Rezultatele au confirmat ca exista un spațiu semnificativ de îmbunătățire în procesarea informației din contexte lungi, chiar și pentru modelele de ultima generație.

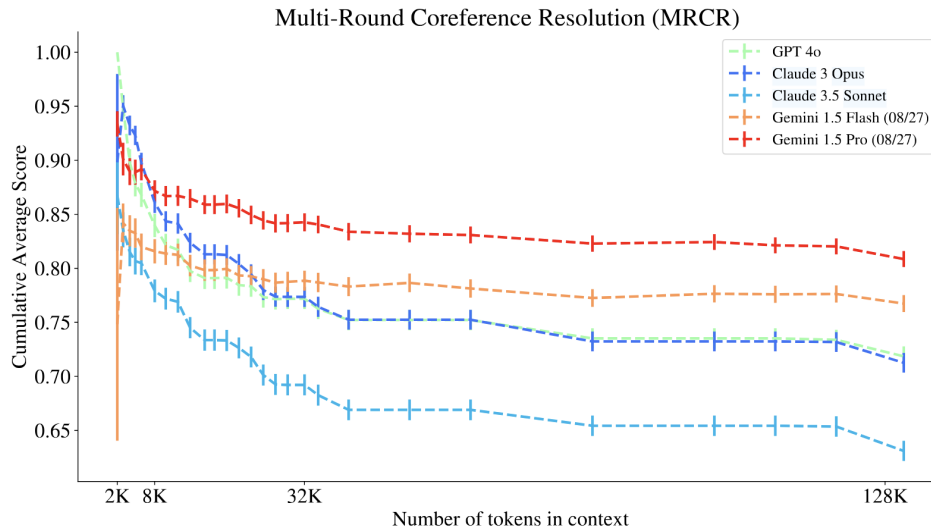


Figura 1.25: În majoritatea modelelor se observa o scădere a performanțelor după 32K tokeni pe sarcini de gândire complexa [14]

Din punct de vedere al eficienței computaționale, Li și colaboratorii săi au propus Lychee-Cluster [28], o metoda de gestionare a cache-ului KV care păstrează coerența semantică locală prin fragmentare bazată pe delimitatori și construiește un index ierarhic recursiv. Aceasta abordare transformă căutarea în cache dintr-o scanare liniară într-un proces de eliminare în timp logaritm, obținând o accelerare de **pana la 3.6 ori** a inferenței cu o degradare neglijabilă a performanțelor modelului. Pentru aplicațiile agentice care implica conversații lungi cu apeluri frecvente la unelte, astfel de optimizări sunt esențiale pentru a menține latentă scăzută și costurile sub control.

1.3.0.1 Analiza preliminară a soluțiilor existente pe piață

Industria dezvoltării software are probabil una din cele mai profitabile întrebunțări ale acestor arhitecturi agentice, datorită volumului masiv de munca existent în domeniu, așa că nu a durat mult până ce marii furnizori de pe piața modelelor de IA au început să ofere produse care să asiste programatorii în sarcinile lor. La momentul actual există mai multe soluții competitive: Codex de la OpenAI, Gemini Command Line Interface (CLI) de la Google sau Claude Code de la Anthropic. Toate acestea sunt soluții proprietare care sunt făcute să ruleze cu modelele proprii și pentru care utilizatorul trebuie să plătească. Există și soluții open-source precum OpenHands, Cline sau Aider cu care deținătorii de modele se luptă pentru a domina piața, deși lupta nu este strânsă din cauza monopolului soluțiilor proprietare.

Companie	Model	Preț [\$/MTok]	Capabilități
Anthropic (link)	Claude Opus 4.8	\$5/input MTok, \$25/output MTok	Cel mai capabil model disponibil, îmbunătățire semnificativa pentru programare agentica față de 4.6 și 4.7, gândire adaptiva, 1M fereastra de context, 128k tokens max. la ieșire
	Claude Opus 4.6	\$5/input MTok, \$25/output MTok	Lent, inteligent, gândire adaptiva, 1M fereastra de context, 128k tokens max. la ieșire
	Claude Sonnet 4.6	\$3/input MTok, \$15/output MTok	Mai rapid, compromite niște inteligența pentru performanță, gândire adaptiva, 1M fereastra de context, 64k tokens max. la ieșire
	Claude Haiku	\$1/input MTok, \$5/output MTok	Cel mai rapid, fără gândire adaptiva, 200k fereastra de context, 64k tokens max. la ieșire
OpenAI (link)	gpt-5.5	\$5/input MTok, \$30/output MTok	Noua clasa de inteligența pentru programare și munca profesionala, gândire adaptiva cu niveluri multiple de efort (low/medium/high/xhigh), suport multi-modal, 1M fereastra de context și 128k tokens max. la ieșire
	gpt-5.4	\$2.5/input MTok, \$15/output MTok	Cel mai inteligent pentru sarcini agentice, de programare și profesionale. 1M fereastra de context și 128k tokens max. la ieșire
	gpt-5.4-mini	\$0.75/input MTok, \$4.5/output MTok	Model mai mic, rapid dar puternic pentru programare și subagenți. 400k fereastra de context și 128k tokens max. la ieșire
	gpt-5.4-nano	\$0.20/input MTok, \$1.25/output MTok	Model mic și foarte rapid pentru sarcini ușoare în volum mare. 200k fereastra de context și 128k tokens max. la ieșire
Google (link)	Gemini 3.1 Pro	\$2/input MTok, \$12/output MTok < 200k tokens, \$4/input MTok, \$118/output MTok ≥ 200k tokens	Model pentru sarcini complexe ce necesita cunostiinte dezvoltate. Fereastra de context 1M in / 64k out
	Gemini 3.1 Flash	\$0.5/input MTok, \$3/output MTok	Inteligența comparabila cu modelul Pro la o viteza mai mare. Fereastra de context 1M in / 64k out
	Gemini 3.1 Flash-Lite	\$0.25/input MTok, \$1.5/output MTok	Model construit pentru costuri reduse și volum mare de sarcini. Fereastra de context 1M in / 64k out
	Gemma 4	Open Source	Familie de modele open-source pentru rulare locala, cost redus și personalizare
DeepSeek (link)	DeepSeek V4	\$0.30/input MTok, \$0.50/output MTok	Model general de ultima generație, suporta mod cu și fără gândire adâncă, 1M fereastra de context, cache hit: \$0.03/input MTok
	DeepSeek R1	\$0.55/input MTok, \$2.19/output MTok	Model specializat în raționament și gândire adâncă pentru probleme complexe, 128k fereastra de context, cache hit: \$0.14/input MTok
Meta (link)	Llama 4	Open Source	Performante similare cu GPT-4o sau Gemini 2.0 Flash, dar open-source. Fereastra de context 10M/1M în funcție de model.
Qwen/Alibaba (link)	Qwen 3	Open Source	Familie de modele open-source (Apache 2.0) cu arhitectura hibrida Mixture of Experts (MoE), gândire adaptiva activabila, performanță de vârf la programare și raționament. Disponibil în variante de 0.6B pana la 235B parametri
	Qwen 3.6 Plus	\$0.325/input MTok, \$1.95/output MTok	Varianta Application Programming Interface (API) proprietara (Alibaba Cloud), arhitectura hibrida MoE cu atenție liniara, gândire adaptiva, 1M fereastra de context, 64k tokens max. la ieșire
Mistral AI (link)	Mistral Large 3	\$0.50/input MTok, \$1.50/output MTok	Cel mai capabil model Mistral, open-weight, multi-modal, performanță de vârf. 262k fereastra de context
	Mixtral 8x22B	Open Source	Model Mixture-of-Experts open-source (Apache 2.0), 8 experți de 22B parametri fiecare, disponibil pentru rulare locala sau prin API. 66k fereastra de context

Tabela 1.1: Comparația câtorva din modelele existente și a capabilitățile acestora în comparație cu prețul de utilizare

Nota: Toate modelele au și versiuni pentru diferite nivele de efort pentru gândire (reasoning effort) sau pentru date multi-modale ce pot influența și costul dar și rezultatele

Din tabelul 1.1 se poate deduce ca utilizarea frecventa a unui model de ultima generație poate ajunge cu ușurință să coste o mica avere. Cu o mica aproximare, 200.000 de tokens sunt echivalentul unei cărți de 400 de pagini. Dar contorizarea nu este făcuta doar pentru instrucțiunile originale. Într-o conversație cu un model de limbaj fiecare pereche răspuns-întrebare din istoricul conversației respective este re-introdus sub forma de date de intrare pentru următorul prompt pe care utilizatorul îl introduce. Astfel, în loc de câțiva tokeni de intrare, într-o conversație lunga cu un istoric dezvoltat, costul de întreținere al conversației crește exponențial.

De asemenea, fiecare apel către unelte externe precum căutările web, rămân în istoricul conversației pentru ca modelul să aibă context în query-urile ulterioare. Fiecare dintre aceste apeluri la unelte consuma o buna parte din fereastra de context a modelului, crescând costul.

Din punct de vedere al capacității de rezolvare a sarcinilor reale din industria dezvoltării software, clasamentul celor mai bune modele se schimba frecvent. Utilizând SWE-Bench Pro [15], putem măsura puterea de abordare a acestor modele de top contra unor sarcini complexe (Figura 1.26), iar Figura 1.27 oferă o perspectiva complementara, în funcție de cost.

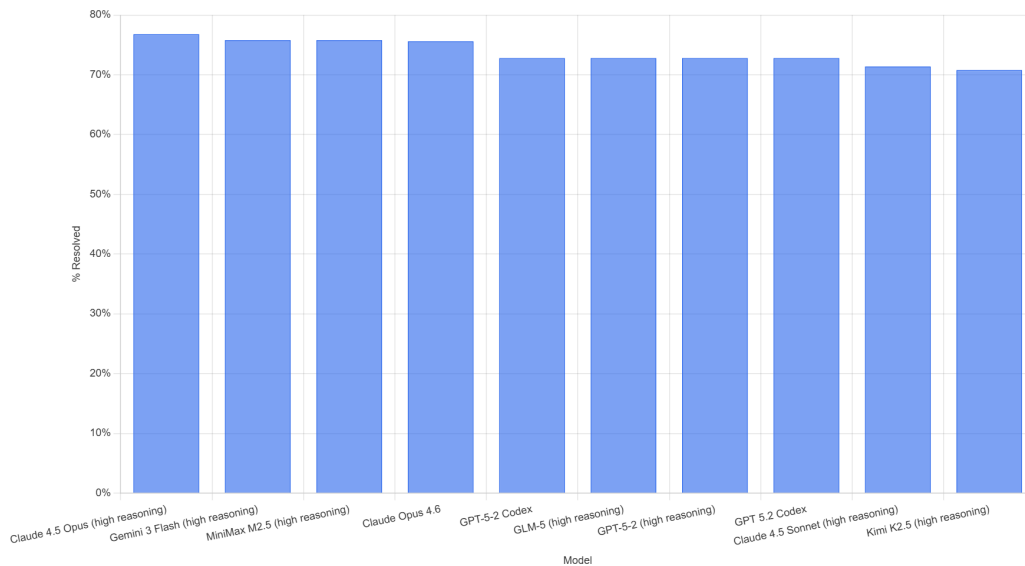


Figura 1.26: Clasamentul modelelor în SWE-Bench Pro [15] la momentul actual în funcție de performanța rezolvării sarcinilor complexe în limba engleza

Pe lângă puterea de rezolvare, a apărut un alt fenomen care influențează rata de adopție a unui model: costul. Deși performanțele modelelor cresc și acestea sunt din ce în ce mai integrate în viațile noastre de zi cu zi, asemenea telefoanelor mobile, costul de rulare al acestor modele crește și el. Cele mai puternice modele au nevoie de foarte multe resurse pentru antrenare dar și pentru inferență. Acest lucru se reflecta asupra utilizatorului final care trebuie să plătească per-token sau un abonament lunar care include un număr limitat de tokeni. Modelele cu capacități de gândire adâncă, pe lângă faptul ca au un cost mai mare per-token, consuma și mai multi tokeni din cauza gândirii dezvoltate sau a ferestrei de context mari, ridicând și mai mult costurile. Pentru asta, fiecare deținător de modele fundamentale, are mai multe versiuni ale aceluiași model, pentru întrebuințări de diverse complexități, pentru a ajuta cu costurile dar și cu balansarea resurselor. Spre exemplu Google deține modelele din gama Gemini, cu versiunile "Flash", "Thinking", "Pro".

Având în vedere problema costului dar și a balansării traficului, aplicațiile de asistenți de programare de pe piață, au implementat o modalitate de a clasifica prompt-urile utilizatorului în funcție de complexitate și rutarea acestui prompt către un model potrivit să rezolve aceasta problemă și optim din punct de vedere al costului.

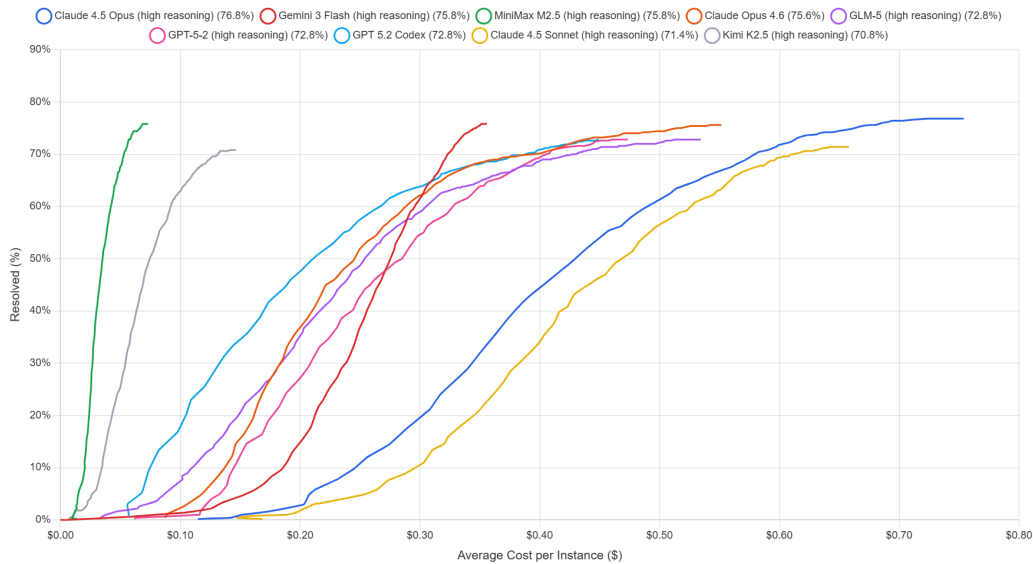


Figura 1.27: Clasamentul modelelor în SWE-Bench Pro [15] la momentul actual în funcție de cost

1.4 Cerințe pentru o soluție agentică în dezvoltarea software

Pentru a realiza un asistent inteligent în domeniul dezvoltării software este nevoie să atingem câteva obiective principale, printre care: o interfață accesibilă din diferite medii de dezvoltare pentru a avea un număr mare de suprafețe de utilizare, unele ce pot rula în terminalul oricărui sistem de operare, posibilitatea de a alege dintr-o sumedenie de modele, comenzi pentru a controla parametrii modelelor și un prompt de sistem și skill-uri specializate pentru operații din domeniu.

Agenții funcționează, pe lângă procedeele expuse anterior, pe una dintre următoarele filozofii de programare: Spec-Driven Development (SDD) sau Test-Driven Development (TDD). Primul se referă la programarea pe baza unor documente de specificație ce expun în detaliu toate necesitățile implementării, iar cel din urmă se referă la programarea după cerințele sumare și executarea testelor la final pentru a confirma comportamentul dorit.

Pe lângă aceste obiective de bază, trebuie să ne asigurăm ca modelul răspunde doar prompturilor cu scop legitim, și să ne asigurăm ca modelul nu poate rula comenzi care pun în pericol integritatea sistemului pe care rulează sau compromite date sensibile, lucruri pe care le vom discuta în capitolele ce urmează.

1.5 Limitări ale arhitecturilor agentice

Deși entuziasmul pentru bula IA atinge cote maxime, încă este loc de îmbunătățire, iar aceste platforme de asistenți de programare au încă multe defecte pe care trebuie să le depășească.

În primul rând, **bariera principală în avansarea acestor asistenți este fereastra de context**. Aceasta fereastră reprezintă cata informație poate să țină minte agentul. Pentru sisteme dezvoltate cu zeci de mii de linii de cod, sau soluții împărțite pe micro servicii în mai multe proiecte, acest context limitează masiv capacitățile agenților. Există multe tehnici de optimizare a acestui context de la compresie, realizarea unor fișiere ce rezumă cunoștințele actuale și începerea unei conversații noi care pleacă de la acel rezumat până la rularea de cod pentru acțiuni repetitive sau alegerea cu grijă a prompt-ului. Dar din păcate toate aceste tehnici nu sunt de ajuns. Multe funcționalități care fac un agent performant precum tool-urile reușesc să consume rapid aceasta fereastră de context. Ca exemplu, 200.000 de tokeni pot

părea un număr mare, dar chiar și acest număr, similar cu memoria hardware, este defapt, mai mic. Din aceasta fereastră doar 160.000 de tokens sunt disponibili cu adevărat. O parte din ei sunt rezervați pentru promptul de sistem, auto-compactare sau pentru ieșire. Depășirea acestei ferestre de context nu poate fi atinsă propriu zis deoarece modelul nu permite ingestia de date de intrare mai mari decât capacitățile sale.

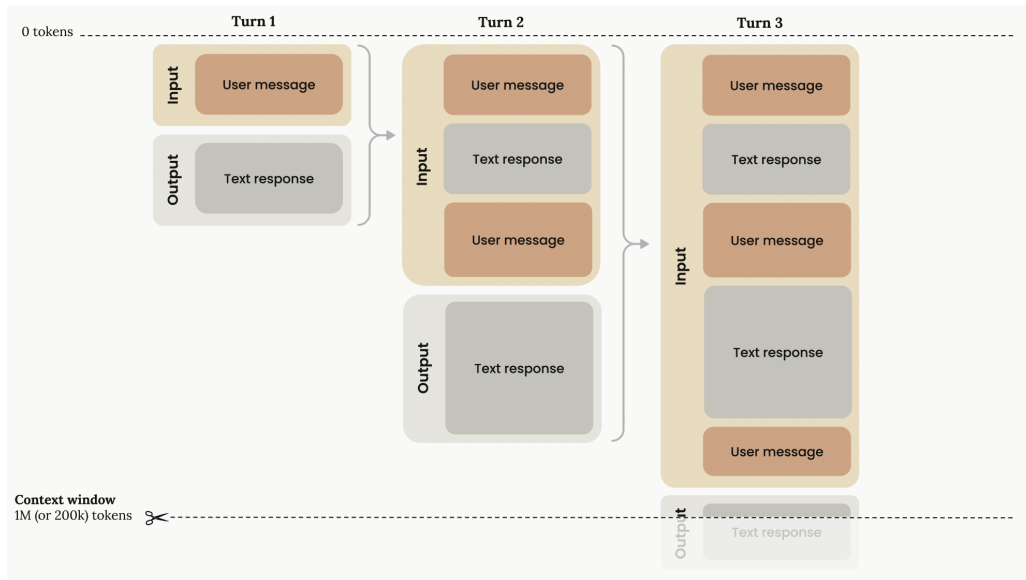


Figura 1.28: Reprezentarea ferestrei de context într-o conversație uzuală (sursa)

În al doilea rând, modelele curente par să genereze date de ieșire mai detaliate decât este necesar, preferând cantitatea peste calitate. De multe ori în sarcini de tipul Pull Request (PR) Review, agenții se grăbesc să relateze multe probleme simple sau de formatare și ratează probleme importante de logica.

În al treilea rând, similar cu punctul anterior, agenții nu au o bună înțelegere legată de când ar trebui să oprească investigațiile sau să încerce alte soluții și rămân blocați în bucle infinite de a încerca aceleași lucruri și a eșua în final.

Pentru toate aceste probleme se lucrează în mod curent la rezolvări de care vom vorbi ulterior.

1.6 Justificarea alegerilor de implementare

Soluția finală include mai multe categorii care trebuie comparate independent.

Pentru început, modelele disponibile inițial vor fi din gama Gemini, Anthropic OpenAI, și modele open-source rulate local prin Ollama sau în cloud.

Pentru partea de framework agentic, ofertele sunt numeroase și variate. Fiecare dintre opțiunile de mai jos are avantaje și dezavantaje care trebuie cântărite în contextul aplicației noastre.

În materie de limbaj de programare, **Python** a fost alegerea evidentă deoarece este aproape folosit în totalitate în astfel de aplicații, devenind un standard în industrie. Ecosistemul este de asemenea foarte matur și există o sumedenie de biblioteci pentru a ne ajuta în implementare. De asemenea încă un atu este portabilitatea cu care vine, Python putând fi instalat cu ușurința pe orice sistem de operare cu o simplă comandă.

Am pornit de la ideea că trebuie să avem nevoie de un framework gândit pentru utilizarea uneltelor cu compatibilitate pentru mai multe modele, care să suporte multiple ramuri de execuție, să încorporeze managementul stării curente și al memoriei și să poată construi un

workflow robust și consistent. Pentru asta, în tabelul 1.2 am adunat o colecție de framework-uri folosite în industrie pentru a le compara și a decide care este cel potrivit.

Criteriu	LangChain	LangGraph	Google Gen AI / Vertex	LiteLLM	OpenAI SDK
Rol Principal	Orchestrator general de LLM-uri și integrări.	Framework pentru agenți stateful și fluxuri ciclice.	Platformă enterprise integrată în Google Cloud.	Proxy sau router universal pentru 100+ LLM-uri.	Client oficial pentru modelele OpenAI.
Nivel Abstr.	Înalt (multe componente gata de utilizat).	Mediu/Înalt (control fin asupra grafurilor).	Înalt (Managed Services în Vertex AI).	Jos (strat de traducere API).	Jos/Mediu (interacțiune directă).
Capabilități Agentice	Limitate (predominant fluxuri liniare).	Excelent (bucle, feedback și multi-agent).	Bun (suport nativ pentru tool-uri).	N/A (doar rutare de mesaje).	Depinde de utilizator (manual sau Assistants API).
Suport Multi-Model	Da (agnostic).	Da (prin ecosistemul LangChain).	Optimizat pentru Gemini.	Absolut (sco-pul principal).	Nu (doar OpenAI).
Management Stare	Încorporat (Memory buffers).	Superior (persistență și „time travel”).	Gestionat de platformă.	N/A (stateless).	Threads (doar în Assistants API).
Învățare	Medie (ecosistem foarte mare).	Abruptă (necesită logica grafurilor).	Ușoară (Graphical User Interface (GUI) intuitiv în Cloud).	Ușoară (identic cu OpenAI Software Development Kit (SDK)).	Foarte Ușoară, documentat

Tabela 1.2: Comparația framework-urilor pentru întrebări agentice

La nivel conceptual, LangGraph modelează un workflow agentic ca un *graf orientat* în care nodurile reprezintă unități de calcul (apeluri către LLM, execuții de tool-uri sau funcții Python arbitrare), iar muchiile descriu tranzițiile dintre acestea. Spre deosebire de abordările liniare de tip “chain”, muchiile pot fi *condicionale*, ceea ce permite ramificări dinamice în funcție de starea curentă, precum și *cicluri*, esențiale pentru bucle de tip “reason–act–observe”. Întregul graf operează asupra unui obiect de *stare* partajată, pe care fiecare nod îl poate citi și actualiza prin *reducers* declarativi, eliminând astfel ambiguitățile legate de propagarea contextului între pași.

Un alt element central este mecanismul de *checkpointing*: starea grafului este serializată după fiecare pas și poate fi reluată, inspectată sau derulată înapoi (*time travel*). Acest lucru oferă două proprietăți importante pentru un asistent de dezvoltare software: persistența conversației între sesiuni și posibilitatea de a introduce puncte de *human-in-the-loop* în care execuția este suspendată până când utilizatorul confirmă o acțiune (de exemplu, aplicarea unei modificări de cod sau rularea unei comenzi).

Luând în calcul cele de mai sus, în final am ales să folosesc **LangGraph** datorită controlului

mare pe care îl oferă asupra desfășurării workflow-ului agentic și a integrării cu Langfuse dar și pentru compatibilitatea cu logica desfășurării procesului de asistentă în dezvoltarea software, respectiv buclele de scriere de cod, feedback, investigare și îmbunătățire.

Pentru interacțiunea cu utilizatorul, ideea de pornire a fost ca trebuie să avem o interfață care poate fi rulată în orice context. Gândul m-a purtat automat către un TUI deoarece orice sistem de operare deține un terminal care poate fi pornit în orice mediu de programare, oferind o gama larga de utilizatori potențiali. În tabelul 1.3 am constituit o lista de abordări și librării pe care le putem folosi pentru a construi aceasta interfață, fiecare cu avantaje și dezavantaje.

Bibliotecă	Arhitectură	Avantaje	Dezavantaje
Textual	Motor TUI pe ecran complet. Arbore de widget-uri bazat pe Document Object Model (DOM)	• Sistem de layout similar cu Cascading Style Sheets (CSS) • Formatare de text încorporată nativ • Complet asincron și bazat pe evenimente	• Folosește un scrollbar fals (nu funcționează scroll-ul nativ) • Supraîncărcarea DOM-ului cauzează lag dacă mesajele vechi nu sunt șterse • Dificultate la copierea/lipirea nativă a textului
Prompt Toolkit și Rich	Experiență interactivă folosind buffer-ul principal al terminalului. Afișare prin Standard Output (ST-OUT).	• Derulare perfectă prin scrollbar-ul nativ al terminalului • Zero lag sau probleme de memorie pe măsură ce istoricul crește • Selectarea și căutarea eficienta	• Nu este un motor real de layout GUI (doar flux liniar) • Difcil de implementat panouri laterale sau complexe • Limitat strict la input ancorat în partea de jos
Urwid	TUI clasic pe ecran complet folosind buffer-ul alternativ al terminalului. Ierarhie strictă de widget-uri.	• Extrem de matur, performant, stabil și testat în producție • Gestionare excelentă a input-ului standard	• API învechit și curbă de învățare mai abruptă • Sincron implicit (necesită integrare suplimentară)

Tabela 1.3: Comparație biblioteci pentru interfața cu utilizatorul

Inițial, abordarea cu Textual părea corectă pentru că oferă un sistem de layout foarte puternic care poate fi folosit pentru a construi o interfață complexă cu panouri laterale și meniuri. Însa, în timpul implementării a trebuit să abandonez această idee deoarece am întâmpinat probleme cu comportamentul interfeței în cadrul unei conversații mai lungi cu elemente animate.

Astfel, am finalizat lista alegerilor arhitecturale principale, motivații suplimentare pentru restul alegerilor vor fi dezvoltate punctual, pe parcursul implementării.

Capitolul 2

PLASAREA ÎN INDUSTRIE

În acest capitol vom aborda problematica plasării proiectului în industrie și alegerile pe care le-am făcut pentru a propune un produs competitiv.

2.1 Software open-source

Momentan, cea mai mare cota de piață o au furnizorii de modele mari de limbaj precum Anthropic, Google sau OpenAI care lansează aplicații care au ca obiectiv blocarea utilizatorilor în ecosistemul lor personal, prin unelte ce urmăresc domenii diverse, fenomen observabil în tabelul 2.1.

În timp ce exista și alternative open source, acestea se lovesc de problemele tipice pentru astfel de programe. Pentru început, companiile ce furnizează accesul la aceste modele au tarife diferențiale pentru utilizarea cheilor API în astfel de programe și orice încercare de a înșela sistemul poate rezulta în restricționarea accesului la platformele furnizorilor.

De asemenea, o alta problemă întâmpinată de platformele open source este lipsa fondurilor sau numărul mic de contribuitori, suportul pentru suita întreaga de funcționalități a programelor deținute de marile firme sau publicitate.

În ciuda acestor dificultăți, având în vedere creșterea vertiginoasă a costurilor, faptul ca accesul la program este gratuit este un mare plus pentru utilizatori. Un alt avantaj este suportul pentru modele de limbaj open source. Acestea sunt foarte ușor de folosit chiar pe calculatorul utilizatorului dacă deține componente hardware suficiente pentru modelul ales, reducând costurile și mai mult. Deși și pentru soluțiile proprietare se pot face ajustări mai mult sau mai puțin promovate de furnizorii de modele care să permită folosirea modelelor open source, este preferat un program a cărui scop este o platforma agnostica de furnizor.

2.2 Flux agentic ce rulează local

Un alt plus este **confidențialitatea datelor și controlul asupra aplicației**. Toate datele rămân pe dispozitivul sau serverul utilizatorului (exceptând cazurile în care acesta alege explicit să folosească modele de la furnizori și nu modele open source). Acesta oferă un mare avantaj pentru domenii în care se lucrează cu date importante sau care necesita o confidențialitate ridicată.

Toate fișierele de configurație sunt disponibile local și au o sumedenie de setări pe care utilizatorul le poate modifica după bunul plac.

Rulând aplicația local, agentul are acces direct la mediul de lucru și astfel are context despre sarcinile pe care trebuie să le execute. Mai mult, este eliminată necesitatea intervenției umane (pe lângă acțiunile periculoase ce necesita permisiuni speciale). Acesta poate executa acțiuni și prelua controlul asupra dispozitivului, permițând astfel automatizări complexe, recurente sau care rulează o singura dată, ușurând sarcini repetitive, salvând astfel timp prețios în favoarea utilizatorului.

Acțiunile pe care le poate rula un agent local sunt numeroase datorită integrării cu terminalul sistemului de operare prin care poate controla o mulțime de aspecte.

2.3 Observabilitate

Majoritatea panourilor cu informații de utilizare de la furnizorii de modele de limbaj sunt sumare sau insuficiente. Aplicațiile în terminal au uzual mai multe date de utilizare spre deosebire de website-urile în care se desfășoare experiența conversațională, dar nu oferă prea multe date despre ce unelte în mod cert consuma cel mai mult context.

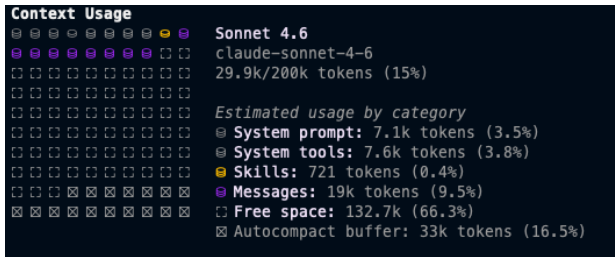


Figura 2.1: Reprezentarea ferestrei de context într-o conversație în Claude Code

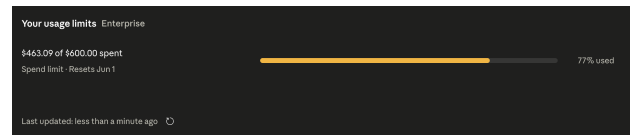


Figura 2.2: Reprezentarea utilizării în Claude (variantea web) (sursa)

Astfel, în implementarea acestui asistent de programare, observabilitatea va juca un rol important pentru transparenta cu utilizatorul. Acesta poate interoga orice metrica disponibilă în legătura cu consumul de tokeni și funcționarea sistemului în timp real folosind Langfuse cat și panoul web inclus cu aplicația.

2.4 Securitate și confidențialitate

Din punctul de vedere al datelor utilizatorului și confidențialitate utilizarea aplicației oferă confidențialitate totală. Datele acestuia nu părăsesc dispozitivul decât dacă acesta optează pentru modele ale furnizorilor sau dacă își configurează Langfuse pentru observabilitate.

În ceea ce privește securitatea și integritatea sistemului, programul va avea mai multe măsuri, opționale sau obligatorii care se asigură ca agentul nu poate realiza acțiuni cu totul distructive. Câteva din aceste sisteme sunt: rularea comenzilor într-un container pe principiul de sandboxing, gardarea logica a apelării uneltelor (protejarea apelării secvențiale a uneltelor, de exemplu trebuie să citească un fișier înainte să scrie în el), și gardarea ieșirilor contra comportamentelor malițioase prin cererea de aprobare a utilizatorului.

2.5 Cost

Costul este minim pentru asistentul de programare dezvoltat. Costul de utilizare este gratuit. De asemenea, dacă utilizatorul alege să opteze pentru modele open source, singurele costuri pe care va trebui să le suporte pe lângă achiziționarea unui dispozitiv capabil să ruleze aceste modele sunt curentul electric și internetul.

Mai mult, aplicația este optimizată pentru a reduce consumul de tokeni, deci și în cazul folosirii furnizorilor de modele, costul va fi unul pe cât se poate de redus, lăsând la o parte potențialele taxe puse de aceștia pe utilizarea modelelor în aplicații terțe.

2.6 Aplicații OSS concurente

Pe lângă aplicațiile furnizorilor exista și câteva aplicații open source care s-au bucurat de o oarecare adopție, regăsite în tabelul 2.2.

Tabela 2.2: Top 6 asistenți open source după numărul de "stele" pe GitHub

Aplicație	Audienta și scop	Puncte forte	Puncte slabe	Funcționalități cheie
OpenClaw (GitHub) Licență MIT	Utilizatori care doresc un asistent IA personal, always-on, accesibil pe orice canal de comunicare	• Cel mai popular proiect open source din categorie • Suport multi-canal masiv (WhatsApp, Telegram, Slack, Discord, iMessage, Signal etc.) • Sandboxing cu Docker, Secure Shell și OpenShell • Aplicații native macOS, iOS, Android • Agnostic față de furnizor, gateway local-first	• Orientat spre asistent personal, nu exclusiv programare • Configurare complexa pentru multi-canal • Necesita Node.js 22+ și infrastructura gateway • Nu are integrare directa în IDE	Gateway local, multi-canal, sandboxing, voice wake, live canvas, cron, skills, multi-agent, apps native
OpenHands (GitHub) Licență MIT	Dezvoltatori și echipe enterprise care necesita un agent autonom complet cu izolare securizata	• Sandboxing în containere Docker • GUI web și CLI disponibile • SDK Python pentru construcția de agenți personalizați • Suport Kubernetes și enterprise • Agnostic față de furnizor (Claude, GPT, Ollama etc.)	• Necesita Docker • Configurare inițială complexa • Nu are integrare directa în IDE • Curba de învățare mai abrupta	Sandboxing, SDK, GUI web, CLI, Kubernetes, integrări Slack/Jira/Linear
Cline (GitHub) Licență Apache 2.0	Dezvoltatori care prefera lucrul direct din IDE cu control granular și automatizări avansate	• Extensie VS Code și plugin JetBrains • SDK Node.js și CLI headless pentru CI/CD • Echipe multi-agent și agenți programați • Suport MCP și plugin-uri extensibile • Agnostic față de furnizor (200+ modele via OpenRouter)	• Consum ridicat de tokeni datorită contextului mare • Fără sandboxing nativ incorporat • Complexitate ridicata, multe funcționalități • Dependent de ecosistemul VS Code/JetBrains	Extensie IDE, SDK, CLI, multi-agent, MCP, agenți programați, integrări Slack/Discord/Telegram
Goose (GitHub) Licență Apache 2.0	Utilizatori generali și dezvoltatori care doresc un agent IA nativ, performant și extensibil	• Aplicație nativa desktop (macOS, Linux, Windows) • Construit în Rust deci are o performanță ridicata • Suport MCP (70+ extensii) • Agent general, nu doar pentru cod • Agnostic față de furnizor (15+ furnizori, suport Agent Communication Protocol)	• Mai puțin specializat pe programare • Ecosistem de extensii mai tânăr • Nu are integrare directa în IDE • Documentație în curs de maturizare	Aplicație nativa, CLI, API, MCP, Agent Communication Protocol, Rust, multi-platform, Linux Foundation
Aider (GitHub) Apache 2.0	Dezvoltatori individuali care prefera un workflow minimalist din terminal cu integrare Git	• Mapare automata a codului sursa (repo map) • Integrare Git nativa cu commit-uri automate • Suport 100+ limbaje de programare • Linting și testare automata • Agnostic față de furnizor (Claude, GPT, DeepSeek, Ollama)	• Fără sandboxing • Exclusiv terminal, fără GUI • Capabilități agentice limitate față de concurența • Nu are extensie nativa de IDE	Terminal, Git auto-commit, repo map, voice-to-code, linting, modele locale

Tabela 2.1: Produse IA ale principalilor furnizori de modele de limbaj

Companie	Produs	Descriere	Audienta	Preț
Anthropic	Claude (web/app)	Asistent IA conversațional pentru analiza, scriere, programare și raționament, cu suport multi-modal și artefacte interactive	Utilizatori generali, profesioniști	Freemium (Pro: \$20/luna, Max: \$100–200/luna)
	Claude Code	Agent de programare agentic care rulează în terminal, Integrated Development Environment (IDE) sau Slack; poate naviga coduri mari, crea PR-uri și rula teste automat	Dezvoltatori software	Inclus în Pro/Max (API: cost per token)
	Claude Cowork	Agent agentic de productivitate care lucrează cu fișiere locale și aplicații cloud (Slack, Google Drive) pentru a automatiza sarcini recurente	Profesioniști, echipe	Inclus în Pro/Max
	Claude Design	Instrument de prototipare vizuala integrat în Claude, care generează interfețe GUI interactive și mockup-uri pe baza descrierilor în limbaj natural	Designeri, dezvoltatori	Inclus în Pro/Max
Google	Gemini (web/app)	Asistent IA conversațional multi-modal cu acces la căutare Google, extensii și generare de imagini; disponibil pe web, Android și iOS	Utilizatori generali	Freemium (Advanced: \$19.99/luna în Google One AI Premium)
	Gemini CLI	Agent de programare open-source (Apache 2.0) care rulează în terminal, cu suport Model Context Protocol (MCP), căutare Google și context de 1M tokeni	Dezvoltatori software	Gratuit (60 req/min, 1000 req/zi cu cont Google)
	NotebookLM	Asistent de cercetare alimentat de Gemini care analizează documente încărcate (Portable Document Format (PDF), site-uri, video-uri) și generează rezumate, podcast-uri audio și conexiuni între surse	Cercetători, studenți	Gratuit (Plus: \$7.99/luna, Business: contact)
	Vertex AI	Platforma enterprise pentru construcția, antrenarea și implementarea de agenți IA și modele Machine Learning (ML), cu acces la 200+ modele și instrumente Machine Learning Operations (MLOps)	Echipe enterprise, ingineri ML	Plătit (pay-as-you-go, de la \$0.0001 per operație)
	Firebase Studio	Mediu de dezvoltare cloud (IDE) cu agenți IA integrați pentru construcția de aplicații full-stack (backend, frontend, mobil) direct din browser	Dezvoltatori software	Gratuit în previzualizare (3 workspace-uri publice; Pro: se include în 2027)
OpenAI	ChatGPT (web/app)	Asistent IA conversațional cu suport multi-modal, căutare web, generare de imagini, analiza de date și plugin-uri; disponibil pe web, iOS, Android și desktop	Utilizatori generali, profesioniști	Freemium (Plus: \$20/luna, Pro: \$200/luna)
	Codex	Agent de inginerie software cloud care executa sarcini de programare în paralel, în sandbox-uri izolate preîncărcate cu codul din repository-ul GitHub	Dezvoltatori software, echipe	Inclus în ChatGPT Pro/Business/Enterprise (API: \$1.50/\$6 per MTok)
Microsoft/ GitHub	GitHub Copilot	Agent de programare integrat în GitHub și disponibil ca aplicație CLI, extensie IDE (VS Code, JetBrains) și în github.com; oferă completare de cod, chat, revizuire PR-uri și rezolvare automata de issue-uri	Dezvoltatori software, echipe	Freemium (gratuit pentru repo-uri publice; Pro: \$10/luna, Business: \$19/luna, Enterprise: \$39/luna)

Capitolul 3

IMPLEMENTARE

3.1 Mediul de dezvoltare

Expunerea elementelor constitutive disponibile care alcătuiesc mediul hardware cat și software în care s-a realizat obiectul acestei lucrări este imperativa pentru relatarea procesului de dezvoltare și reproducerea pașilor prin care s-a ajuns la produsul final. De asemenea în acest capitol vom prezenta și cerințele hardware și software pentru rularea aplicației.

Cerințele hardware pentru dezvoltarea și rularea acestei aplicații sunt minime. În cazul în care folosim modelele din cloud este necesar doar un laptop sau computer ce poate rula Python 3 cu conexiune activa la internet. Deoarece inferență nu este realizata local, nu este necesar un GPU performant sau o cantitate mare de Random Access Memory (RAM). Dacă se dorește rularea unui model local, cerințele hardware depind de modelul ales, dar cresc exponențial necesitând acces la o placa video moderna și o cantitate semnificativa de RAM.

Tabela 3.1: Hardware recomandat pentru modele open-source

Model	Parametri	GPU Recomandat	Video Random Access Memory (VRAM)	RAM
LLaMA 3	70B	2x NVIDIA A100 80GB sau 2x RTX 4090	40–160 GB (cuantizat vs FP16)	64–128 GB
Mistral	7B	NVIDIA RTX 3060 / RTX 4090	4–16 GB	8–32 GB
Qwen 2.5 (14B)	14B	NVIDIA RTX 3090 / RTX 4090	16–24 GB	16–64 GB
Gemma 4 (E4B)	4.5B efectivi (8B total)	NVIDIA RTX 3060 / RTX 4060	4–16 GB (cuantizat vs BF16)	8–32 GB

Concret, pentru dezvoltare s-a folosit un laptop MacBook Pro cu un chip Apple M5 Max și 48GB de memorie RAM.

Din punct de vede software, pentru dezvoltare s-a folosit un sistem de operare macOS Tahoe 26.4.1 cu Python 3.10 instalat.

3.2 Structura

Structura aplicației reflecta separarea responsabilităților în mai multe directoare specializate. Directorul `src` conține logica agentului: graful LangGraph, uneltele, sistemul de permisiuni,

clientul MCP, pipeline-ul de compactare, sub-agentii, sandbox-ul Docker și tracing-ul. Directorul `ui` conține interfața cu utilizatorul: bucla principală, comenzile slash, sistemul de skill-uri, gestionarea sesiunilor și preferințelor. Directorul `web` conține un dashboard analitic Flask care vizualizează sesiunile, consumul de tokeni și activitatea uneltelor. Directorul `eval` conține un runner de evaluări cu metrice precum Bilingual Evaluation Understudy (BLEU), Recall-Oriented Understudy for Gisting Evaluation (ROUGE) și potrivire semantica. Documentația detaliată per-subsistem se găsește în `docs`, iar configurarea implicită în `config/config.yml`.

<code>vibe-cli/</code>	<code>src/</code>	<code>config/</code>
- <code>config/</code>	- <code>main.py</code>	- <code>config.yml</code>
- <code>docs/</code>	- <code>tools.py</code>	\ - <code>eval.yml</code>
- <code>eval/</code>	- <code>tool_guards.py</code>	
- <code>examples/</code>	- <code>providers.py</code>	<code>ui/</code>
- <code>scripts/</code>	- <code>permissions.py</code>	- <code>app.py</code>
- <code>src/</code>	- <code>sandbox.py</code>	- <code>ui.py</code>
- <code>tests/</code>	- <code>mcp_client.py</code>	- <code>commands.py</code>
- <code>ui/</code>	- <code>artifacts.py</code>	- <code>skills.py</code>
- <code>web/</code>	- <code>attachments.py</code>	- <code>sessions.py</code>
- <code>ARCHITECTURE.md</code>	- <code>capabilities.py</code>	- <code>preferences.py</code>
- <code>FIRST_START.md</code>	- <code>clipboard.py</code>	- <code>provider_handler.py</code>
- <code>HEADLESS_USAGE.md</code>	- <code>log_context.py</code>	- <code>pastes.py</code>
- <code>Makefile</code>	- <code>logging_setup.py</code>	- <code>paths.py</code>
- <code>pyproject.toml</code>	- <code>multimodal.py</code>	- <code>profile.py</code>
- <code>README.md</code>	- <code>tracing.py</code>	- <code>config_init.py</code>
- <code>UPDATING.md</code>	- <code>compaction/</code>	- <code>vibe_md.py</code>
\ - <code>uv.lock</code>	\ - <code>subagents/</code>	\ - <code>updater.py</code>

Figura 3.1: Structura directoarelor vitale ale proiectului

3.3 Configurare

Configurarea aplicației se face prin intermediul fișierului `config.yml` care conține catalogul de furnizori (Ollama, OpenAI, Claude, Amazon Bedrock, Gemini), modelele disponibile pentru fiecare, pipeline-ul de compactare, setările de tracing, configurația sandbox-ului sau a subagenților și a uneltelor. Cheile API sunt declarate ca variabile de mediu sau în fișierul de preferințe și sunt încărcate la pornire. Dacă nici o cheie nu este setată și Ollama nu este accesibil, aplicația solicită interactiv introducerea unei chei la pornire prin comanda `/model` sau `/key`.

De asemenea pentru procesul de evaluare al capabilităților folosit pentru teste de sanitate și regresie, se pot modifica setările din fișierul `./.vibe-cli/eval/eval.yml`

Preferințele utilizatorului (provider-ul implicit, modelul selectat per provider, nivelul de efort, serverele MCP, listele de permisiuni) sunt persistate în `~/.vibe-cli/preferences.json` și supraviețuiesc între sesiuni. Sesiunile de conversație sunt salvate ca log-uri de evenimente în `~/.vibe-cli/sessions/`, sortate per proiect (directorul de lucru).

3.4 Interfața cu utilizatorul

Interfața servește ca un mediu de legătura între utilizator și agent, dar și ca un spațiu de configurare al aplicației. Aceasta apelează funcții ale agentului în același proces Python, comunicarea fiind prin apeluri de funcții, nu prin rețea.

În dezvoltarea interfeței, am pornit folosind soluția oferită de librăria Textual. La finalul primului prototip interfața arăta modern și funcționa corect. Însa, pe măsura ce am populat conversația cu asistentul, am observat că funcționalitatea de scroll a ferestrei nu funcționa corect, sărind către partea de sus a ecranului. După ce m-am interesat mai atent la soluțiile implementate de alte programe asemănătoare precum Claude Code, am observat că acestea nu folosesc un TUI propriu zis, ci doar scriu în terminal folosind STDOUT, lăsând astfel scroll-ul nativ al terminalului să se ocupe de istoricul conversației. Am decis astfel să abandonez soluția

cu Textual și să implementez o interfață bazată pe **prompt_toolkit** și **Rich** care să scrie în terminal, oferind astfel o experiență mult mai fluidă și fără probleme de scroll sau lag.

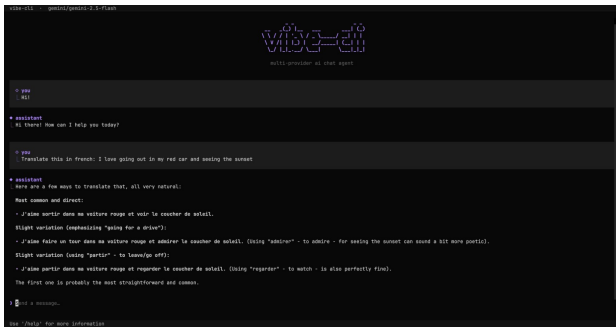


Figura 3.2: Prima versiune a interfeței, realizată cu ajutorul librăriei Textual

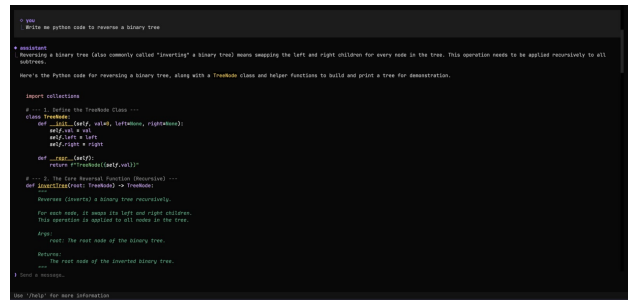


Figura 3.3: Continuarea conversației cu prima versiune a interfeței, realizată cu ajutorul librăriei Textual

În final, a fost o decizie bună să abandonez implementarea cu Textual, deoarece am reușit să produc rezultate mai bune folosind terminalul clasic. Soluția finală este estetică, simplă, funcțională și performantă.

Schimbarea nu s-a produs într-un stadiu avansat al dezvoltării, ci chiar de la primele prototipuri de interfață, lucru care nu a întârziat cu mult implementarea.

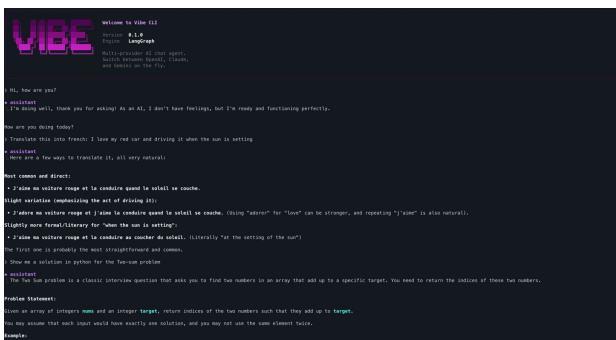


Figura 3.4: A doua versiune a interfeței, realizată fără librăria Textual

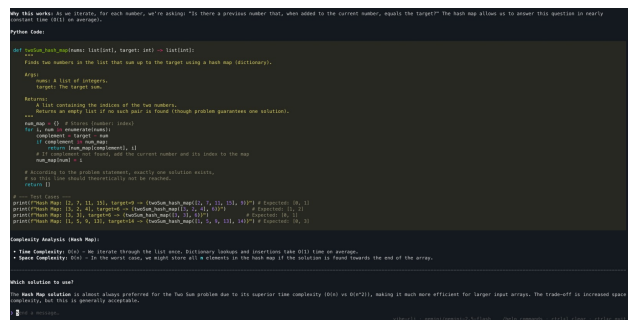


Figura 3.5: Continuarea conversației cu a doua versiune a interfeței, realizată fără librăria Textual

După înlocuirea implementării, am continuat să adaug funcționalități pentru a îmbogăți experiența utilizatorului. Interfața oferă mai multe elemente vizuale menite să informeze utilizatorul în timp real despre activitatea agentului:

- **Spinner animat cu eticheta de fază și cronometru:** textul spinner-ului reflecta faza curentă și măsoară timpul scurs, de exemplu `◊ thinking... 1.2s` în timpul inferenței LLM, sau `◊ Read(README.md)...` 0.4s în timpul execuției unei unelte
- **Log persistent de unelte:** fiecare apel de unealtă lasă o linie $\leftrightarrow$ `Tool(arg)` în istoricul vizibil, astfel încât și uneltele care se execută rapid sunt vizibile
- **Panouri de diff pentru write_file:** apelurile reușite de scriere sunt redactate ca un diff unificat într-un panou Rich cu numărul de linii adăugate și șterse
- **Paste folding:** textele lipite mai lungi de 200 de caractere sunt înlocuite în prompt cu un placeholder scurt `[Paste #N <len> chars]` pentru lizibilitate, conținutul original fiind restaurat înainte de trimitere.

- **Footer de stare:** o linie care indica durata turei, setările active, atașamente active, și informații legate de context.

Pe lângă elementele vizuale, funcționalitate principale includ:

- Unelte inteligente de interacțiune cu mediul virtual (creare/editare/citire fișiere, căutare web, executare comenzi în terminal) care instruiesc modelul de limbaj să le folosească într-un mod inteligent din punct de vedere al consumului ferestrei de context.
- Gardarea apelului logic al uneltelor. Unele au anumite restricții pentru a asigura folosirea corectă. De exemplu, modelul nu poate apela unealta `edit_file` fără ca mai întâi să apeleze unealta `read_file`. De asemenea sunt gardări de citire a fișierelor. Modelul nu poate citi un fișier întreg pentru a garanta ca nu își va umple toată fereastra de context cu output-ul acelei unelte.
- Alegerea modelului din 5 provideri (Ollama, OpenAI, Claude, Bedrock, Gemini) și a efortului de gândire. Furnizorii de modele de limbaj nu sunt limitați și lista poate crește prin extinderea clasei de baza `BaseProvider`.
- Suport pentru intrări multi-modale (imagini, fișiere PDF, fișiere text și documente), atașate în prompt prin cale locală sau Uniform Resource Locator (URL), atunci când modelul selectat oferă capacități multi-modale (vezi tabelul 4.5); pentru modelele exclusiv textuale, atașamentele sunt respinse cu un mesaj explicativ.
- Gestionarea sesiunilor cu posibilitatea de a salva, șterge, lista sau continua conversații anterioare. Fiecare proiect include istoricul propriu al conversațiilor.
- Sistemul de permisiuni per-unelte cu liste de `always-allow` și `always-deny` persistente
- Conectarea cu servere MCP prin transport Standard Input-Output (STDIO), Hyper-Text Transfer Protocol (HTTP) sau Server-Side Events (SSE)
- Skill-uri bazate pe fișiere Markdown cu încărcare progresivă (doar numele și descrierea în prompt-ul de sistem, corpul încărcat la cerere)
- Sub-agenți specializați extensibili (câțiva agenți de baza precum: general, code search, web research, dar pot fi adăugați mai mulți sub-agenți prin extinderea clasei `Subagent`) care rulează în paralel cu modele configurabile per-agent.
- Pipeline de compactare a contextului cu mai multe stadii (trunchiere, sumarizare, deduplicare) configurabil.
- Sandbox Docker opțional pentru execuția izolată a comenzilor
- Tracing prin Langfuse Cloud pentru observabilitate
- Dashboard web analitic (Flask) cu vizualizare de metrice, cost și activitate
- Auto-completare pentru comenzile slash și actualizare automată a versiunii

3.5 Instalare și utilizare

3.5.1 Cerințe preliminare

Aplicația necesită Python 3.10 sau o versiune ulterioară și este disponibilă pe sistemele de operare Linux și macOS; instalarea pe Windows nu este suportată. Pentru accesul la modelele de limbaj este necesar fie un server Ollama local pornit prin `ollama serve`, fie cel puțin o cheie API validă pentru unul dintre furnizorii cloud (`OPENAI_API_KEY`, `ANTHROPIC_API_KEY` sau `GOOGLE_API_KEY`). În lipsa oricărei configurații prealabile, aplicația oferă la pornire un flux interactiv pentru selecția furnizorului și introducerea cheii.

3.5.2 Instalare

Metoda recomandată este executarea script-ului oficial de instalare, care detectează și instalează la nevoie gestionarul de pachete `uv`, după care obține aplicația direct din proiectul găzduit pe GitHub:

```
curl -fsSL https://raw.githubusercontent.com/Moshulika/vibe-cli/main/scripts/install.sh | bash
```

Alternativ, instalarea se poate efectua manual prin `uv tool install` sau `pipx install`, indicând proiectul GitHub ca sursă. Dezinstalarea se face prin `uv tool uninstall vibe-cli`. Pentru dezvoltare, după clonarea proiectului, comanda `make dev-setup` instalează dependențele și hook-urile `pre-commit`, iar `make ui` lansează interfața de chat. Sunt expuse de asemenea ținte `make` pentru rularea suitei de teste (`make test`), formatarea codului (`make format`) și pornirea dashboard-ului analitic web (`make web`).

3.5.3 Configurare la prima rulare

După instalare, comanda `vibe` este disponibilă global din orice director. La prima pornire, aplicația creează arborele de configurație sub `~/.vibe-cli/`, ce conține fișierul de preferințe `preferences.json` (furnizorul implicit, modelul selectat per furnizor, nivelul de efort, cheile API și permisiunile per unealta), un identificator anonim `profile.json`, directorul cu skill-uri exemplificative și log-urile structurate. Furnizorii configurați anterior sunt validați automat printr-un apel ușor către endpoint-ul fiecăruia, fără a consuma tokeni; cheile invalide sunt șterse și semnalate utilizatorului. În absența unei configurații valide, un flux interactiv solicită selecția furnizorului și introducerea cheii API (cu mascarea intrării), validând-o înainte de salvare.

3.5.4 Utilizare

După configurare, utilizatorul interacționează cu modelul printr-o sesiune persistentă de chat, salvată automat în evenimente granulare sub directorul corespunzător proiectului curent. Comenzile pot fi rulate în orice moment al conversației și au formatul `/comanda`, cu auto-completare la apăsarea tastei Tab. Sunt disponibile și scurtături de tastatură precum `CTRL+L` pentru a goli conversația și `CTRL+C` pentru închiderea aplicației cu salvarea automată a sesiunii. Pe lângă intrarea textuală, utilizatorul poate atașa imagini, fișiere PDF, fișiere text sau documente, transmise nativ modelului atunci când acesta dispune de capabilități multi-modale. Dashboard-ul web analitic poate fi pornit separat prin `vibe web`, expus implicit pe portul 5050, și oferă vizualizări pentru consumul de tokeni, costuri, latente, evenimente de compactare și activitatea sub-agentilor.

Comanda	Descriere
/help	Listarea tuturor comenzilor și a scurtăturilor de tastatură
/model	Selectarea provider-ului și modelului, sau setarea cheii API
/key	Setarea sau actualizarea cheii API per provider
/effort	Selectarea nivelului de efort al modelului (low/medium/high)
/mcp	Configurarea serverelor MCP (adăugare/stergere/activare)
/skills	Listarea, activarea sau reîncărcarea skill-urilor
/subagents	Listarea tipurilor de sub-agenți și configurarea modelelor per tip
/plan	Activarea/dezactivarea modului de planificare
/agent	Activarea/dezactivarea modului agentic
/permissions	Vizualizarea și editarea listelor de permisiuni per unealta
/clear	Ștergerea conversației curente și rotirea la o sesiune nouă
/sessions	Listarea sesiunilor salvate pentru proiectul curent
/resume	Continuarea ultimei sesiuni sau a uneia specifice prin ID
/compaction	Inspectarea pipeline-ului de compactare și selectarea modelului
/compact	Forțarea unei compactări imediate a contextului
/tracing	Configurarea și activarea tracing-ului Langfuse
/score	Evaluarea ultimei interacțiuni pentru Langfuse (good/bad)
/attach	Atașarea datelor multi-modale de tip imagini, PDF sau fișiere text ca și cale de sistem
/paste	Atașarea datelor multi-modale de tip imagini, PDF sau fișiere text din clipboard
/dataset	Adăugarea ultimei interacțiuni la un dataset Langfuse
/update	Actualizarea la ultima versiune disponibilă în GitHub
/exit	Salvarea conversației și închiderea aplicației

Tabela 3.2: Comenzi disponibile în interfață

3.5.4.1 Modul headless

Pe lângă sesiunea interactivă, aplicația expune un mod *headless*, neinteractiv, activat prin parametrul `-p/-prompt` (cu suport pentru citirea promptului din `stdin` prin `vibe -p -`). Acesta execută o singură tură (un apel către model urmat de bucla de procesare a uneltelor) și tipărește rezultatul final, în format text sau `json` (`-o text|json`), după care iese cu un cod conform convențiilor Portable Operating System Interface (POSIX): 0 la succes, 1 la eroare de execuție, 2 la argumente invalide și 130 la întreruperea prin `SIGINT`. Modul este *hermetic* în mod implicit: dezactivează MCP, salvarea sesiunii, tracing-ul și interacțiunile cu utilizatorul, cu opțiunea de a salva sesiunea prin `-save-session`. Autentificarea se realizează automat prin detectarea variabilelor de mediu corespunzătoare furnizorilor suportați (`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `GOOGLE_API_KEY`, `BEDROCK_API_KEY`, `OLLAMA_URL`), scurtcircuitată de flag-ul `-provider`.

Sunt expuse de asemenea opțiuni pentru controlul fin al execuției: selecția modelului (`-model`), reglarea efortului de gândire prin `-effort` (low/medium/high), modificarea promptului de sistem (`-system` pentru înlocuire, exclusiv cu `-append-system-prompt` pentru extindere), restricționarea uneltelor disponibile (`-allowedTools`) și limitarea bugetului de iterații (`-max-turns`). Ieșirea `json` include furnizorul, modelul, răspunsul final, lista apelurilor de unelte, skill-urile utilizate, statistici de consum, numărul de iterații și durata execuției, facilitând integrarea în pipeline-uri automatizate de tip CI/CD sau cu wrappere ¹.

¹Un wrapper este o aplicație care înglobează o altă aplicație și folosește output-ul acesteia pentru a-l prelucra în continuare

3.6 Gestionarea contextului și a memoriei

Aplicațiile ce utilizează principii agentice au nevoie de o logica complexa pentru management-ul contextului și al memoriei din doua motive diferite: personalizare și consistenta conversației.

3.6.1 Memoria și istoricul conversației

Memoria este folosita pentru a extrage informații și preferințe pe termen lung. Câteva întrebări sunt extragerea de informații cu legătură la mediul de lucru și preferințe specifice acestuia. Majoritatea aplicațiilor existente pe piață folosesc o tactica asemănătoare pentru stocarea acestor metode:

1. Un folder ascuns (`.claude` în cazul Claude Code, `.copilot` în cazul GitHub Copilot, `.codex` în cazul Codex) care conține preferințe în legătura cu modul de lucru în cadrul proiectului, precum și rutine sau capacități specifice lui în format text, pe care agentul le citește și le utilizează după bunul plac.
2. Pentru preferințele globale ce au legătura cu personalitatea utilizatorului și modul de lucru preferat de acesta, sau indicații și decizii pe care ar trebui să le ia în anumite situații, exista uzual un folder similar cu cel local pentru proiecte, doar ca într-o locație globala, ce conține de asemenea un mic rezumat text pe care agentul îl extrage automat în timpul utilizării și îl folosește în viitoarele acțiuni.
3. Istoricul conversațiilor este stocat de obicei local, pe dispozitivul utilizatorului. Fiecare conversație este atribuita unui proiect (director distinct în funcție de calea pe care se afla acesta în sistemul utilizatorului), căreia îi este atribuit suplimentar un identificator unic. Acest istoric conține fiecare prompt pe care utilizatorul îl trimite către modelul de inteligența artificială, răspunsul pe care agentul îl da și toate apelurile către unelte și alte operațiuni de sistem. Acest istoric este reintrodus integral în contextul agentului dacă se reia conversația. Un dezavantaj în acest caz, este ca uzual memoria este salvata în doua locuri diferite:
 - Memoria contextuala reprezintă o limitare a modelelor actuale. Aceasta este memoria care este inserata în model cu fiecare prompt. Modelele funcționează cu o fereastră maxima de context. În cazul conversațiilor lungi cu multe unelte utilizate de către agent sau cu multiple fișiere (sau doar de o mărime foarte mare) video/audio sau alte tipuri de documente, aceasta fereastră poate fi epuizata rapid, necesitând un procedeu numit compactarea contextului. Prin aceasta compactare, se sumarizează sau se șterg efectiv părți neimportante din conversație pentru a păstra aceasta fereastră de context în limite normale. Depășirea ferestrei rezulta în nefunctionarea modelului deoarece acesta nu va accepta cererea de interacțiune.
 - Memoria integrala este stocata uzual separat față de memoria contextuala și conține tot istoricul conversației pe care utilizatorul îl poate consulta (conține text, răspunsuri și artefacte încărcate), neafectat.

În implementare, s-au folosit metode standard din industrie:

- Utilizează un singur fișier pentru stocarea memoriei cu doua câmpuri pentru fiecare eveniment. Unul pentru evenimentul original, unul pentru cel compactat.
- Preferințe locale și globale. Pe măsura ce utilizatorul folosește agentul, acesta își notează preferințele acestuia și le ia în considerare în cererile viitoare.
- Toate acestea sunt stocate local pentru păstrarea intimității utilizatorului.

3.6.2 Contextul

Contextul este unul dintre punctele de interes pentru dezvoltarea sistemelor agentice datorită implicațiilor pe care le are în executarea sarcinilor complexe și de lungă durată sau cu cantități mari de date.

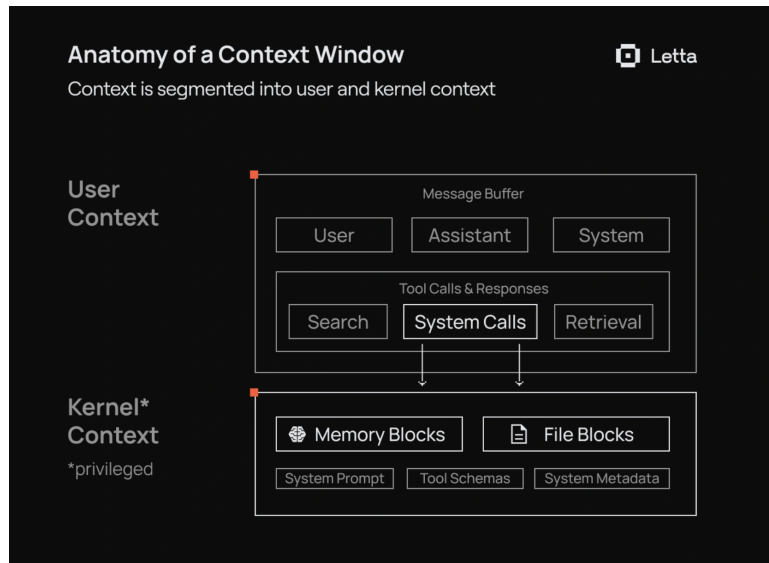


Figura 3.6: Împărțirea ferestrei de context (sursa)

Mai multe structuri se luptă pentru o parte din fereastra de context:

- Promptul de sistem - instrucțiuni pentru comportamentul agentului. Conține printre altele rolul pe care acesta îl joacă, restricții comportamentale, exemple de răspunsuri și capacități accesibile.
- Mesajele utilizatorului - prompt-ul pe care utilizatorul îl trimite către agent.
- Răspunsurile asistentului - răspunsul oferit către utilizator la cererea acestuia. Întotdeauna contextul declarat (spre ex. 200k tokeni) va avea mărimea adevărată $C = C_{\max} - O_{\max}$, unde C este contextul adevărat, $C_{\max}$ contextul declarat, și $O_{\max}$ numărul maxim de tokeni de ieșire suportați de model. Din aceasta valoare se scad și alte valori rezervate de sistem, precum spațiul tampon pentru compactare.
- Atașamente - documente, imagini, audio și videoclipuri
- Skill-uri, sau alte instrucțiuni - descrierea și titlul tuturor skill-urilor sau a rutinelor salvate care pot fi folosite de către agent sunt injectate în context pentru ca agentul să poată alege în cunoscuta de cauza un skill sau o rutină din cele disponibile.
- Unelte - similar skill-urilor, semnătura uneltelor, descrierea acestora, datele de ieșire și instrucțiunile lor sunt disponibile modelului prin injectarea în context. Se aplică atât uneltelor locale sau celor expuse prin MCP.
- Output-ul uneltelor - datele de ieșire ale uneltelor, spre exemplu conținutul unui fișier rezultat din utilizarea unei unelte pentru citirea fișierelor poate fi injectat direct în context. În cazul fișierelor mari, acest lucru poate însemna o problemă pentru fereastra de context.

- Căutări pe internet - căutările pe internet returnează răspunsuri Java Script Object Notation (JSON) sau Hyper-Text Markup Language (HTML) care uzual nu conțin strict date punctuale, ci și întregul conținut al paginii. Acest lucru ajută pentru poziționarea elementelor în pagina dar adaugă multe caractere ce umplu fereastra de context.

3.6.2.1 Retrieval Augmented Generation (RAG)

Pentru conservarea contextului în lucrul cu documente de mărime mare, sau cu colecții de documente, a fost adoptată tehnica RAG [29]. Acesta se bazează pe un proces independent care ingestează documentele de interes prin vectorizare și partiționare semantică prin care agentul poate căuta cu ușurință, ajungând astfel să extragă dintr-un document de dimensiuni mari doar bucăți de interes.

Dezavantajul acestei tehnici este complexitatea infrastructurii necesare. În primul rând este nevoie de un serviciu separat care să proceseze și să stocheze aceste date, iar în final modelul trebuie să poată accesa aceste date într-un mod eficient.

Odată cu progresele aduse mărimii acestei ferestre de context, modelele de vârf ajung să aibă până la unul sau două milioane de tokeni ca fereastra de context. Astfel, nevoia pentru RAG scade. Pentru majoritatea întrebunțărilor, aceasta mărime a ferestrei este mai mult decât suficientă, chiar și pentru conversații mai complexe, în ciuda limitărilor cunoscute precum scăderea performanțelor pe date ce se afla în mijlocul ferestrei de context [13].

În implementarea actuală *nu au fost folosite tehnici RAG* din cauza complexității de implementare, a infrastructurii necesare și a beneficiilor estompate de performanțele curente ale modelelor de limbaj.

3.6.2.2 Gestionarea datelor multi-modale

Data fiind alegerea de abandonare a tehnicii RAG, în implementare avem nevoie de o modalitate inteligentă de a prelucra datele multi-modale pentru conservarea contextului.

Prima problemă care apare în lucrul cu date multi-modale apare din cauza necesității pentru suportul unei plaje foarte largi de modele de limbaj. Datorită arhitecturii agnostice de furnizor care permite folosirea oricărui model, avem nevoie să detectăm capacitățile de prelucrare a tipurilor de fișiere pentru fiecare model în parte. Din păcate, furnizorii de modele sau creatorii care le includ în Ollama, nu specifică nicăieri consistent în metadatele modelului dacă acesta prezintă capacități pentru lucrul cu anumite tipuri de fișiere.

Din acest motiv, am avut de ales între două soluții de implementare:

- Punerea utilizatorului în situația în care trebuie să se documenteze și să declare în configurație capacitățile modelului pe care dorește să îl folosească
- Încercarea automată de trimitere de date multi-modale către model, iar în cazul în care acesta refuză datele, să bifăm în fișierul de preferințe faptul că acest model nu suportă respectivul tip de date.

Din motive legate de experiența utilizatorului am ales să continuiam cu a doua soluție.

La finalul deciziei legate de detectarea capacităților, a trebuit să lucrăm la gestionarea contextului pentru aceste date. Am ales astfel o abordare hibridă, compusă din două ramuri pe care agentul poate alege să le apeleze. În prima fază pentru fișierele de tip text sau PDF, agentul poate citi fișierele în format pur textual folosindu-se de uneltele uzuale de citit fișiere, beneficiind de optimizările acestora, precum folosirea punctuală în zone de interes sau trunchierea datelor de ieșire. Dacă modelul consideră că are nevoie de o vedere completă, pentru fișiere PDF, dacă acesta suportă formatul ca tip de date pentru intrare, se poate uita direct pe întregul fișier prin

mai multe citiri succesive de maxim 20 de pagini (configurabil). Pozele sunt introduse direct în context dacă modelul suporta acest tip de date și sunt limitate la o mărime de 10MB.

Compactarea este realizata prin metode ce rulează imediat după folosirea uneltelor, cum ar fi trunchiere, referentiere sau sumarizare, fișierele fiind stocate automat ca artefacte.

3.6.2.3 Compactarea

Compactarea conversației este un cumul de tehnici ce asigura compatibilitatea conversației curente cu restricțiile de intrare ale modelului. Aceasta este folosita predominant în sistemele agentice, deoarece aici se pot genera cantități mari de date ce epuizează fereastra de context rapid.

Compactarea are loc uzual la produsele existente în piață în mod automat când conversația atinge în jur de 80% din mărimea maxima a ferestrei de context pentru modelul respectiv. Acest lucru oferă o margine de eroare pentru a evita pornirea procesului de compactare pana nu se termina interacțiunea curenta ²

Aceasta este realizata prin mai multe tehnici printre care cele mai utilizate sunt:

- Formatarea rezultatului returnat de unelte - reprezintă acțiuni de formatare deterministe. Un exemplu bun ar fi formatarea output-ului JSON pentru a elimina caracterele nefolositoare precum spațiile.
- Truncarea rezultatului returnat de unelte - reprezintă truncarea efectiva a datelor de ieșire ale uneltelor. Mai exact se păstrează un număr de caractere de la începutul datelor și un număr de caractere de la sfârșit. Restul este șters.
- Sumarizarea rezultatului returnat de unelte - aceasta metoda implica apelul către un model de limbaj care sumarizează datele de ieșire și înlocuiește datele reale cu acest rezumat.
- Substituiția rezultatului returnat de unelte cu referințe - tehnica care salvează rezultatul returnat de unelte în fișiere externe și îl înlocuiește cu un tag pe care modelul îl folosește pentru a accesa fișierul și datele.
- Deduplicarea mesajelor - obiectele care au un scor de similaritate prea ridicat sunt șterse.
- Ștergerea mesajelor de importanta scăzuta - reprezintă ștergerea mesajelor de un anumit tip pre-determinat, de exemplu `tool_call` sau `assistant_thinking`
- Sumarizarea întregii conversații - fiecare mesaj este sumarizat de către un model de limbaj
- Truncarea întregii conversații - ștergerea celor mai vechi mesaje din conversație

Consultând metodele enumerate mai sus, putem observa ca exista o sumedenie de metode de optimizare a istoricului conversației, dar în principal acestea abordează problema datelor de ieșire ale uneltelor. Uzual, acestea se executa imediat după terminarea turei curente. Doar metodele de compactare pentru întreaga conversație se realizează la atingerea pragului de compactare, și se executa pana când mărimea contextului este sub un anumit prag.

În implementarea curenta, adoptam toate metodele de compactare enumerate mai sus. În produsele existente aceste setări nu pot fi modificate, dar în implementarea curenta am ales să

²O interacțiune reprezintă întrebarea utilizatorului și toate procesele pe care le executa agentul pana la producerea răspunsului final

facem **aceste setări configurabile** pentru a studia comportamentul agentului și al istoricului cu diferite strategii de compactare și efectul pe care acesta îl are asupra coerenței acestuia. Configurarea mecanismului de compactare poate fi consultată în figura 3.7.

```

1 compaction:
2   tool_persistence: false
3   enforce_session_model: true
4   trigger:
5     threshold: 0.80
6     min_turns_kept: 4
7   pipeline:
8     - tool_truncate:
9       enabled: false
10      max_tool_tokens: 2000
11      head_tail: [800, 400]
12     - tool_summarize:
13       enabled: false
14       max_tool_tokens: 2000
15       head_tail: [800, 400]
16       timeout_s: 30
17       max_summary_tokens: 500
18     model:
19     - format_compress:
20       enabled: false
21       per_tool: {}
22     - ref_substitute:
23       enabled: false
24       min_ref_tokens: 1500
25       excerpt_tokens: 200
26     - dedupe:
27       enabled: false
28       similarity: 0.92
29       backend: string
30     - importance_prune:
31       enabled: false
32       keep_kinds: [human,
33        system_prompt, assistant_text,
34        tool_result_last]
35       drop_kinds: [
36        assistant_thinking]
37     # Strategies:
38     # - trim - Drop the oldest whole
39     #   turns until size <= target.
40     # Cheapest, no LLM,
41     # deterministic. Loses information
42     # .
43     # - summarize - Drop the oldest
44     #   whole turns AND LLM-summarize
45     #   their
46     # content into one SystemMessage
47     # so the model retains
48     # high-level continuity.
49     # - hybrid - Keep the most
50     # recent 'compaction.trigger.
51     # min_turns_kept'
52     # SystemMessage. Recommended
53     # default.
54     - bulk:
55       enabled: false
56       strategy: hybrid
57       summarize:
58         model: ${COMPACTION_MODEL}
59         max_summary_tokens: 1500
60   storage:
61     backup: true
62     keep_backups: 3
63     artifacts: true

```

Figura 3.7: Configurarea pipeline-ului de compactare a contextului

Nota: Unele strategii de compactare sunt de tipul *eager*, adică rulează imediat cum interacțiunea se termina, iar altele rulează doar la atingerea limitei de context. De asemenea unele strategii nu sunt compatibile între ele.

Compactarea conversației funcționează pe principiul unui sir de execuție configurabil. Acest sir de execuție este pornit când nivelul de ocupare al contextului atinge valoarea `threshold` sau după fiecare interacțiune. Apoi, în funcție de ce pași sunt setați ca fiind porniți, fiecare executa un număr de transformări localizate sau globale care pot fi configurate în secțiunile corespunzătoare. Pentru pașii care necesita conexiunea cu un model de limbaj, acesta poate fi cel principal, sau poate fi ales un model mai mic disponibil. Un mic îndrumar al setărilor poate fi consultat aici:

- `tool_truncate` poate fi configurat pentru a porni la un rezultat al uneltelor mai mare de `max_tool_tokens` și poate fi ales numărul de tokeni păstrați de la începutul și finalul răspunsului.
- `tool_summarize` similar cu unealta anterioara, doar ca în loc să taie cu totul mijlocul răspunsului de la unelte, acesta îl sumarizează cu ajutorul unui model de limbaj configurabil.
- `format_compress` poate specifica căror unelte le este aplicata aceasta transformare și care este pragul.
- `ref_substitute` specifica care este numărul minim de tokeni pentru a extrage răspunsul uneltei într-un artefact extern și a injecta în locul sau un tag.
- `dedupe` conține pragul de similaritate de la care un mesaj este considerat duplicat și metrica după care se calculează similaritatea.
- `importance_prune` conține listele cu tipurile de mesaje care trebuiesc păstrate și cele care trebuiesc șterse în cazul compactării.
- `bulk` conține rutina de sumarizare a întregii conversații cu mai multe strategii.

3.6.2.4 Tehnici pentru eficientizarea ferestrei de context

Exista de asemenea numeroase tehnici care ajuta semnificativ la păstrarea în limite normale a ferestrei de context. Printre acestea se număra utilizarea eficienta a uneltelor, realizata prin utilizarea inteligenta a comenzilor. Spre exemplu, rularea comenzilor în terminal cu argumente ce limitează mărimea output-ului, precum `tail` sau `head`, sau prin comenzi ce găsesc anumite fișiere și fraze punctual precum `grep` sau `find`.

O alta practica pentru conservarea contextului este pornirea de subagenți care realizează cercetarea proiectului sau executa diverse comenzi întorcând doar un rezumat al acțiunilor sau informațiilor pe care le-au aflat către agentul principal. Astfel, contextul agentului orchestrator rămâne curat în timp ce agenții muncitori realizează acțiunile necesare completării sarcinii. Aceasta arhitectura multi-agent permite agentului orchestrator să se concentreze pe logica problemei complexe și să delege sub-sarcini altor agenți, acesta doar interpretând rezultatele acestora.

Ambele tehnici sunt utilizate în implementarea actuala și sunt detaliate în următoarea secțiune.

3.7 Componente ale arhitecturilor agentice moderne

Pentru a avea un flux de lucru agentic, modelul de limbaj trebuie să fie adaptat pentru a urma instrucțiuni ce îl ghidează în a folosi diverse instrumente, capabilități predefinite sau pentru interacțiunea cu o echipa de agenți.

3.7.1 Unelte

Inima arhitecturii agentice este reprezentata de unelte pe care modelele de limbaj învață să le utilizeze. Acestea nu reprezintă nimic mai mult decât metode uzuale ale limbajului de programare cu precizarea ca modelul de limbaj are acces la semnătura acesteia (gruparea de nume al funcției, parametrii de intrare, date returnate) împreună cu o descriere detaliata de utilizare, un exemplu fiind prezent în figura 3.8.

```

1 @tool
2 def list_directory(path: str = ".", show_hidden: bool = False) -> str:
3     """List files and directories at the given path.
4     Args:
5         path: Directory path to list (default: current directory).
6         show_hidden: Whether to include hidden files (default: false).
7     """
8     try:
9         p = Path(path).expanduser().resolve()
10        if not p.exists():
11            return f"Directory not found: {path}"
12        if not p.is_dir():
13            return f"Not a directory: {path}"
14        entries = sorted(p.iterdir(), key=lambda e: (not e.is_dir(), e.name.lower()))
15        lines = [f"Contents of {p}:", ""]
16        for entry in entries:
17            if not show_hidden and entry.name.startswith("."):
18                continue
19            prefix = "[Folder] " if entry.is_dir() else "[File] "
20            size = ""
21            if entry.is_file():
22                s = entry.stat().st_size
23                if s < 1024:
24                    size = f" ({s}B)"
25                elif s < 1_048_576:
26                    size = f" ({s / 1024:.1f}KB)"
27                else:
28                    size = f" ({s / 1_048_576:.1f}MB)"
29            lines.append(f" {prefix}{entry.name}{size}")
30        if len(lines) == 2:
31            lines.append(" (empty)")
32        return "\n".join(lines)
33    except PermissionError:
34        return f"Permission denied: {path}"
35    except Exception as e:
36        return f"Error listing directory: {e}"

```

Figura 3.8: Implementarea unei funcții pe care agentul o folosește pentru a lista directoarele și fișierele la o anumită cale pe sistem

Aceste unelte sunt alese cu grija pentru a menține un echilibru între numărul total de unelte și suita de capabilități necesare agentului.

Numărul de unelte afectează direct modul de gândire al modelului și performanța în executia sarcinilor complexe. Cu prea puține, acesta poate întâmpina probleme în executia problemelor cu adevărat sofisticate deoarece ii poate lipsi o capabilitate necesara. Pe de alta parte, dacă are prea multe acesta nu se va putea decide asupra unei singure unelte, iar în lipsa unei descrieri detaliate crește riscul de folosire greșita sau ineficienta.

De asemenea, pentru o eficiența sporită, scopul acestora nu trebuie să se suprapună. Uneltele trebuie să fie unice, și să aibă un scop clar. Mai mult, este recomandată exemplificarea cazurilor în care o unealta ar trebui folosită, evitata sau preferată în detrimentul alteia. Câteva dintre capabilitățile utilizate frecvent de agenți sunt:

- Listarea directoarelor și fișierelor
- Citirea fișierelor
- Crearea fișierelor cu sau fără conținut
- Editarea fișierelor
- Căutarea pe internet
- Execuția comenzilor în terminal
- Utilizarea skill-urilor
- Pornirea subagenților

Utilizarea acestor instrumente trebuie îngrădită totuși de noțiuni logice. Execuția acestora este determinată de șiruri logice de evenimente care trebuie îndeplinite anterior execuției. Spre exemplu, un agent nu ar trebui să scrie într-un fișier dacă nu l-a citit înainte, deoarece acesta

nu are contextul aceluși fișier și poate returna lucruri care nu sunt ancorate în realitate. Acest lucru este implementat cu ajutorul framework-ului LangGraph printr-un hook ³ ce este apelat înaintea sau la finalul execuției unei unelte. Rezumându-ne la exemplul cu uneltele de citire și scriere din fișiere ilustrat și în fișierul de configurare din care am extras figura 3.9, aceste hook-uri salvează un Secure Hash Algorithms (SHA) ⁴ al fișierului citit în memorie, pe care mai apoi unealta de scriere a fișierului îl compara cu SHA-ul calculat individual. Dacă aceste două coduri coincid, rezulta ca fișierul citit este exact cel care urmează să fie scris, astfel este garantată integritatea contextului înainte de scrierea. Deoarece uneltele sunt un bloc logic determinist, acest lucru garantează verificarea integrității contextului înainte de fiecare apel potențial distructiv. Agentul are acces doar la umplerea parametrilor funcției cu valori arbitrare, dar nu are control asupra logicii efective a uneltei.

```

1 tool_guard:
2   enabled: true # master switch - false
3   read_before_write:
4     enabled: true
5     # When true (default), a freshness check
6     # requires the file's current SHA
7     # to match what was read. When false,
8     # only "ever read in this session"
9     # is required - looser, but skips the
10    # disk read on every write/edit.
11    check_freshness: true

```

Opțiunea de a configura aceste gardări revine utilizatorului ce poate porni sau opri aceasta funcționalitate. Fiind o aplicație OSS, dacă utilizatorul este și dezvoltator software, acesta poate modifica codul sursă și extinde logica de gardare cu mai multe reguli ce pot fi oprite sau pornite din aceasta secțiune.

Figura 3.9: Configurarea gardării execuției uneltelor

Agentul trebuie să utilizeze în mod inteligent aceste unelte. Pe lângă gardarea logica a secvenței de instrumente, acesta trebuie să știe cum poate folosi de ele la potențialul maxim. Un bun exemplu pentru acest argument este execuția de cod scris de agenți în terminal. Pentru opțiuni repetitive care ar consuma o bună parte din context, modelele sunt antrenate să scrie porțiuni mici de cod, uzual în limbajul Python sau Bash datorită simplității, și a disponibilității crescute. Astfel, agentul va scrie aceste bucăți de cod și le va executa în terminal pentru a realiza acțiuni complexe în doar câteva linii de cod, sporind eficiența cercetării și a testării. Aceasta utilizare este fructificată de promptul de sistem care instruește agentul, dar și din procesul de antrenare.

Antrenarea pentru scrierea, folosirea și execuția de cod a devenit o prioritate pentru agenți, modelele de limbaj fiind antrenate explicit folosind RL să folosească secvențe de cod pentru facilitarea rezolvării problemelor care ar beneficia după urma unei astfel de abordări. Aceasta logica este utilizată și pentru testarea de bază a ideilor și explorarea acestora înainte de implementarea propriu zisă, agentul rulând un mic script demonstrativ în terminal pentru a dovedi ca principiul pe care se bazează ideea ce urmează a fi implementată este valid.

3.7.2 Modul de planificare

Planificarea sarcinilor complexe folosește principiul Chain of Thought [9]. Astfel, execuția unei sarcini complexe este împărțită în două părți: planificare și execuție.

³Un hook este un punct de interceptare a execuției unei funcții care permite inserarea altor blocuri logice ce sunt rulate în momentul apelării acestuia

⁴Codarea SHA este cea mai comună metoda de verificare a integrității unui fișier. Este folosit un algoritm care calculează un cod unic în funcție de toate datele conținute de un fișier, astfel orice mică modificare a fișierului schimbă complet codul, fără a fi necesară citirea integrală a acestuia de fiecare dată

În prima faza, agentul realizează cercetarea necesară pentru implementare și documentează totul într-un fișier. Mai apoi, se va folosi de contextul din acel fișier pentru a ține minte toate punctele pe care trebuie să le bifeze. Multe implementări împart planul în câteva secțiuni bine delimitate printre care se regăsesc: ideea și descrierea planului, fișiere afectate, modificări punctuale și cel mai important niște căsuțe pe care acesta trebuie să le bifeze, împărțindu-și o execuție lungă în mai multe sub-sarcini mai mici, individuale și de complexitate redusă.

Acest mod de planificare nu este nimic mai mult decât o schimbare de prompt de sistem într-o combinație cu o antrenare de tip RL care recompensează crearea de planuri detaliate și consistente în situații care necesită așa ceva.

Modul de planificare reprezintă materializarea SDD în contextul programării de către sisteme agentice. O altă abordare folosită în aceste sisteme este aceea de TDD, care permite mai întâi experimentarea și scrierea de cod fără o documentație bine pusă la punct, dar testarea amănunțită la final.

3.7.3 Skills

Skill-urile sau capacitățile nu sunt decât un set predefinit de instrucțiuni pentru realizarea punctuală a unei sarcini repetitive și sunt compuse din două componente care se afla într-un fișier de tip Markdown pentru formatare augmentată.

Prima componentă este în format YAML Ain't Markup Language (YAML), și reprezintă date referitoare la skill, precum numele, descrierea, o recomandare pentru cazuri în care utilizarea acestuia este dorită, și un steag ce setează activarea acestei capacități. Aceste date sunt disponibile agentului și sunt inserate în contextul tuturor conversațiilor pentru ca modelul să aibă acces la lista din care poate alege.

A doua componentă este reprezentată de capacitatea propriu zisă. Aceasta este formată din instrucțiuni scrise în limbaj natural. De obicei aceste instrucțiuni sunt structurate în pași bine definiți, cu reguli și exemple.

```

1 ---
2 name: pr-review
3 description: Structured pull-request review - risk surface, testing, naming, error handling.
4 when_to_use: When the user asks for a code review, PR review, or feedback on a diff.
5 enabled: false
6 ---
7 You are now performing a pull-request review. Walk through the diff and respond with these
  sections, in this order. Skip a section only if there's truly nothing to say there.
8
9 1. Summary - one short paragraph: what does this change do, in plain language.
10 2. Risk surface -- what could break in production. Concurrency, data migrations, error
  paths, rollback story, blast radius.
11 3. Correctness -- bugs, off-by-ones, race conditions, mishandled None/empty cases, wrong
  types, security issues (injection, auth, secrets).
12 4. Tests -- is the change covered. Are the tests testing the right thing. Any missing edge
  cases.
13 5. Style & naming -- readability nits only if they actually hurt comprehension. Don't
  bikeshed.
14 6. Suggestions -- concrete proposed changes, ideally with a snippet.
15
16 Rules of engagement:
17 - Be specific: cite file paths and line numbers.
18 - Distinguish blockers from nits - say so explicitly.
19 - If the diff is too large to review thoroughly, say so and pick the riskiest files first.
20 - No fluff, no praise sandwich. The reader is technical and time-constrained.
```

Figura 3.10: Implementarea unei capacități de PR review

Aceste capacități permit execuția consistentă și reproductibilă (deoarece agentul primește același prompt de fiecare dată) a unui flux de execuție dorit în rezolvarea unei sarcini.

3.7.4 Subagenți

Odată cu creșterea abilităților de gândire ale agenților, a apărut conceptul de echipa de agenți. În aceasta echipa, unul este agentul principal numit și orchestrator, care comanda execuția subagenților și centralizează informațiile returnate de aceștia. Acesta are contextul integral al problemei și gândește la nivel înalt. Subagenții nu au context decât pentru partea lor de problemă, iar la finalul execuției sau cercetării, comunica orchestratorului un rezumat al descoperirilor sau al acțiunilor adoptate.

Un subagent este într-o forma simplificată, un alt apel la un model de inteligență artificială cu o altă personalitate și care trebuie să rezolve o problemă specifică. Acesta are acces la un număr limitat sau specializat de unelte pe care numai el le poate folosi și are un prompt de sistem diferit care îi da instrucțiuni cu privire la task-ul specific pe care acesta trebuie să îl ducă la bun sfârșit. De asemenea îi este comunicat formatul în care trebuie să returneze răspunsul orchestratorului și faptul că acesta trebuie să respecte intrusiunile acestuia.

Subagenții au aceleași limitări precum agentul principal. Un exemplu punctual ar fi contextul datorită faptului că uzual subagenții folosesc modele mai puțin performante decât modelul principal și în acest caz este necesară compactarea contextului. Agentul orchestrator pornește agenții secundari uzual printr-o unealtă specială suportată de framework-ul în care este dezvoltat, în cazul acesta, LangGraph, precum în figura 3.11.

În final, și aceștia necesită monitorizare și instrumentare pentru înregistrarea corectă a metricilor.

```

1 @tool
2 async def spawn_subagent(
3     subagent_type: str, prompt: str, timeout_s: int | None = None,
4     tool_call_id: Annotated[str, InjectedToolCallId] = "",
5 ) -> str:
6     """Spawn a specialised subagent to handle a focused sub-task in isolation.
7     The subagent runs its own tool-calling, then returns a single final report.
8     You only see that report. Use this to delegate a deep dive without
9     polluting your own context.
10    Pick a 'subagent_type' that matches the work: 'general' - open-ended research,
11    'code_search' - read-only code lookups, 'web_research'
12    Args:
13        subagent_type: One of the registered types above.
14        prompt: The task description for the subagent. Be specific: it has
15            none of your conversation context.
16        timeout_s: Optional hard timeout in seconds. Defaults to the type's
17            built-in default; capped by config.
18    """
19    parent_ctx = _ctx()
20    result = await run_subagent(
21        subagent_type=(subagent_type or "").strip(),
22        prompt=(prompt or "").strip(),
23        parent_ctx=parent_ctx,
24        timeout_s=timeout_s,
25        parent_tool_call_id=tool_call_id or None,
26    )
27    if tool_call_id:
28        parent_ctx.setdefault("pending_subagent_results", {})[tool_call_id] = {
29            "subagent_type": (subagent_type or "").strip(),
30            "prompt": (prompt or "").strip(),
31            "text": result.text,
32            "iterations": result.iterations,
33            "tool_call_count": result.tool_call_count,
34            "timed_out": result.timed_out,
35            "error": result.error,
36            "duration_s": result.duration_s,
37            "usage": result.usage,
38        }
39    return result.text

```

Figura 3.11: Implementarea unei funcții pe care agentul o folosește pentru a porni un subagent

La momentul actual *nici un produs de pe piață nu permite configurarea în detaliu a subagenților*. Implementarea noastră permite **configurarea completă a fiecărui subagent**, precum modelul utilizat pentru fiecare subagent în parte, promptul de sistem, posibilitatea unui subagent de a porni subagenți proprii și alte setări precum numărul maxim de iterații per subagent, adâncimea maxima de subagenți, și timpul maxim pe care orchestratorul îl așteaptă pentru a primi un răspuns. Configurarea de baza poate fi consultata în figura 3.12

```

1 subagents:
2   enabled: true
3   default_timeout_s: 120
4   max_timeout_s: 600
5   max_depth: 2 # max recursive subagent spawn
6   types:
7     general:
8       model_override: opus4-7 # model
9       override
10      max_iterations: 25
11      default_timeout_s: 180
12     code_search:
13       model_override: sonnet4-6
14       max_iterations: 15
15       default_timeout_s: 90
16     web_research:
17       model_override: gemma4
18       max_iterations: 18
19       default_timeout_s: 120
20 max_parallel: 3
21 child_context:
22   max_tool_tokens: 2000 # tool_result
23   body_cap (tokens, ~4 chars each)
24   head_tail: [800, 400] # head/tail
25   excerpt sizes when truncating

```

Opțiunea de utilizare aparține utilizatorului, împreună cu configurarea acestora. Aplicația furnizează 3 subagenți de început: cel general, cel pentru cercetarea codului și cel pentru cercetări online. Fiecare vine cu setul sau de instrucțiuni și unelte. Utilizatorul poate alege ce model folosește pentru fiecare, cat de lungi pot fi căutările sale și timpul de așteptare după răspuns al subagenților. De asemenea o alta setare de interes este posibilitatea subagenților de a porni multipli subagenți proprii. Agenții sunt executați uzual în paralel și pot exista mai multi agenți de același tip ce rulează în paralel cu instrucțiuni diferite. Acest lucru poate fi limitat de asemenea din configurație. În final, contextul subagenților are și el o setare pentru gestionarea răspunsului de la unelte.

Figura 3.12: Configurarea subagenților

3.8 Extensibilitate

Deoarece aplicația este gândita pentru a fi open-source, arhitectura aplicației trebuie să fie ușor extensibila de utilizatori ce doresc să aducă modificări în modul de funcționare, în special în punctele de extindere. Câteva din acesta puncte de extindere gândite de la bun început sunt: logica de compactare, setul de unelte, setul de subagenți, setul de furnizori și setul de skill-uri.

3.8.1 MCP și adăugarea de unelte

Aplicația conține un client MCP pentru adăugarea uneltelor externe și integrarea cu servicii terțe. Acesta accepta metodele de comunicare comune în industrie: STDIO, HTTP și SSE. Pentru a configura un server MCP și a accesa instrumentele pe care le pune la dispoziție în aplicație, utilizatorul poate folosi comanda `/mcp` în interiorul interfeței, sau poate adăuga personal serverul în fișierul `/.vibe-cli/preferences.json` și să repornească aplicația.

Serverele MCP pot expune atât unelte cat și skill-uri sau prompt-uri. Acestea pot fi reîncărcate sau gestionate prin comanda `/mcp`

Adăugarea de unelte poate fi făcuta cu ușurință, fiind o aplicație open-source. Dacă utilizatorul este un dezvoltator și a clonat proiectul local, acesta poate adăuga un instrument nou după modelul din figura 3.8 și să își implementeze propria logica. Unelte sunt definite la calea `src/tools.py`. După adăugarea uneltelor, aplicația trebuie reinstalată/repornită.

3.8.2 Adăugarea de subagenți

Adăugarea de subagenți este la fel de simplă precum adăugarea de unelte. Utilizatorul trebuie doar să își adauge un fișier pentru agentul sau în `src/subagents/types` conform modelului furnizat în figura 3.13. Crearea unui nou subagent implica pe scurt extinderea clasei `Subagent` și completarea câmpurilor cu date despre subagent precum: nume, descriere, rol.

```

1 @register
2 class CodeSearchSubagent(Subagent):
3     name = "code_search"
4     description = (
5         "Fast read-only code search. Finds files/functions/symbols and "
6         "reports paths + line ranges + relevant snippets. Cannot write or "
7         "execute side-effectful commands."
8     )
9     default_timeout_s = 90
10    max_iterations = 15
11    # Read-only toolset. 'run_command' is included for grep/rg/find use.
12    allowed_tools = ["read_file", "list_directory", "run_command", "fetch_artifact"]
13
14    def role_prompt(self, prompt: str, parent_ctx: dict) -> str:
15        return (
16            "ROLE: read-only code-search subagent.\n\n"
17            "Your job is to LOCATE code that matches the orchestrator's "
18            "request and report findings concisely. You are an advisor only "
19            "- even though 'run_command' is available, it is for read-only "
20            "searches (rg / grep / find / ls / cat / wc). NEVER invoke "
21            "installers, builds, test runs, package managers, or any "
22            "command that mutates state.\n\n"
23            "Stop searching once you have enough to answer; do not enumerate "
24            "exhaustively.\n\n"
25            "Final report format:\n"
26            " - For each match: 'path:line' + a one-line description.\n"
27            " - When useful, include a short snippet (<=6 lines).\n"
28            " - End with a one-sentence takeaway for the orchestrator."
29        )

```

Figura 3.13: Înregistrarea unui nou subagent

În final, agentul trebuie adăugat și pornit din fișierul de configurație `config.yml` conform figurii 3.12. Pentru a se aplica schimbările aplicația trebuie reinstalată/repornită.

Definiția unui subagent depășește însă simpla pereche nume/descriere: fiecare tip fiind o subclasa Python a clasei `Subagent` înregistrată prin decoratorul `@register`, ceea ce oferă dezvoltatorului un set bogat de puncte de extensie:

- `role_prompt(prompt, parent_ctx)` – fragment specific de rol, atașat contractului comun `SUBAGENT_CONTRACT`, cu acces la prompt și la contextul părintelui pentru adaptare dinamică;
- `allowed_tools` / `excluded_tools` – lista de comenzi permise și blocate intersectate cu setul activ de unelte al părintelui;
- `tool_names(parent_ctx)` și `system_prompt(prompt, parent_ctx)` – hook-uri prin care setul de unelte și prompt-ul de sistem pot fi recalculat la fiecare invocare;
- `max_iterations` și `default_timeout_s` – limite pentru bucla de tool-calling, ce pot fi suprascrise din `subagents.types.<tip>` în `config.yml`;
- model dedicat per tip, configurabil prin `/subagents <tip> <provider> <model>`, cu fallback pe modelul principal de chat.

La execuție, fiecare subagent rulează o buclă completă de tool-calling cu același flux de permisiuni și aceleași `tool_guards` deterministe ca părintele (de exemplu *read-before-write*, prin partajarea `ctx["file_reads"]`), beneficiază de compactare locală a mesajelor de tip `ToolMessage` supradimensionate, de telemetrie `Langfuse` introdusă în span-ul părintelui, de agregarea consumului de tokeni (*input/output/cache_read/cache_write*) și de respectarea limitelor `subagents.max_depth` și `subagents.max_parallel`. În cazul în care modelul iese din buclă fără raport final, este reapelat o singură dată pentru a emite raportul.

Modelul actual nu suportă definiții declarative bazate pe fișiere Markdown (analog skill-urilor), scoping de servere MCP per subagent, streaming intermediar al ieșirii subagentului către orchestrator, și nici politici de *retry/replan* dincolo de `max_iterations`.

3.8.3 Creare de skill-uri

Crearea de skill-uri este reprezentată de scrierea unui fișier de tip Markdown și de plasarea acestuia fie în containerul global de skill-uri aflat la `/.vibe-cli/skills`, fie în folderul `.vibe` din cadrul proiectului în care se lucrează curent pentru skill-uri locale. Pentru aplicarea schimbărilor sau gestionarea skill-urilor, utilizatorul poate folosi comanda `/skills`

3.8.4 Suportul pentru furnizori de modele de limbaj

LangGraph furnizează un strat grozav de abstractizare a furnizorilor de modele de limbaj, dar aduce și constrângerea suportului doar pentru furnizori suportați de LangGraph. Astfel, adăugarea unui nou furnizor implică schimbări în mai multe module ale aplicației. Pentru a reduce aceste schimbări la minimum, întreaga suprafață a furnizorilor este concentrată în modulul `src/providers.py`, unde fiecare furnizor este o subclasa a clasei `BaseProvider`. O instanță a noii clase este înregistrată în dicționarul `PROVIDER_REGISTRY`, iar restul aplicației (`get_model` din `src/main.py`, selecția modelului per subagent din `src/subagents/base.py`, comenzile `/model` și `/key` din interfață, precum și validarea și persistarea cheilor API din `ui/provider_handler.py`) derivă automat comportamentul din acest registru.

O subclasa `BaseProvider` declară câteva elemente esențiale:

- `name` - cheia utilizată în `config.yml` și în `preferences.json`;
- `env_var` - numele variabilei de mediu (sau lista de nume, rezolvate în ordine) pentru cheia API, sau `None` pentru furnizorii care nu necesită autentificare, precum Ollama;
- `validate(cfg)` - o verificare fără apel LLM, de tipul `models.list()`;
- `create_model(cfg)` - metoda care returnează instanța `ChatModel` LangChain. Opțional, sunt disponibile metode precum `base_url(cfg)` și `unreachable_help(cfg)` pentru mesaje de ajutor afișate când validarea eșuează.

Concret, adăugarea unui nou furnizor presupune patru modificări mici:

- o nouă subclasa în `src/providers.py` adăugată în `PROVIDER_REGISTRY`;
- un bloc `providers.<nume>` în `config/config.yml` cu `api_key: ${ENV_VAR}`, lista de modele și temperatura, plus eventuale câmpuri specifice (de exemplu `region` pentru Bedrock sau `base_url` pentru Ollama);
- tarifele per model în `web/data/pricing.py`, sau adăugarea numelui furnizorului în `_FREE_PROVIDERS` dacă este local sau gratuit, astfel încât estimarea costurilor din panoul web să funcționeze corect.

În final, LangGraph fiind un proiect open-source, dependența de suportul pentru furnizorii de modele de limbaj nu este o problemă, comunitatea putând oricând să contribuie cu furnizori noi.

3.8.5 Adăugarea tehnicilor de compactare

Pipeline-ul de compactare este construit ca un sistem de extensii înmagazinate într-un registru, astfel încât adăugarea de strategii noi nu necesita modificări exhaustive ale codului. Fiecare strategie este o subclasa a uneia dintre clasele de baza `Stage` (execuție *eager*, per eveniment) sau `ThresholdStage` (execuție la atingerea pragului de ocupare, cu vizibilitate asupra întregului istoric), declarata în `src/compaction/stages/` și înregistrata explicit în `src/compaction/__init__.py`. Adăugarea unei strategii noi presupune patru modificări:

- implementarea metodei `apply_eager(event, *, store)` sau `apply_threshold(tctx)`, care schimbă `event["projected"]`, respectiv declara identificatorii evenimentelor eliminate;
- înregistrarea clasei în `EAGER_REGISTRY` sau `THRESHOLD_REGISTRY`;
- opțional, adăugarea ei într-un grup de exclusiune mutuala `EAGER_EXCLUSION_GROUPS` dacă scrie în același câmp al evenimentului ca o strategie existentă (validarea eșuează la pornire cu `CompactionConfigError` dacă doua strategii incompatibile sunt active simultan);
- adăugarea ei în `config/config.yml` sub `compaction.pipeline`.

Strategiile asincrone care depind de un model de limbaj urmează un protocol în doua etape: o prima rulare sincrona produce un rezultat sigur (de exemplu, trunchiere cap-coada) marcat cu starea `truncated_pending`, iar o sarcina de fundal pornita prin `ctx.spawn(...)` înlocuiește ulterior proiecția cu varianta sumarizata și o rescrie în sesiune prin `on_event_updated`. În caz de eșec sau indisponibilitate a modelului, starea trece în `truncated_fallback`, garantând astfel ca bugetul de tokeni este respectat indiferent de rezultatul apelului LLM. Câmpul `event["content"]` rămâne în toate cazurile imutabil, iar orchestratorul realizează înainte de fiecare trecere o copie de siguranța a fișierului de sesiune, astfel încât orice transformare este reversibila.

Pe baza acestui mecanism, următoarele strategii sunt candidate naturale pentru extinderea setului de strategii de compactare:

- **memorarea apelurilor de unealta:** indexarea rezultatelor după perechea (nume, argumente normalizate) și reutilizarea lor pentru apeluri identice ulterioare;
- **proiecția diferențială a citirilor repetate:** pentru reapelarea `read_file` pe aceeași cale, înlocuirea rezultatului cu un `diff` unificat față de citirea anterioara, sau cu marcajul `[unchanged]` când suma SHA coincide, valorificând mecanismul `ctx["file_reads"]` deja existent;
- **fereastra glisanta de fapte ancorate:** un `SystemMessage` persistent, regenerat la fiecare trecere de prag, care păstrează obiectivele utilizatorului, convențiile proiectului și fișierele aflate în focus, transportat printr-un tip de eveniment rezervat `KIND_ANCHOR` ce nu este niciodată eliminat;

- **proiectori structurați per unele:** extractoare declarative care păstrează doar câmpurile efectiv consumate de agent (de exemplu `{title, url, snippet}` pentru căutarea web, `nume + tip` pentru listarea de directoare, codul de ieșire și ultimele linii pentru comenzile de terminal), oferind un compromis între costul și agresivitatea celor existente;

Strategiile bazate pe scorare de importanță cu LLM ca arbitru au fost evaluate și respinse pentru această etapă, deoarece sunt acoperite determinist de `importance_prune` și `bulk.hybrid`, iar caracterul lor nereproductibil și costul suplimentar nu justifică adoptarea.

3.9 Securitatea în mediul agentic

Atunci când încredințăm controlul deplin, sau cel puțin accesul la comenzi cu potențial periculos, trebuie să avem niște măsuri de siguranță pentru a ne asigura ca agentul nu încearcă voit sau accidental să întreprindă o acțiune nedorită.

Există multe motive pentru care un agent ar putea încerca să ruleze comenzi periculoase, iar studiul recent al amenințărilor specifice sistemelor agentice [30] le grupează în trei mari clase.

În primul rand, *prompt injection* indirect: conținutul ingerat de agent dintr-un fișier, o pagină web sau ieșirea unei unelte poate conține instrucțiuni adversariale care deturnează obiectivul inițial, fenomen demonstrat empiric împotriva unor soluții de producție precum Azure Prompt Shield și Meta Prompt Guard, unde tehnici simple de perturbare la nivel de caracter sau bazate pe Adversarial Machine Learning (AML) ating rate de evaziune apropiate de 100% [31].

În al doilea rand, *utilizarea greșită a uneltelor*, fie prin halucinarea argumentelor, fie prin compunerea unor secvențe de apeluri inofensive individual, dar distructive în ansamblu (de exemplu `rm -rf` aplicat unei cai derivate eronate).

În al treilea rand, *data poisoning* la nivelul memoriei sau al contextului persistent, prin care un atacator plantează fapte false ce vor fi recuperate și tratate drept context autoritar la invocări ulterioare.

În ultimul rand, o altă sursă de acțiuni nedorite poate fi constituită de frustrare. În unele cazuri, agentul determină ca cel mai ușor mod de a rezolva o eroare este de a șterge cu totul fișierul sau funcționalitatea care produce aceasta eroare.

Pentru ca aliniamentul modelului [32] și filtrele de tip guardrail ⁵ [33], [34] nu oferă garanții suficiente în fața acestor vectori, aplicația adoptă o apărare în adâncime, structurată pe trei niveluri descrise în continuare: izolarea execuției prin sandboxing, restricționarea acțiunilor prin permisiuni explicite și delegare către subagenți cu privilegii reduse.

3.9.1 Sandboxing

Tehnica de sandboxing reprezintă rularea comenzilor într-un mediu izolat, bazat pe principiul *least privilege*. În aplicație, mecanismul izolează comenzile de shell ale agentului într-un container Docker dedicat sesiunii curente, în care directorul de lucru al utilizatorului este montat prin `bind-mount` la `/workspace`, astfel încât modificările asupra fișierelor se propaga imediat către gazda, fără a fi necesară o etapă explicită de sincronizare. Funcționalitatea este *opt-in* și degradează elegant la execuția pe gazda atunci când procesul Docker nu este disponibil, păstrând astfel portabilitatea aplicației.

⁵Guardrail-ul reprezintă o combinație între instrucțiuni de prompt care reglementează comportamentul modelului pentru a nu executa sau întoarce către utilizatoru secvențe nedorite, cat și filtre deterministe ce blochează execuția sau întoarcerea către utilizator a acestor fraze nedorite

Configurarea este expusa printr-o secțiune dedicata în `config.yml` conform figurii 3.14. La pornirea sesiunii, aplicația citește aceasta configurație, sondează disponibilitatea procesului cu un *timeout* scurt și aplica o politica de validare a directorului de lucru organizata pe trei niveluri: refuzul ferm al cailor sensibile (`/`, `/tmp`, `$HOME` și subdirectoarele sale directe), acceptul automat al directoarelor care conțin marcaje de proiect (`.git`, `pyproject.toml` etc.) și validarea pe baza de dimensiune și număr de fișiere pentru cazurile ambigue. Politica poate fi suprascrisa explicit de utilizator prin variabila de mediu `VIBE_SANDBOX_FORCE`.

```

1 sandboxing:
2   enabled: false
3   image: python:3.13-slim
4   network: bridge # bridge / none
5   memory: 2g
6   cpus: 2
7   timeout_default: 30 # default 'run_command'
8     timeout in seconds
9   blocked_paths:
10    - "~/"
11   ambiguous_max_mb: 500
12   ambiguous_max_files: 10000

```

Configurația permite controlul parametrilor de izolare: imaginea de baza, modul de rețea, limitele de memorie și Central Processing Unit (CPU), timpul maxim de execuție, lista de cai interzise și pragurile (dimensiune maxima, număr maxim de fișiere) ce delimitează directoarele considerate ambigue.

Figura 3.14: Configurarea sandbox-ului Docker pentru execuția izolata a comenzilor

Containerul este pornit cu `docker run -d -rm`, având toate capabilitățile retrase (`-cap-drop ALL`), executând sub identitatea numerica a utilizatorului gazda (`-user <uid>:<gid>`), cu limitele de memorie, Central Processing Unit (CPU) și rețea impuse de configurație, și trăiește pe toată durata sesiunii. Toate comenzile sunt rulate ulterior prin `docker exec`. În caz de eșec la pornire, modulul de sandboxing întoarce o pereche (`None`, *motiv*) fără a propaga excepția, iar orchestratorul continua execuția pe gazda, anunțând modul de lucru. Închiderea containerului este garantata la terminarea sesiunii printr-o rutina dedicata.

Rutarea apelurilor este realizata la nivelul uneltei `run_command`, care consulta la fiecare invocare un `ContextVar` ce conține eventualul *handle* al sandbox-ului și deleagă execuția fie containerului, fie gazdei. Unelte de fișiere (`read_file`, `write_file`, `list_directory`) rămân pe gazda, deoarece `bind-mount`-ul face cele doua perspective echivalente, iar evitarea apelului prin Docker reduce semnificativ latenta. Atunci când sandbox-ul este activ, setul de unelte este extins cu `run_command_host`, o *escape hatch* explicit condiționată de permisiuni, care permite execuția controlata pe gazda atunci când este strict necesar. Subagenții moștenesc *handle*-ul prin copierea superficiala a contextului părintelui, astfel încât toate apelurile imbricate de unealta împărtășesc același container și beneficiază în mod uniform de izolare.

Gărzile transversale (de exemplu `read_before_write`) și verificările de permisiuni se aplica identic pentru apelurile rutate către sandbox și pentru cele executate pe gazda, fiind plasate înaintea execuției propriu-zise. În acest fel, izolarea oferita de container reprezintă o linie de apărare suplimentara, complementara mecanismului de permisiuni descris în continuare, și nu îl înlocuiește.

3.9.2 Permisii

O măsura de apărare necesara este aprobarea execuției uneltelor de către utilizator. Unele instrumente, precum cele de citire din fișiere sunt aprobate din oficiu pentru a scădea necesitatea de a cere permisiuni omului. În schimb, operatorul este totdeauna cel care are ultimul cuvânt în execuția comenzilor. Acesta poate opta să aprobe fiecare comanda în parte, toate comenzile de un tip în timpul sesiunii curente sau permanent, precum în figura 3.15. Acestuia i se prezintă ce unealta se dorește a fi folosita și cu ce argumente.



Figura 3.15: Cererea de aprobare a permisiunii de a scrie un fișier

Toate aceste restricții privind permisiunile se aplica atât agentului orchestrator cât și subagenților.

3.9.3 Guardrails, intenții periculoase și cereri în afara scopului (Out of Scope (OOS))

O aplicație în producție trebuie să răspundă doar la cereri care corespund cerințelor pentru care a fost proiectat. Cu un sistem nedeterminist precum cele care folosesc modele de limbaj, acest lucru este excepțional de dificil. În ciuda diverselor metode de combatere a acestor încercări de a face aplicația să facă ceva ce nu ar trebui să facă, uzual încă exista o sumedenie de metode prin care acestea pot fi păcalite.

În general se folosesc o combinație de instrucțiuni de prompt, antrenament, selecții de cuvinte interzise, sau un apel către un model extern care ar decide dacă cererea este validă sau nu.

De asemenea, furnizorii de modele de limbaj au propriile metode de apărare prin care încearcă să combată acest lucru.

Aplicația nu prezintă momentan astfel de filtre, bazându-se pe cele impuse de furnizorii de modele.

3.10 Observabilitate

Observabilitatea aplicației este oferită pe două niveluri complementare: tracing extern, prin integrarea cu Langfuse Cloud, și introspecție locală, prin panoul web analitic livrat împreună cu aplicația. Ambele componente sunt opționale, dezactivate implicit, și nu transmit date către alte servicii în afara celor configurate explicit de utilizator.

3.10.1 Langfuse

Pentru tracing extern, aplicația integrează Langfuse Cloud prin intermediul unei interceptări al grafului agentului. Activarea funcționalității se face fie prin editarea fișierului `config.yml`, fie, prin comanda interactivă `/tracing setup`, care persista cheile API în `preferences.json`. Rezolvarea iredentistelor urmează o ordine deterministă de precedență: preferințele locale au prioritate, urmate de fișierul de configurație și, în final, de variabilele de mediu `LANGFUSE_*`.

La fiecare invocare a grafului, evenimentele sunt interceptate și publicate către Langfuse, asincron. Datele încărcate în cloud sunt sub forma unui trace ⁶ principal care conține întreaga

⁶Un trace este un mesaj de tip JSON care conține date. Este folosit în monitorizarea distribuită și reprezintă 'urma' sau 'dovada' unor acțiuni și conține detalii despre acestea. Un trace poate conține mai multe obiecte.

interacțiune, alături de span-uri ⁷ ierarhice pentru fiecare apel către modelul de limbaj, fiecare invocare de unealta și fiecare traversare de nod din graf. Pentru fiecare span sunt înregistrate printre altele: numele modelului, intrările, ieșirile, consumul de tokeni și latentă. Trace-urile sunt etichetate cu identificatorul sesiunii curente (Universally Unique Identifier (UUID)v7 generat local), un identificator anonim al utilizatorului și tag-uri descriptive (de ex. modul activ, furnizorul și modelul selectat), permițând astfel agregarea per sesiune și filtrarea ulterioară în interfața Langfuse.

Pe baza identificatorului ultimului trace, doua comenzi suplimentare oferă bucla de feedback necesară evaluării continue: `/score` atașează scoruri numerice sau calitative ultimei interacțiuni, iar `/dataset` adaugă interacțiunea curentă într-un set de date persistent, util pentru benchmarking și teste de regresie comportamentală. În combinație cu mecanismul *LLM-as-judge* oferit de Langfuse, aceste comenzi formează fundamentul unei suite de monitorizare pentru calitatea răspunsurilor agentului.

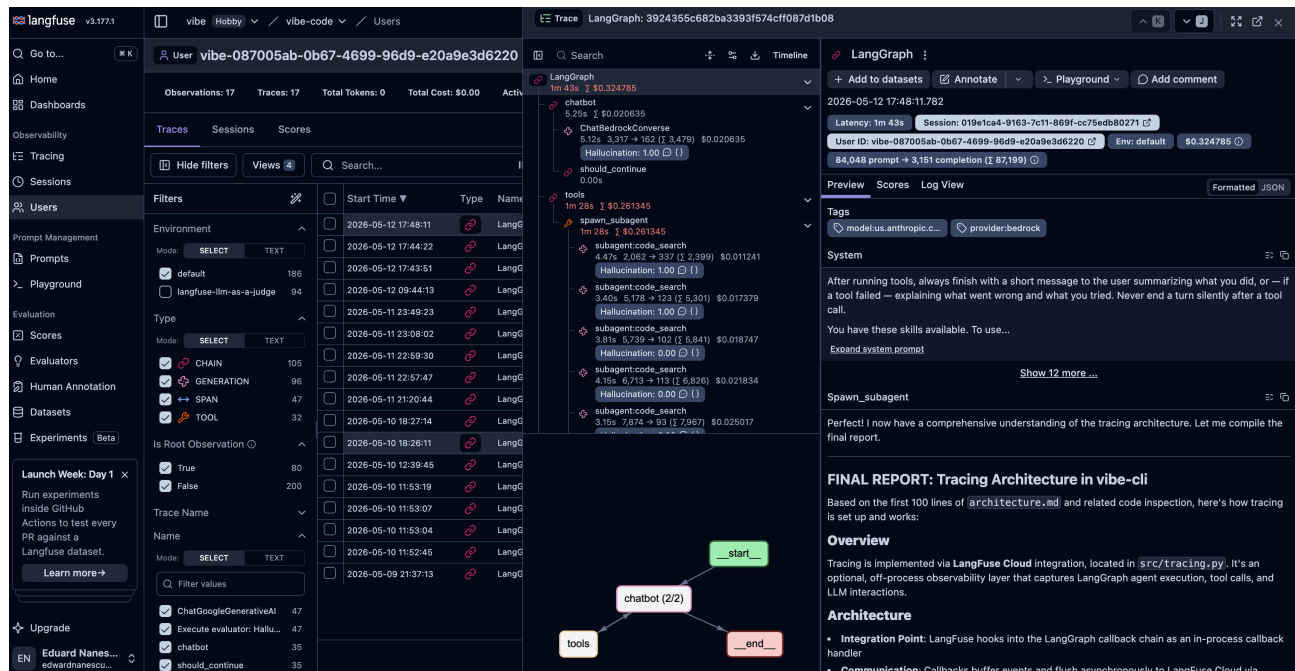


Figura 3.16: Interfața Langfuse

În figura 3.16 putem observa interfața Langfuse și capacitățile pe care ni le oferă.

În partea stânga avem navigarea care ne permite să accesăm diferitele funcționalități, printre care și sortarea trace-urilor după sesiuni și utilizatori, rubrica de management al prompt-ului în care putem stoca și versiona diferite prompt-uri pentru testare și a determina cauzalitatea între prompt-uri și degradări de performanță. În final avem și zona de calitate unde putem agrega recenziile utilizatorilor (spre exemplu din butoanele de like/dislike ale conversației cu o interfață web a unui asistent de tipul ChatGPT), evaluatori automați ce ne oferă scoruri pentru diferite categorii și chiar și verificare umană.

În partea centrală observăm că ne aflăm în meniul *Users*, unde putem vedea toate sesiunile (și trace-urile dacă deschidem o sesiune) făcute de un utilizator, cu filtre și alte mecanisme de căutare.

În partea dreaptă am deschis trace-ul asociat unei sesiuni și putem observa locul în care Langfuse strălucește. Putem investiga desfășurarea conversației pas cu pas, investigând graficul

⁷Un span reprezintă o unitate de observare a unei singure acțiuni

de execuție, fiecare apel de unealta și răspunsul acestora, gândirea efectivă a modelului de limbaj, întregul istoric al conversației în format augmentat cat și metrice cu privire la tokenii. De asemenea tot aici putem vedea și evaluările automate pe care le-am configurat, care ne spun dacă răspunsurile agentului sunt corecte factual.

3.10.2 Panou web

Panoul web este o aplicație Flask locală, read-only, ce vizualizează în mod direct fișierele de sesiune din `~/vibe-cli/sessions/`, fără intermedierea unei baze de date. Aplicația este lansată prin comanda `vibe web` și este expusă implicit pe `http://127.0.0.1:5050`, cu posibilitatea de a configura adresa și portul prin parametri de linie de comandă.

Interfață, organizată pe o singură pagină, oferă selectoare pentru proiectul și sesiunea dorită și afișează, pentru sesiunea selectată, un sumar al consumului de tokeni pe categorii de evenimente, evoluția dimensiunii contextului raportată la limita modelului, frecvența și latentă apelurilor către unelte (cu percentile p_{50} și p_{95}), debitul de generare exprimat în tokeni pe secundă, costul estimat per apel pe baza unui tabel de preturi configurabil, precum și istoricul evenimentelor de compactare cu diferențele corespunzătoare. Componente dedicate permit inspectarea granulară a fiecărei unelte invocate, cu argumentele și rezultatul aferent, precum și reconstrucția turului conversațional eveniment cu eveniment.

Sincronizarea cu starea actuală a sesiunilor și a log-urilor este realizată prin SSE, o singură conexiune persistentă fiind multiplexată pentru ambele fluxuri. Detectia modificărilor se face prin compararea metadatelor sistemului de fișiere (cale, timp de modificare, dimensiune) la intervale de 1.5 secunde, iar reîncărcările sunt spațiate la 250 de milisecunde una de cealaltă pentru a evita actualizări redundante. Un panou inferior afișează ultimele 1000 de linii din log-ul textual al aplicației, cu auto-derulare inteligentă care suspendă fixarea la baza atunci când utilizatorul consultă înregistrări anterioare. Atunci când tracing-ul Langfuse este activ, un buton dedicat din bara superioară deschide proiectul corespunzător în interfața externă, fără a expune cheile API către browser.

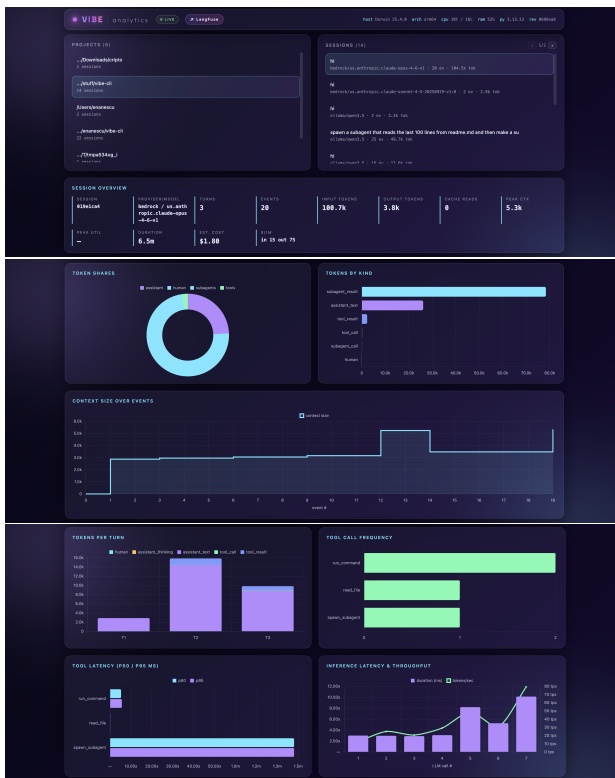


Figura 3.17: Imagini cu interfața web a aplicației

Scopul panoului web este acela de a aduce transparenta pentru un produs care are acces la sistemul utilizatorului. Astfel, toate acțiunile sunt contorizate și toate costurile sunt explicate, spre deosebire de restul aplicațiilor disponibile pe piață care ascund aceste numere.

Capitolul 4

REZULTATE EXPERIMENTALE

Având în vedere finalizarea implementării, am condus un set de teste în timpul utilizării asistentului de programare cu diferite variații ale configurației pentru extragerea unui mod optim de utilizare și strângerea de date legate de mai multe aspecte ale folosirii care influențează performanțele aplicației.

Data fiind natura nedeterministă a răspunsurilor modelelor de limbaj, schimbările de configurație precum modelul utilizat sau promptul de sistem, cat și uneltele puse la dispoziție și alte aspecte pot avea un impact consistent asupra datelor de ieșire și acțiunile întreprinse de agent.

4.1 Poziționarea față de concurența

La finalul implementării, rezultatul este comparabil din punct de vedere al funcționalităților dar și al performanței cu aplicațiile existente pe piață în momentul de față.

Calitățile ce diferențiază abordarea noastră de restul competiției sunt: integrarea ușoară cu modele OSS ce pot rula local pe mașina utilizatorului, scăpând de dependența de un furnizor de modele de limbaj și de costurile asociate lor, transparenta interacțiunii cu agentul prin panoul web și integrarea cu Langfuse dar și prin opțiunile exhaustive de configurație prin care utilizatorul poate controla majoritatea funcționalităților vitale.

Pentru a susține cantitativ aceasta diferențiere, în tabelul 4.1 am rezumat principalele caracteristici ale celor mai populare unelte agentice de programare pentru terminal (*Claude Code* dezvoltată de Anthropic, *Codex CLI* dezvoltată de OpenAI și *Copilot CLI* dezvoltată de GitHub) în raport cu soluția propusă.

Tabela 4.1: Comparație între soluția dezvoltată și principalele unelte agentice de programare pentru terminal disponibile pe piață

Aplicație	Furnizori și model de cost	Puncte forte	Puncte slabe (raportat la soluția propusă)
Claude Code (Anthropic) Sursa închisă, proprietara	Doar modele Anthropic Claude (Amazon Web Services Bedrock și Vertex AI doar în ediția enterprise). Abonament Pro/Max sau cheie API cu plata per-token.	• Maturitate și integrare cu modelele Claude • Extensibilitate prin MCP, subagenți, skills, hooks și extensii • Permisuni granulare și sandboxing pe macOS/Linux/Windows • Suprafețe multiple (terminal, VS Code, JetBrains, web)	• Cod sursa închis, dependență totală de un singur furnizor • Fără suport documentat pentru modele OSS locale • Cost obligatoriu (abonament sau per-token), fără regim gratuit • Fără panou web propriu pentru transparenta; doar consola Anthropic de facturare și comanda pentru context în aplicație. • Tracing nativ minim (suport parțial OpenTelemetry)
Codex CLI (GitHub) Licență Apache-2.0	Doar modele OpenAI. Inclus în plan ChatGPT Plus/Pro/Business sau cheie API cu plata per-token.	• Singura soluție comercială cu cod sursa complet deschis (Apache-2.0) • Implementare în Rust, performanță ridicată și executabil unic • Suport pentru MCP, subagenți, skills, reguli, interceptări și sandboxing • Autentificare directă cu un cont ChatGPT existent	• <i>Furnizor unic</i>: doar modele OpenAI, fără rutare alternativă documentată • Fără suport pentru modele OSS locale (Ollama, llama.cpp) • Cost obligatoriu (abonament ChatGPT sau cheie API) • Documentație minimă privind transparenta utilizării (tokeni, cost, istoric) • Fără integrare cu Langfuse sau alt backend de tracing
Copilot CLI (GitHub Docs) Sursa închisă, proprietara	Modele găzduite de GitHub (Claude, GPT); rutare opțională către endpoint-uri OpenAI-compatibile (Ollama, vLLM, Azure OpenAI, Anthropic) prin variabile de mediu. Abonament Copilot Pro/Business/Enterprise + credite consumate per interacțiune.	• Singura soluție comercială cu rutare explicită către Ollama și endpoint-uri locale • Configurabilitate avansată: agenți per utilizator/repo/organizație, MCP, skills, hooks, memorie persistentă • Integrare nativă cu workflow-urile GitHub (PR-uri, issue-uri, context de repo) • Comenzi de inspecție a sesiunii (/usage, /context, /compact)	• Cod sursa închis, abonament Copilot obligatoriu • Preț opac (credite consumate per interacțiune, fără mapare publică la cost) • Status de previzualizare publică • Fără panou web sau interfață grafică; analiza disponibilă numai prin comenzi în terminal • Fără integrare cu Langfuse sau OpenTelemetry
VIBE-CLI (soluția propusă) (GitHub) Licență MIT	OpenAI, Anthropic, Google Gemini și Ollama , comutabile pe loc prin /model. Aplicație gratuită; cost zero pentru rulare locală cu Ollama, sau exclusiv tariful furnizorului ales în regim cloud.	• Suport nativ multi-provider, inclusiv modele OSS locale prin Ollama, fără cheie API • Panou web Flask local, read-only, cu metrice per sesiune (tokeni pe categorii, evoluția contextului, cost estimat, latența uneltelor cu p_{50}/p_{95}, debit, compactări, reconstrucția interacțiunii) și Integrare nativă Langfuse prin callback-uri LangChain, cu <i>tagging</i> per subagent și evaluări automate • Suport multi-modal (imagini, PDF, fișiere text și documente), condiționat de capabilitățile modelului selectat • Subagenți tipizați paraleli (general, code_search, web_research) cu model selectabil per tip • Configurație YAML exhaustivă: provideri, MCP, subagenți, skills, <i>tool guards</i> deterministe, permisiuni <i>allow/deny</i>, sandboxing Docker, compactare și tracing	• Maturitate redusă față de produsele furnizorilor mari, ecosistem în dezvoltare • Fără extensie nativă de IDE (utilizare exclusiv din terminal) • Suport oficial limitat la macOS și Linux (fără instalare nativă pe Windows) • Curba de învățare mai abruptă pentru configurarea avansată din <code>config.yml</code>

În tabelul 4.1, se observă mai multe diferențe între implementarea realizată și industrie, în special independența față de un furnizor unic de modele, integrarea cu modele locale OSS, panoul web propriu și tracing-ul nativ cu Langfuse, poziționează soluția propusă într-un punct

distinct față de concurența, oferind în același timp un set comparabil de capabilități agentice (subagenți, MCP, sandboxing, configurabilitate avansată).

În continuare, în tabelul 4.2 prezentăm o comparație cu cei mai relevanți asistenți de programare OSS din ecosistem (*OpenHands*, *Cline* și *Aider*), care împart cu soluția propusă avantajul codului sursă deschis și al independenței față de un furnizor unic de modele. Menționăm că *OpenClaw*, deși este cel mai popular proiect OSS din tabelul 2.2, nu a fost inclus în această comparație deoarece este poziționat ca asistent personal multi-canal (mesagerie, voce, automatizări), nu ca asistent dedicat dezvoltării software.

Tabela 4.2: Comparație între soluția propusă și principalii asistenți de programare OSS din ecosistem

Aplicație	Furnizori și model de cost	Puncte forte	Puncte slabe (raportat la soluția propusă)
OpenHands (GitHub) Licență MIT	Agnostic față de furnizor (Claude, GPT, Ollama și altele). Aplicație gratuită; cost limitat la tariful furnizorului ales.	• Sandboxing robust în containere Docker • Interfață grafică web și cli disponibile în paralel • SDK Python pentru construirea de agenți personalizați • Suport Kubernetes și scenarii enterprise • Integrări Slack, Jira, Linear	• Cerințe de resurse ridicate, Docker obligatoriu pentru fluxul principal • Configurare inițială complexă, curba de învățare abruptă • Fără integrare nativă Langfuse pentru tracing și evaluări automate • Fără panou de transparență cost și tokeni la nivel de sesiune • Fără suport explicit pentru subagenți tipizați cu model dedicat per tip
Cline (GitHub) Licență Apache 2.0	Agnostic față de furnizor, cu rutare prin OpenRouter (200+ modele: Anthropic, OpenAI, Ollama). Aplicație gratuită; cost limitat la tariful modelului ales.	• Extensie nativă pentru VS Code și plugin JetBrains • SDK Node.js și cli headless pentru CI/CD • Echipe multi-agent și agenți programați • Suport MCP și plugin-uri extensibile • Acces la peste 200 de modele prin OpenRouter	• Consum ridicat de tokeni datorită contextului mare încărcat permanent • Fără sandboxing nativ incorporat • Dependent de ecosistemul VS Code/JetBrains, fără aplicație standalone • Fără panou web propriu de transparență tokeni și cost • Fără integrare Langfuse sau alt backend de tracing
Aider (GitHub) Licență Apache 2.0	Agnostic față de furnizor (Claude, GPT, DeepSeek, Ollama). Aplicație gratuită; cost limitat la tariful modelului ales.	• Mapare automată a codului sursă (<i>repo map</i>) pentru context relevant • Integrare Git nativă cu commit-uri automate per modificare • Suport pentru peste 100 de limbaje de programare • Linting și testare automată cu reîncercare la eșec • Suport explicit pentru modele locale	• Fără sandboxing • Capabilități agentice limitate, fără subagenți tipizați paraleli • Fără suport MCP pentru extinderea uneltelor • Fără panou web propriu de transparență • Fără integrare Langfuse pentru tracing și evaluări automate

Pentru a sintetiza poziționarea *vibe-cli* față de întregul ansamblu al concurenței (atât unelte comerciale cât și OSS), în continuare listăm succint funcționalitate pe care soluția propusă le preia de la celelalte aplicații (demonstrând caracterul de înlocuitor complet) și, simetric, funcționalitățile prezente la concurență dar neacoperite direct în implementarea curentă, clasificate în trei categorii: funcționalități în afara scopului proiectului, funcționalități care există indirect prin mecanisme deja prezente și funcționalități total absente.

4.1.0.1 Funcționalități prezente în concordanță cu aplicațiile concurente proprii-tare și OSS:

- | | |
|--|--|
| 1. Suport multi-provider; | 9. Comenzi cu autocompletare; |
| 2. Modele OSS locale prin Ollama; | 10. Sesiuni persistente cu posibilitatea de continuare a conversației; |
| 3. Suport MCP; | 11. Compactare automata a istoricului conversației; |
| 4. Subagenți specializați paraleli; | 12. Căutare web integrată; |
| 5. Skills; | 13. Citire, scriere și editare de fișiere; |
| 6. Interceptări per tura și <i>tool guards</i> deterministe; | 14. Execuție de comenzi shell; |
| 7. Permisuni de aprobare sau respingere per unealta; | 15. Persistarea preferințelor utilizatorului; |
| 8. Sandboxing opțional în container Docker; | 16. Intrări multi-modale (imagini, PDF, fișiere text și documente) când modelul selectat le suporta; |

4.1.0.2 Funcționalități absente față de aplicațiile concurente:

1. În afara scopului proiectului:

- | | |
|--|--|
| (a) Aplicație nativă desktop, mobilă sau extensii IDE; | (e) Sandbox în cloud și execuție la distanță; |
| (b) Control prin vorbire; | (f) Suport Kubernetes și orchestrare enterprise; |
| (c) Suport multi-canal (Slack, Discord, WhatsApp); | (g) Magazin propriu de extensii. |
| (d) Artefacte interactive; | |

2. Existente indirect, prin mecanisme deja prezente:

- (a) Extensia pentru mediile de dezvoltare este acoperită prin terminalul integrat al oricărui IDE sau sistem de operare;
- (b) Integrarea cu GitHub pentru PR-uri sau GitHub Issues este realizată prin servere MCP sau invocarea utilitarului `gh`;
- (c) Rutare prin agregatoare externe de tip OpenRouter este ușor extensibilă prin editarea `PROVIDER_REGISTRY` din codul sursă;
- (d) Nu există un SDK extern pentru terți, dar modulele Python pot fi importate direct;
- (e) Suport pentru operațiuni de commit Git per modificare nu este suportat nativ dar este realizabil prin skills și prin execuție de comenzi shell;
- (f) Maparea codului sursă de tip *repo map* e acoperită parțial de subagentul `code_search`.

3. Absente:

(a) Instalare nativa pe Windows

(b) Execuție de acțiuni la intervale predefinite sau la momente declarate de timp

Așadar, prin acoperirea functionalitatilor centrale ale concurenței la care se adaugă caracterul open-source multi-provider, panoul web local și integrarea Langfuse, subagenții configurabili și capabilitățile de personalizare, `vibe-cli` se poziționează ca un înlocuitor funcțional complet pentru sarcinile uzuale de programare asistată din terminal, în timp ce diferențele în raport cu concurența țin în principal de suprafețele de utilizare (extensii IDE, aplicații desktop, canale de mesagerie) și de maturitatea ecosistemului din jurul aplicației.

4.2 Influenta lungimii și a exactității prompt-ului asupra performanței. De ce un prompt prea exact nu garantează rezultate mai bune

Unul dintre experimentele rulate a fost evaluarea performanțelor agentului în sarcini de PR review pe 6 proiecte open-source scrie în limbaje de programare diferite cu două tipuri de prompt de sistem:

1. Un prompt sumar, de tipul "Review these PRs, output your findings in this format". Promptul complet este disponibil **aici**.
2. Un prompt detaliat, de aproximativ 300 de linii care încearcă să maximizeze găsirea tuturor problemelor și îndeamnă agentul să își dea silința să găsească doar probleme reale și să evite problemele triviale. Promptul complet este disponibil **aici**

Experimentul s-a desfășurat în următoarele condiții:

- Baza de date pe care o folosim conține 50 de review-uri umane considerate ideale. Dorim să testăm dacă agentul nostru reușește să găsească toate problemele pe care le-au găsit inginerii umani în aceste proiecte.
- Baza de date conține o listă cu fiecare review uman, linia la care acesta a adăugat comentariul dar și întregul proiect de la momentul producerii review-ului pentru context.
- A fost folosită o arhitectura ce pornește mai multe procese paralele ce rulează agentul cu acces complet la unelte și subagenți, pentru a procesa mai repede cele 50 de review-uri.
- La finalul procesului de review, un alt model de limbaj mai mic se uita la review-urile umane și la cele generate de agent și verifica dacă acestea relatează aceleași probleme.
- Derivăm scorurile de precizie, recunoaștere și F1 din aceste metrici.
- Modelele folosite au fost Claude Opus 4.7 pentru review-ul efectiv și Claude Haiku 4.5 pentru agentul judecător și deduplicare.

Pentru a cuantifica performanța agentului, fiecare comentariu produs în raport cu setul de referință uman este clasificat în una dintre următoarele categorii:

- **True Positive (TP):** o problemă semnalată de agent care corespunde unei probleme identificate și în review-ul uman de referință.
- **False Positive (FP):** o problemă semnalată de agent căreia nu i se asociază nicio observație din review-ul uman (zgomot sau alertă greșită).

- **False Negative (FN):** o problemă prezenta în review-ul uman pe care agentul nu a reușit să o identifice (omisiune).
- **True Negative (TN):** o linie de cod pentru care nici agentul, nici recenzentul uman nu au formulat o observație; în contextul PR review-ului acest număr este dominant și dificil de definit riguros, motiv pentru care nu intervine în metricile raportate.

Pornind de la aceste categorii, definim *precizia* ca fracțiunea de alerte corecte din totalul alertelor emise de agent, *recunoașterea* (engl. *recall*) ca fracțiunea de probleme reale efectiv detectate, iar scorul F_1 ca media armonică a celor două, oferind o singură valoare scalară ce penalizează dezechilibrul între precizie și recunoaștere:

$$\begin{aligned} \text{Precizie} &= \frac{TP}{TP + FP}, \\ \text{Recunoaștere} &= \frac{TP}{TP + FN}, \\ F_1 &= 2 \cdot \frac{\text{Precizie} \cdot \text{Recunoaștere}}{\text{Precizie} + \text{Recunoaștere}} = \frac{2 TP}{2 TP + FP + FN}, \end{aligned} \quad (4.1)$$

unde Precizie, Recunoaștere, $F_1 \in [0, 1]$, iar valori apropiate de 1 corespund unei performanțe superioare. Un agent care raportează foarte puține probleme poate atinge o precizie ridicată cu o recunoaștere scăzută, în timp ce un agent care raportează agresiv obține recunoaștere mare în detrimentul preciziei; scorul F_1 surprinde acest compromis într-o singură metrică.

Problemele noi sunt probleme pe care modelul judecător le-a considerat ca fiind valide în ciuda faptului că nu se regăseau prin review-urile umane.

Rezultatele experimentului pot fi regăsite în tabelele 4.3 și 4.4.

Prompt	Proiect	Precizie	Recunoaștere	F1	Probleme noi	PRs
Detaliat	cal.com	64.3%	80.6%	71.5%	11	10
Detaliat	discourse-graphite	71.1%	75.0%	73.0%	11	10
Detaliat	grafana	72.7%	68.2%	70.4%	9	10
Detaliat	keycloak	73.3%	63.6%	68.1%	8	9
Detaliat	keycloak-greptile	100.0%	100.0%	100.0%	0	1
Detaliat	sentry	81.2%	52.6%	63.9%	3	6
Detaliat	sentry-greptile	37.0%	75.0%	49.6%	1	4
Detaliat	TOTAL	66.5%	70.6%	68.5%	43	50

(a) Metrici per proiect

Severitate	Precizie	Recunoaștere	F1	TP	FP	FN	Probleme noi
critical	58.8%	88.9%	70.8%	8	7	1	2
high	75.7%	80.5%	78.0%	33	18	8	23
medium	59.5%	72.3%	65.3%	34	34	13	16
low	67.6%	53.8%	60.0%	21	11	18	2

(b) Metrici pe nivele de severitate

Tabela 4.3: Rezultatele pentru prompt-ul detaliat: metrici per proiect și pe nivele de severitate

Prompt	Proiect	Precizie	Recunoaștere	F1	Probleme noi	PRs
Simplu	cal.com	18.2%	83.9%	30.0%	1	10
Simplu	discourse-graphite	24.4%	100.0%	39.2%	2	10
Simplu	grafana	16.2%	90.0%	27.5%	0	9
Simplu	keycloak	25.8%	95.5%	40.7%	2	9
Simplu	keycloak-greptile	33.3%	100.0%	50.0%	0	1
Simplu	sentry	27.8%	73.7%	40.3%	1	6
Simplu	sentry-greptile	15.1%	83.3%	25.5%	1	4
Simplu	TOTAL	20.9%	88.8%	33.8%	7	49

(a) Metrici per proiect

Severitate	Precizie	Recunoaștere	F1	TP	FP	FN	Probleme noi
critical	17.3%	100.0%	29.5%	9	43	0	0
high	25.0%	95.0%	39.6%	38	120	2	2
medium	22.6%	89.4%	36.1%	42	161	5	5
low	16.3%	78.9%	27.0%	30	154	8	0

(b) Metrici pe nivele de severitate

Tabela 4.4: Rezultatele pentru prompt-ul simplu: metrici per proiect și pe nivele de severitate

Analizând datele din tabelele 4.3 și 4.4, putem observa un rezultat neașteptat. Având în vedere tehnica "Chain of Thought" [9], ne-am fi așteptat ca un agent căruia i se spune pas cu pas cum trebuie să opereze să aibă performanțe mai bune decât unul care este lăsat liber.

Totuși, uitându-ne mai atent la datele de tip "False Positive", putem intui care este cauza acestor rezultate. Din cauza restricțiilor impuse de prompt, modelul este reticent în a spune sau cerceta probleme din afara ariei pe care o acoperă prompt-ul pe sistem. Acesta încearcă să maximizeze precizia și scorul F1, astfel face review-uri despre care este sigur ca sunt corecte, având astfel un scor F1 mare și o precizie buna, dar o recunoaștere mai mica decât prompt-ul simplu. Numeric, acest tip de prompt a produs în total 70 de rezultate fals pozitive.

Pe de alta parte, folosind prompt-ul simplu, agentul a fost lăsat să scrie orice problemă, verificata sau nu. Acesta a scris orice pseudo-probleme (reală sau nu) care ar fi putut fi posibilă. Astfel, semnalând mai multe probleme (fictive sau nu) a reușit să acopere o suprafață mai mare de probleme, iar probabilitatea ca o problemă reală să se afle prin cele sugerate crește. Acest lucru este clar uitându-ne la secțiunea de "False Positives" unde, în loc de 70 de review-uri ca și în cazul prompt-ului detaliat, aici avem în total 478 de comentarii marcate ca fals pozitive.

Acest studiu reflecta o problemă dominantă în lumea IA: alegerea între precizie și recunoaștere. Cum realizăm un agent care are o recunoaștere perfectă și identifică corect toate instanțele ce trebuiesc recunoscute fără a returna rezultate nedorite. În domenii sensibile precum cel medical sau militar, acest subiect este foarte dezbătut, deoarece în general dacă încercăm să mărim recunoașterea, precizia va scădea. O alternativă este folosirea scorului F1, dar nu ajută când doar putem modifica doar prompt-ul.

În experimentul de față, se ridică aceeași problemă: preferăm un agent care găsește toate problemele critice dintr-un sistem, dar acest detaliu este ascuns printr-o sumedenie de raportări de severitate mica sau false, ajungând să fie ignorat de ingineri? Sau ne bazăm ca inginerul este judecătorul final și ca agentul este doar o unealta, în final tot inginerul ajungând să facă un review complet, bazându-se cat mai puțin pe IA din cauza ca poate rata probleme majore?

Răspunsul este cel mai probabil, undeva la mijloc. Proiectantul sistemului agentic trebuie să găsească un echilibru bun între precizie și recunoaștere astfel încât produsul să aibă sens.

4.3 Contextul în mediul agentic și compactare. Consecințele metodelor de compactare

Compactarea este un procedeu necesar, după cum am menționat și în capitolele anterioare. Dar, în practica, acesta nu este un procedeu perfect și are un număr de consecințe, în funcție de metoda, sau setul de metode de compactare alese. Din utilizarea soluției agentice dezvoltate cu mai multe combinații de metode de compactare, am extras un set de repercusiuni ale folosirii acestor metode, pe categorii, dar și un set de metode recomandat pentru performanțe optime în raport cu integritatea informațiilor din fereastra de context.

Cea mai utila clasificare în funcție de momentul în care rulează compactarea (imediat după apelul uneltelor sau când depășește o limita), ci pe natura pierderii pe care o introduc în context. Stadiile care taie literal blocuri din rezultatele uneltelor (`tool_truncate` și `ref_substitute`) pierd date întregi, dar lasă o urma vizibilă în conversație: un marker de trunchiere sau o referință `@artifact:<sha>`. Stadiile care parafrazează prin LLM (`tool_summarize` pe rezultatele individuale ale uneltelor, `bulk` cu strategie `summarize` sau `hybrid` pe întregul istoric) păstrează întinderea conversației, dar pierd nuanțe fără nicio urma vizibilă pentru agent. Toate trei stadiile care rescriu rezultatele uneltelor (`tool_truncate`, `tool_summarize`, `ref_substitute`) operează asupra aceluiași slot proiectat și sunt declarate mutual exclusive în pipeline-ul `vibe-cli`; oricare configurație validă conține cel mult unul dintre ele.

4.3.1 Stadii care taie date din context

Spre exemplu, `tool_truncate` păstrează un cap și o coadă peste `max_tool_tokens` (implicit $800 + 400$ la prag de 2000) și externalizează restul în artefacte externe. `ref_substitute` merge mai departe: lasă doar un extras de `excerpt_tokens` și o referință. În ambele cazuri, în sesiunile testate, agentul ajunge în scurt timp să **reapeleze unealta cu argumente mai înguste** (`grep` după un identificator, `read_file` pe un interval mai mic) pentru a recupera fragmentul dispărut, în loc să cheme `fetch_artifact`. Costul real al acestor stadii nu este, deci, pierderea de informație care este recuperabilă, ci tura suplimentară pe care agentul o consumă cautându-o, vizibilă în creșterea numărului de apeluri de unealta per sarcină.

4.3.2 Stadii care parafrazează date din context

În contrast, `tool_summarize` este, în cazul de succes, strict superior trunchierii pure: rulează în două faze (o trunchiere sincronă, apoi o sumarizare LLM asincronă care înlocuiește forma trunchiată când termină), iar în caz de eșec rămâne la forma trunchiată. `bulk` face același tip de transformare la nivelul întregului istoric. Diferența față de stadiile care taie este că **pierderea nu lasă urma**: rezultatul are aceeași formă textuală, dar afirmații cantitative (numere de linii, nume de fișiere, identificatori) pot dispărea silențios. Efectul practic observat este pierderea de *recall*: agentul nu mai cere date pe care nu le mai are, pentru că rezumatul îi spune că le-a văzut deja. Când modelul de compactare este unul mai mic decât cel principal (configurabil via `preferences.compaction_model`), același mecanism produce ocazional afirmații incorecte în rezumat, care apoi se propaga ca premisa în turele următoare.

4.3.3 Stadii auxiliare

Restul stadiilor operează cu alte principii și pot însoți orice combinație de mai sus. `format_compress` reformatează JSON-ul fără apel LLM și fără pierdere semantică: în practica nu am observat efecte negative și poate fi ținut activat necondiționat. `dedupe` elimină apeluri identice și rezultate similare peste un prag (implicit 0,92); pragurile sub 0,90 au șters în sesiunile

noastre rezultate de teste cu output similar dar semnificație diferită, ceea ce a mascat regresii reale. `importance_prune` este robust cu setările implicite: pierderea evenimentelor `assistant_thinking` reduce vizibil contextul fără consecințe observabile, iar păstrarea doar a ultimului `tool_result` per tura funcționează pentru ca rezultatele intermediare sunt subsumate de cel final.

4.3.4 Secvența de stadii de compactare recomandată

Configurația care a oferit cel mai bun compromis între reamintire și ocupare a ferestrei de context, în experimentele rulate, are următoarea formă:

1. `format_compress` activat aduce câștig de spațiu garantat, pierdere zero;
2. `tool_summarize` ca singur reprezentant al grupei mutual exclusive de rescriere a rezultatelor uneltelor (`tool_truncate` rămâne fallback-ul sau sincron, deci este implicit prezent);
3. `dedupe` cu pragul implicit 0,92, poziționat după `format_compress` pentru ca normalizarea formătărilor crește rata de detecție;
4. `importance_prune` cu configurarea implicită;
5. `bulk` cu strategia `hybrid` și `min_turns_kept = 4`, suficient pentru a acoperi o iterație completă cerere-investigare-răspuns.

Pentru sarcini scurte de tip întrebare și răspuns un pipeline complet dezactivat este perfect funcțional. Pentru sarcini cu citiri exhaustive de cod, înlocuirea `tool_summarize` cu `ref_substitute` reduce semnificativ ocuparea, cu condiția ca promptul de sistem să instruiască explicit agentul când să apeleze `fetch_artifact`. În modul `tool_persistence: true`, în care întregul istoric al uneltelor trăiește între interacțiuni, prezenta a cel puțin unui stadiu ce rulează imediat devine obligatorie: în lipsa lui, fereastra se poate umple într-un număr relativ mic de ture cu volum normal de unelte, înainte ca pragul `threshold` să apuce să declanșeze `bulk`.

4.4 Interacțiunea dintre modele, context și unelte. Modele optime pentru sarcini de programare

Nu toate modelele au capabilități similare când vine vorba despre sarcini de dezvoltare software. Pe lângă necesitatea funcționalităților de gândire rațională adâncă, acestea trebuie să fie antrenate pentru a apela funcții în mod eficient și pentru a respecta dogme de programare precum SDD sau TDD.

Pentru a cuantiza exact cât de importantă este alegerea unui tip de model pentru sarcini de dezvoltare software, am încercat rularea programului cu mai multe tipuri de modele, cu capabilități diferite și mărime (ca număr de parametri) diferită.

Modelele folosite, împreună cu caracteristicile lor relevante, sunt prezentate în tabelul 4.5. Selecția acoperă atât modele accesate prin API (familiile Claude și Gemini), cât și modele open-weight rulate local prin `ollama`, într-o plajă largă de dimensiuni (de la 1.5 miliarde la 32 miliarde de parametri) și specializări (uz general, raționament, generare de cod).

Model	Tip	Parametrii	Mărime pe disc	Multi-modal
Claude Opus 4.7	LLM raționament	nepublicat	nepublicat	da
Claude Sonnet 4.6	LLM uz general	nepublicat	nepublicat	da
Claude Haiku 4.5	LLM compact, uz general	nepublicat	nepublicat	da
Gemini 2.5 Flash	LLM uz general	nepublicat	nepublicat	da
deepseek-r1:32b	LLM, raționament	32 B	19 GB	nu
gemma4:26b	LLM, uz general	26 B	17 GB	da
phi4:14b	LLM, uz general	14 B	9.1 GB	nu
llama3.2:3b	Small Language Model, uz general	3 B	2.0 GB	nu
devstral-small-2	LLM, generare cod	24 B	15 GB	nu
qwen3-coder:30b	LLM, generare cod	30 B	18 GB	nu
qwen2.5-coder:7b	LLM, generare cod	7 B	4.7 GB	nu
qwen2.5-coder:1.5b	Small Language Model, generare cod	1.5 B	986 MB	nu

Tabela 4.5: Modelele testate: tip, număr de parametrii, mărime pe disc și suport multi-modal.

Pentru rezultate comparabile, toate modelele au avut de realizat aceleași operații. Testul a constatat din 6 sarcini de intensitate diferită, executate pe un proiect real. Fiecare etapă a avut propriul mod de recompensare: de la metrici de similaritate, la LLM-as-judge, la teste ascunse. Toate acestea testează fie exactitatea cu care agentul a relatat faptele, uneltele sau skill-urile pe care le-a apelat și dacă în final operațiunea trece testul. Aceste metrici sunt fie procentaje, binare (admis sau respins) sau un scor în funcție de cât de mult a fost efectuată rezolvarea și logica cu care modelul a gândit.

Configurația testului împreună cu sarcinile pe care agenții le-au avut de rezolvat pot fi consultate aici

4.4.1 Rezultate

Rezultatele agregate ale rulărilor sunt prezentate în tabelul 4.6, sortate după scorul mediu obținut pe ansamblul cazurilor de test.

#	Furnizor	Model	Param. (Mld.)	Scor	Durata	Cost (USD)	Tokeni totali
1	anthropic	claude-opus-4-7	—	0.9062	7 min 27 s	13.40	803 585
2	anthropic	claude-sonnet-4-6	—	0.8743	10 min 4 s	0.06	17 606
3	anthropic	claude-haiku-4-5	—	0.8081	7 min 0 s	0.02	17 361
4	gemini	gemini-2.5-flash	—	0.7871	3 min 6 s	0.16	400 841
5	ollama	devstral-small-2	24	0.7051	14 min 22 s	0.00	14 620
6	ollama	gemma4	26	0.6803	9 min 49 s	0.00	6 241
7	gemini	gemini-2.5-pro	—	0.6431	4 min 14 s	0.67	435 404
8	ollama	qwen3-coder	30	0.6256	4 min 33 s	0.00	13 926
9	ollama	gemma4	8	0.3803	3 min 26 s	0.00	6 852
10	ollama	qwen3.5	9.7	0.2963	7 min 28 s	0.00	15 031
11	ollama	deepseek-r1	32	0.0747	4 min 15 s	0.00	1 484
12	ollama	llama3.2:	3	0.0590	35 s	0.00	5 380
13	ollama	qwen2.5-coder	7	0.0442	32 s	0.00	834
14	ollama	qwen2.5-coder	1.5	0.0298	21 s	0.00	828
15	ollama	phi4	14	0.0000	Eroare	0.00	—
16	gemini	gemini-2.0-flash	—	0.0000	Eroare	0.00	—

Tabela 4.6: Clasamentul modelelor după scor mediu, durata totală de rulare și cost. Modelele rulate local prin ollama au cost zero. Modelele proprietare nu au datele referitoare la mărimea modelului publice

4.4.1.1 Analiza rezultatelor

Interpretând rezultatele obținute putem face mai multe observații:

1. Numărul de parametri nu este întotdeauna direct proporțional cu capabilitățile. O bună parte din capacitatea de rezolvare a problemelor în dezvoltarea software este extrasă din specializarea modelelor pe sarcini de acest tip. De aceea modelele care au fost pregătite pentru așa ceva au performanțe mai bune în ciuda numărului de parametri. Un viitor experiment ar fi încercarea modelelor de la Anthropic care aici au dominat tabela pe alte domenii de activitate;
2. Putem vedea influența competențelor de gândire adâncă asupra numărului de tokeni folosiți (și automat a costului). Prin tehnica "Chain of Thought" [9] aceste modele reușesc să își formeze o logică superioară care ajută la rezolvarea problemelor complexe. Dar, acest procedeu vine cu un cost. Iar în cazul Opus 4.7, acest cost este mare. Pentru 6 sarcini de programare acesta a reușit să producă prin gândire și cercetarea fișierelor 800.000 de tokeni, în valoare de aproximativ 13 dolari. Un cost enorm comparat cu celelalte modele, chiar și de la Anthropic;
3. Modelele prea mici fie nu suportă apelarea de unelte din cauza depășirii ferestrei de context, fie nu sunt antrenate pentru așa ceva și pur și simplu întorc o eroare. Gemini 2.0 Flash și Phi4 pur și simplu întorc erori care denotă incompatibilitatea cu sarcini agentice. Restul modelelor cu scoruri sub 10%, pur și simplu au halucinat rezolvarea problemei și nu au apelat nici o unealtă cu adevărat. Unele dintre ele, chiar dacă ar fi reușit să apeleze uneltele, nu ar fi putut să ducă o conversație prea complexă din cauza ferestrei mici de context;
4. Modelele locale necesită hardware performant pentru a rezolva probleme reale. Deși rezultatele sunt bune și comparabile cu cele ale modelelor de top, la o fracțiune din numărul de parametri, timpul de execuție încă crează o experiență a utilizatorului neoptimală pe sisteme neperformante;
5. Variante ale modelelor antrenate special pentru sarcini de dezvoltare software arată o diferență semnificativă în performanță, precum Qwen3.5 și Qwen3-Coder.

Anomalii în rezultate:

1. Modele care dețin în teorie capabilități de gândire (Sonnet, DeepSeek R1) nu au exercitat în timpul testului aceste capabilități la potențialul maxim, ceea ce sugerează setări implicite de efort ale modelului reduse;
2. Modelul Haiku a avut o performanță surprinzătoare având în vedere că acesta este considerat cel mai slab din flota Anthropic. Acest lucru se datorează în mare măsură discrepantei de concurență din cadrul testului. Nu am avut acces la modele de aceeași generație OpenAI sau Gemini cu care să putem face o comparație corectă, iar experimentul urmarea în principal relația dintre cost și performanță;
3. Modele cu mai puțini parametri au avut un scor mult mai mic decât modele cu număr mai mic de parametri. Spre exemplu DeepSeek R1 nu a fost antrenat pentru apelarea uneltelor și deși acesta știa teoretic cum ar rezolva problema, nu a apelat nici o unealtă. Pe scurt, pur și simplu a fabricat o narațiune, o poveste despre cum ar rezolva el problema, iar la final a declarat-o rezolvată.

Recomandări extrase din acest experiment:

1. Pentru rularea modelelor locale, este necesar un hardware bun. Timpul de inferență poate fi mai crescut decât la modelele furnizorilor, sau mai mic pentru unele modele;
2. Costul modelelor crește odată cu capabilitățile de gândire și exponențial cu mărimea contextului;
3. În general, modelele prea mici (mai mici de 20 de miliarde de parametrii) nu reușesc să se descurce în sarcini de programare. Pot răspunde la întrebări punctuale, dar nu pot lucra într-un mediu agentic. Risc mare de halucinații;
4. Modelele neantrenate pe sarcini agentice (apel de unelte, gândire) nu au performanțe în mediul agentic pentru dezvoltarea software. Risc mare de halucinații;
5. Modelele mai mici ale furnizorilor de modele de limbaj sunt mai avantajoase din punct de vedere al raportului calitate-preț;
6. Gândirea crește și ea timpul de inferență. O comparație buna ar fi între Devstral Small 2 și Gemma4:26b. Deși Devstral este un model mai mic, din numărul de tokeni utilizați putem deduce ca a avut o gândire mai adâncă ce a dus la timpul ridicat. În același timp, Gemma a avut rezultate similare cu un timp mai scăzut de inferență (deși Gemma are și o arhitectura diferită, mai performantă);
7. Modele open-source recomandate: Gemma4 pentru echilibrul dintre scor și performanță, Devstral pentru scor, cu dezavantajul timpului mare de inferență și Qwen3-Coder pentru timpul foarte scurt de inferență și scor bun;
8. Din punctul de vedere al modelelor proprietare, Sonnet și Haiku au avut performanțe foarte bune, dar și Gemini 2.5 Flash a surprins cu performanțele sale (având în vedere ca nu mai este de ultima generație în familia Gemini);
9. Folosirea limbii engleza pentru discuțiile cu modelele de limbaj. Acestea, în special cele de mărimi mici sunt antrenate preponderent folosind limba engleza. Performanța acestora poate scădea folosind limbi diferite;
10. Similar, pentru limbajul de programare, unele modele vor avea performanțe mai ridicate pe anumite limbaje (precum Python) și mai slabe în sarcini pe alte limbaje, în funcție de datele de antrenament.

4.4.1.2 Modelele open-source: evaluarea valorii și a potențialului de utilizare

La finalul analizei rezultatelor, putem sumariza performanța modelelor open-source într-un singur cuvânt: surprinzătoare și într-un sens negativ, dar și într-unul pozitiv.

A fost neașteptat ca 6 din cele 16 modele să aibă scor aproximativ 0 în cadrul experimentului, dar nu imposibil. Era de așteptat ca modelele sub 10 miliarde de parametrii să întâmpine probleme, motiv pentru care le-am și ales. Dar cele între 10 și 20 de miliarde, dar și DeepSeek R1 au surprins prin neajunsurile lor.

Pe de altă parte am fost foarte plăcut impresionat de rezultatele Devstral Small 2, Qwen3-Coder și Gemma 4. Acestea au un potențial imens, mai ales cele doua din urma care oferă un timp de inferență foarte bun. Gemma4 este interesant și deoarece are o arhitectura specială de tipul Mixture Of Experts.

În final, consider ca folosirea modelelor open-source locale pentru sarcini de dezvoltare software (considerând deținerea unui sistem performant) este motivată de costurile inexistente, controlul asupra datelor, timpul scurt de inferență dar și performanțele bune (care pe măsură ce se dezvoltă vor crește).

Capitolul 5

DEZVOLTĂRI VIITOARE

Finalizând analiza post-implementare, împreună cu experimentele rulate, putem extrage un set de potențiale îmbunătățiri pe care le putem aduce aplicației în viitor.

5.1 Automatizări, integrări cu alte programe

O zona potențială de dezvoltare este suportul pentru rularea de sarcini agentice la un interval de timp, sau la un moment în timp predefinit. Astfel de implementări sunt deja regăsite în unelte precum Claude Code, prin comanda `/loop`. Aceasta funcționalitate permite rularea unui prompt automat la un moment determinat. Un exemplu bun pentru aceasta utilizare în domeniul dezvoltării software ar fi un prompt de tipul:

```
Realizează un rezumat al celor mai importante issue-uri sau PR-uri apărute  
în ultimele 24 de ore pe GitHub, și analizează canalele de Slack, Discord,  
Teams la care ai acces. Fa-mi un rezumat complet cu ceea ce găsești.
```

Acest prompt ar fi foarte util pentru ingineri pentru a fi la curent cu starea proiectelor la care lucrează și la deciziile ce se întâmplă cat timp alți colegi lucrează în alte fusuri orare.

5.2 Integrarea cu mediile de dezvoltare

Deși interfața de tip CLI este foarte versatila, compatibilitatea cu mediile de dezvoltare aduce o granularitate mai mare asupra interacțiunii cu agentul. Acesta poate sugera schimbări restrânse (linie cu linie), pe care utilizatorul le poate accepta sau rejecta individual.

5.3 Suport nativ pentru Windows

Sistemul de operare Windows nu este la fel de prietenos pentru dezvoltatorii software precum macOS sau Linux din cauza terminalului (și a comenzilor) diferite și a structurii fișierelor, motiv pentru care implementarea inițială are suport doar pentru cele două sisteme de operare listate anterior.

Acestea fiind spuse, o buna adiție viitoare ar fi suportul nativ pentru Windows, în ciuda diferențelor dintre sistemele de operare, pentru a deschide calea către o nouă audiență de utilizatori.

5.4 Construirea unui graf de cunoștințe pentru optimizarea înțelegerii proiectelor

O metoda inovativa care are potențialul de a îmbunătăți contextul unui agent din punctul de vedere al fabricării legăturilor între componentele unui proiect, dar mai ales din punctul

de vedere al numărului de tokeni de intrare și al reducerii apelului de unelte este aceea de a construi grafuri ale legăturilor între diverse părți ale proiectului.

Una dintre soluțiile pentru implementarea acestei facilități este utilizarea Graphify. Acest program construiește grafurile necesare înțelegerii proiectului și folosește modele de limbaj pentru a detalia grafurile din punct de vedere semantic. Acest lucru reduce numărul de apeluri la unelte pentru citirea fișierelor, îmbunătățind astfel utilizarea contextului și sporind cunoașterea amănunțită a proiectul pe care se lucrează.

Integrarea este una simplă, graful se construiește o dată (și ulterior automat dacă apar schimbări în cod) iar aplicațiile agentice precum Claude Code, pot folosi integrarea oficială sau pot folosi un server MCP pentru a interacționa cu acesta.

5.5 Suport pentru metode de compactare și sandboxing avansate și SDK pentru agenți

Tehnicile avansate de compactare bazate pe similaritatea semantică a frazelor ce urmează a fi șters din cauza criteriilor de similaritate sau din cauza categorisirii ca fiind neimportante sunt o adădire bună la setul de metode deja existente. Aceste tehnici pot cimentă consistența acestora dacă sunt considerate informații vitale, excluzându-le de la metodele de compactare uzuale.

Din punctul de vedere al securității, containerele Docker nu sunt impenetrabile. O îmbunătățire ce poate fi adusă în zona de sandboxing ar fi introducerea suportului pentru platforme cloud specializate în acest domeniu precum E2B.

O ultimă schimbare care ar ușura munca potențialilor contributory la dezvoltarea programului ar fi dezvoltarea unui SDK prin care aceștia ar putea apela cu ușurință metodele programului aducând astfel un plus de valoare.

Capitolul 6

CONCLUZII

În cadrul acestei lucrări am reușit să atingem obiectivele pe care ni le-am impus la început. Am dezvoltat o aplicație competitivă, ce își propune să rivalizeze cu soluțiile existente pe piață prin costurile reduse de operare, transparența și personalizare. De asemenea, pe lângă echivalarea capabilităților aplicațiilor din industrie și costurile reduse, am reușit să aducem transparența în utilizarea modelelor de limbaj pentru utilizatori ce doresc să studieze cum funcționează acestea în domeniul dezvoltării software în vederea optimizării și îmbunătățirii sistemelor agentice sau modului în care folosim astfel de aplicații și să extragem un set de recomandări în vederea utilizării aplicațiilor agentice în scopul dezvoltării software formulate în urma experimentelor pe care le-am condus.

În vederea implementării, lucrarea a presupus înainte de toate o planificare exhaustivă, care a început cu motivarea deciziei de a alege aceasta temă, cercetarea lucrărilor de specialitate din domeniu și un studiu al pieței și al funcționalităților necesare pentru a fi un produs util și competitiv. Motivarea a fost una simplă: arhitecturile agentice devin din ce în ce mai prezente în viața noastră de zi cu zi, investițiile confirmă faptul că aceasta abordare are un suport puternic, iar domeniul dezvoltării software este cel care poate beneficia cel mai mult de pe urma ajutorului pe care aceste platforme îl pot oferi. Din punctul de vedere al cercetării lucrărilor din domeniu, nici acest lucru nu a reprezentat un impediment datorită numărului mare de studii și dezvoltări în aceasta zonă care crește de pe o zi pe alta. În final, pentru studiul pieței am reușit să sondez majoritatea soluțiilor deja existente, folosindu-le în sarcinile de zi cu zi, investigând puncte de îmbunătățire. În ciuda diversității și a abilităților avansate, tuturor acestora le lipsea cu desăvârșire compatibilitatea cu o plajă largă de modele, transparența și observabilitatea, dar și opțiunea de a configura și personaliza în profunzime sistemul.

Trecând la partea practică a lucrării, am întâmpinat un număr de obstacole. Deși planul era bine pus la punct, am învățat că rareori acesta rămâne neschimbat pe parcursul implementării. De la decizia de a schimba framework-ul pentru interfața cu utilizatorul, până la alegerea deciziilor de implementare în partea de date de intrare multi-modale, am observat că procesul de dezvoltare software este unul dinamic, care se adaptează continuu pe măsură ce produsul se maturizează.

Finalizând primul prototip al aplicației, am continuat procesul de dezvoltare prin îmbunătățiri aduse sistemului. De la lucruri mici, cum ar fi îmbunătățiri de performanță la încărcarea și testarea modelelor, până la implementarea datelor de intrare multi-modale.

După mai multe runde de îmbunătățiri, am constatat concretizarea proiectului într-un produs consistent. Au urmat apoi etapa elaborării procesului de instalare și finalizarea proiectului dar și scrierea documentației.

În final, având un rezultat complet, studiul utilizării acestuia putea începe pentru a putea formula concluzii cu privire la eficacitatea acestuia în contextul actual.

Cu datele din aceste studii, am reușit să extrag o serie de recomandări pentru utilizarea eficientă a aplicațiilor agentice în domeniul dezvoltării software.

Însumând toate acestea, în final, prin soluția prezentată am demonstrat valoarea arhitecturilor agentice în domeniul dezvoltării software, care promite să schimbe complet peisajul acestui domeniu.

BIBLIOGRAFIE

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.
- [2] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. A survey of large language models.
- [3] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer.
- [4] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding.
- [5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. version: 1.
- [6] Ruixiang Zhang, Richard He Bai, Huangjie Zheng, Navdeep Jaitly, Ronan Collobert, and Yizhe Zhang. Embarrassingly simple self-distillation improves code generation.
- [7] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. 36:10088–10115.
- [8] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744. Curran Associates, Inc.
- [9] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. 35:24824–24837.
- [10] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 11809–11822. Curran Associates, Inc.
- [11] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models.
- [12] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. ReWOO: Decoupling reasoning from observations for efficient augmented language models.

- [13] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. 12:157–173.
- [14] Kiran Vodrahalli, Santiago Ontanon, Nilesch Tripuraneni, Kelvin Xu, Sanil Jain, Rakesh Shivanna, Jeffrey Hui, Nishanth Dikkala, Mehran Kazemi, Bahare Fatemi, Rohan Anil, Ethan Dyer, Siamak Shakeri, Roopali Vij, Harsh Mehta, Vinay Ramasesh, Quoc Le, Ed Chi, Yifeng Lu, Orhan Firat, Angeliki Lazaridou, Jean-Baptiste Lespiau, Nithya Attaluri, and Kate Olszewska. Michelangelo: Long context evaluations beyond haystacks via latent structure queries. version: 2.
- [15] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. SWE-bench pro: Can AI agents solve long-horizon software engineering tasks?
- [16] F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain.
- [17] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. 86(11):2278–2324.
- [18] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. 9:1735–1780.
- [19] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners.
- [20] Yichun Xu, Navjot K. Khaira, and Tejinder Singh. KV cache optimization strategies for scalable and efficient LLM inference. version: 1.
- [21] Weian Mao, Xi Lin, Wei Huang, Yuxin Xie, Tianfu Fu, Bohan Zhuang, Song Han, and Yukang Chen. TriAttention: Efficient long reasoning with trigonometric KV compression.
- [22] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding.
- [23] Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. TurboQuant: Online vector quantization with near-optimal distortion rate.
- [24] Zhongmou He, Yee Man Choi, Kexun Zhang, Ivan Bercovich, Jiabao Ji, Junting Zhou, Dejia Xu, Aidan Zhang, Yixiao Zeng, and Lei Li. HARDTESTGEN: A high-quality RL verifier generation pipeline for LLM algorithmic coding.
- [25] Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, HyoJung Han, Sevien Schulhoff, Pranav Sandeep Dulepet, Saurav Vidyadhara, Dayeon Ki, Sweta Agrawal, Chau Pham, Gerson Kroiz, Feileen Li, Hudson Tao, Ashay Srivastava, Hevander Da Costa, Saloni Gupta, Megan L. Rogers, Inna Goncearenco, Giuseppe Sarli, Igor Galynker, Denis Peskoff, Marine Carpuat, Jules

- White, Shyamal Anadkat, Alexander Hoyle, and Philip Resnik. The prompt report: A systematic survey of prompt engineering techniques.
- [26] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan A, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into state-of-the-art pipelines.
 - [27] Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2609–2634. Association for Computational Linguistics.
 - [28] Dongfang Li, Zixuan Liu, Gang Lin, Baotian Hu, and Min Zhang. LycheeCluster: Efficient long-context inference with structure-aware chunking and hierarchical KV indexing.
 - [29] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc.
 - [30] Asif Shahriar, Md Nafiu Rahman, Sadif Ahmed, Farig Sadeque, and Md Rizwan Parvez. A survey on agentic security: Applications, threats and defenses.
 - [31] William Hackett, Lewis Birch, Stefan Trawicki, Neeraj Suri, and Peter Garraghan. Bypassing LLM guardrails: An empirical analysis of evasion attacks against prompt injection and jailbreak detection systems. In Leon Derczynski, Jekaterina Novikova, and Muhao Chen, editors, *Proceedings of the The First Workshop on LLM Security (LLMSEC)*, pages 101–114. Association for Computational Linguistics.
 - [32] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosuite, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemi Mercado, Nova DasSarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. Constitutional AI: Harmlessness from AI feedback.
 - [33] Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails. In Yansong Feng and Els Lefever, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 431–445. Association for Computational Linguistics.
 - [34] Yue Liu, Hongcheng Gao, Shengfang Zhai, Yufei He, Jun Xia, Zhengyu Hu, Yulin Chen, Xihong Yang, Jiaheng Zhang, Stan Z. Li, Hui Xiong, and Bryan Hooi. GuardReasoner: Towards reasoning-based LLM safeguards.